
Boltrig

Requirements

1,837

REQUIREMENTS

20

AREAS

65

ROWS EXCLUDED

referent 19bcae7fa81663fe8998377c86451ba08fb16e48
branch origin/main · github.com/wlilley93/boltrig
captured 2026-08-24

What this is

Every requirement Boltrig satisfies at the pinned commit, stated once, with nothing said about how it was verified. The full corpus records a status against each row (IMPLEMENTED, IMPLEMENTED-UNTESTED, DEAD, SCAFFOLDED, SEAM) together with a `path:line` citation and its bound invariant. None of that appears here. This document answers only one question: what does the system do.

1,837 of the register's 1,902 rows are reproduced. The 65 that are not do not state requirements, and stripped of their status tag they would read as though they did. They are reproduced in full in the final chapter rather than dropped in silence.

Statements are as written in the corpus. They are descriptive present tense rather than *shall* form, and each one traces to a citation in the full specification under the same identifier.

Contents

Area 01 · The kernel dispatch chokepoint · 97	4
Area 02 · The kernel HTTP surface · 95	9
Area 03 · The permanent fleet: hierarchy, spawn, budget · 98	13
Area 04 · The runtime protocol and the Codex lane · 98	18
Area 05 · Persistence: the store layer and the migration ledger · 84	23
Area 06 · Adapters, egress policy, and MCP consumption · 98	27
Area 07 · Identity, authentication, seats and capabilities · 95	32
Area 08 · Memory planes, the Knowledge extension, and distillation · 96	37
Area 09 · Workflows, work items, skills and the YAML libraries · 92	42
Area 10 · Model catalogue, routing, and the sensitive-to-local rule · 100	47
Area 11 · Configuration: the manifest, the environment, and release mode · 62	52
Area 12 · Observability, readiness, doctor and log safety · 98	55
Area 13 · Process composition, bootstrap and the CLI surfaces · 86	60
Area 14 · Presence, devices, camera, emotion and the Familiar · 97	65
Area 15 · The worker UI, the primary surface · 58	70
Area 16 · The SDKs and the marketing site · 98	73
Area 17 · The channel gateway, messaging bridges and calls · 98	78
Area 18 · The iOS companion application · 92	82
Area 19 · Deployment topology, the release train and recovery · 98	87
Area 20 · Governance: invariants, gates, decisions and the court · 97	92
Excluded rows · 65	97

AREA 01

The kernel dispatch chokepoint

97 requirements

- 0100 Every external action reaches an adapter only through `Dispatcher._execute_adapter`, the single `adapter.execute` call site in the kernel.
- 0101 `Kernel.invoke` delegates without policy to the dispatcher, and the kernel composes rather than implements policy.
- 0102 The executed order is resolve, grant check, validate params, idempotency claim, approval verdict, gate/rate limit, execute, validate output, complete, audit.
- 0103 The grant check runs BEFORE param validation, diverging from the doctrine sentence in AGENTS.md and from dispatch.py's own docstring.
- 0104 `docs/ARCHITECTURE.md` documents idempotency replay after the HITL gate and rate limit, which the code contradicts.
- 0105 An unresolvable verb, an ineligible binding, a missing binding row and an unknown target type all raise `BindingNotFound`.
- 0106 Only a name that is not a stored verb reaches the capability router, so routing can add a destination and never move an existing one.
- 0107 The capability grant is authorised after the capability is known to exist and before any destination is read.
- 0108 Two or more eligible bindings with no selecting policy raise `RouteRequired` naming the destinations, never a first-match.
- 0109 A routing policy naming a disabled or deleted binding is skipped rather than fatal.
- 0110 An unrecognised capability suffix is classed `update`, the write path that never fans out.
- 0111 A routed call is grant-checked against BOTH the capability id and the source operation id.
- 0112 A capability grant is matched unversioned so a version move cannot silently stop it matching.
- 0113 A verb is authorised only when both the tenant ceiling and the caller's grants permit it.
- 0114 An active deny beats every allow and is checked first.
- 0115 An unmatched, empty or unknown grant set denies.
- 0116 A verb id that is not a safe NFKC identifier can never match any grant pattern.
- 0117 A grant wildcard matches only on a namespace boundary, so `jira.*` never matches `jira.read`.
- 0118 `GrantSet.intersect` narrows a wildcard against a ceiling rather than dropping it, and unions denies.
- 0119 Params and output are validated through one seam so the output twin cannot be forgotten.
- 0120 Schema findings carry only `schema_path` and an allowlisted `keyword`, never an instance-derived path or a schema value.
- 0121 Schema findings are bounded at 10 errors, depth 10 and 64-character segments before the row is written.
- 0122 A verb whose registered schema is empty is not validated at all, on input or output.

- 0123 A schema failure records the schema digest so a later read is told the schema moved rather than diffed against the wrong one.
- 0124 Caller-facing schema hints are returned but never audited.
- 0125 An idempotency key is bound to tenant, actor, delegation, workspace, noun, verb and the canonical request hash.
- 0126 A completed idempotent result replays before the approval gate and the rate limit.
- 0127 A key supplied for an `idempotency_mode: disabled` verb is refused with `IdempotencyConflict`.
- 0128 Secret-shaped output is completed UNCACHEABLE without persisting bearer material.
- 0129 Any raise between the claim and completion releases the claim rather than parking it until lease expiry.
- 0130 An expired executing lease becomes UNCERTAIN and demands reconciliation instead of silently re-running.
- 0131 The sensitive-key normaliser uses zero-width regexes so an adapter-chosen key cannot cause polynomial backtracking.
- 0132 The approval verdict is the operator block list, then stored-binding capabilities, then a binding override, then the posture gate.
- 0133 The operator block list is matched against the typed name, the capability and the source operation.
- 0134 A version-pinned block-list entry is read as the capability it names.
- 0135 Blocking a capability gates the source operation it routes to, resolved from approved stored bindings.
- 0136 A binding `consequence_override` may only raise the gate; a `low` override is inert.
- 0137 The approval posture applies only to delegated agent calls on adapter bindings, never to control verbs, agent bindings or direct human calls.
- 0138 A missing or malformed stored posture falls back to `risk_based`.
- 0139 An approval is fingerprinted over tenant, noun, verb, verbatim params, the authenticated initiator and the resource snapshot.
- 0140 An approval is consumed only after an exact type, verb, timeout and constant-time fingerprint match plus a winning CAS.
- 0141 Replaying a CONSUMED approval raises a typed 409 rather than silently re-pending.
- 0142 An answered approval past its deadline can never authorise an execution; the gate re-pends.
- 0143 An answer arriving after `timeout_at` expires the request and is refused with a typed 409.
- 0144 An escalation or clarification request cannot be laundered into authorisation on a gated verb.
- 0145 No approval is minted unless a redeeming lane can be named from the record; on refusal nothing is created.
- 0146 On a leg carrying an approval id the rate limit is enforced before the gate consumes it.
- 0147 A verb with no configured `rate_limit` is not throttled at all.
- 0148 An unrecognised `RateLimit.per` silently becomes a 60-second window with no validation anywhere.

- 0149 A tenant-scoped rate limit keys on the bare tenant id, so all such verbs share one counter bucket.
- 0150 The rate-limit key uses the caller's spelling, so one binding addressed by two names gets two buckets.
- 0151 The shipped rate-limit counter is Redis when `REDIS_URL` is set and the in-memory counter otherwise.
- 0152 The rate-limit window is a fixed calendar window, so a configured max admits up to 2x across a boundary.
- 0153 No credential is resolved for a call that dies at the grant check, the HITL gate or the rate limit.
- 0154 The adapter credential is resolved for `on_behalf_of` when present and otherwise for `actor`.
- 0155 Conflicting configured and durable credential references raise `CredentialResolution`, except where a user-level connection deliberately overrides the org.
- 0156 A sealed per-run adapter bearer overrides the static credential only for the same run, adapter and owner, and is minted straight into the credential argument.
- 0157 A verb param carrying a run-scoped credential reference is resolved to material inside the kernel on a copy; another run, purpose or owner fails closed.
- 0158 A sealed row without an owner resolves for nobody.
- 0159 `Credential.material` is repr-suppressed and `__str__` never renders it.
- 0160 A missing adapter record is authoritative absence: the loader is unloaded and the credential binding cleared.
- 0161 A generated adapter serves only when activated and an MCP consumer only when its lifecycle row is active.
- 0162 `chat.ask_user` is intercepted in-kernel after the full gate order, creates a QUESTION HITL and always raises `PendingHuman`.
- 0163 An adapter re-registration never replaces an existing AGENT binding, so a native or re-pointed verb survives restart.
- 0164 Adapter-over-adapter binding replacement is allowed so manifest order picks the provider.
- 0165 Adapter error classes map to 404, 403, 400, 409 and 502 with a bounded reason code, and a detail-free failure maps to 502.
- 0166 An adapter `UNAVAILABLE` result degrades when the verb declares a degraded mode and otherwise fails.
- 0167 Exactly one audit row is written in the `finally` block on every outcome, allowed or denied.
- 0168 An audit append fault defers the scrubbed payload to the outbox rather than dropping it.
- 0169 When the outbox write also fails the action stands effectful and unaudited, logged as a governance incident rather than masking the caller's exception.
- 0170 The outbox janitor re-derives `seq`, `prev_hash` and `hash` at drain time so the chain stays contiguous, preserving the action `ts` and marking the late admission.
- 0171 The outbox drain backs off `attempts * 30s` capped at 10 minutes and quarantines a non-dict payload.
- 0172 Each audit row is HMAC-chained to its predecessor per tenant, serialised by a per-tenant lock and backstopped by `UNIQUE (tenant_id, seq)`.

- 0173 Exactly one HMAC key verifies each row, bounded by the seq at which a retired key was retired.
- 0174 The live audit HMAC key is read once at module import, so rotating it requires a process restart.
- 0175 Audit detail is scrubbed on both keys and values, recursing through nested dicts and lists, and user_agent is scrubbed as a chain field.
- 0176 Chain verification re-derives the entire per-tenant chain from seq 1 ascending and cannot loop on a non-advancing page.
- 0177 The Opbox-depth audit fields are folded into the hash only when non-None, so pre-enrichment rows still verify.
- 0178 A denied grant and a throttle trip are recorded on the distinct security stream at the same field depth as the audit row.
- 0179 The chokepoint's security-event call has no guard of its own, so its fail-safety depends entirely on the injected writer swallowing.
- 0180 Run events are published to this run and to the delegating parent run, and a publish failure is swallowed.
- 0181 The tool_call and tool_result frames carry a redacted input/output plus a value-free key summary, never raw params on the chat stream.
- 0182 A voice_tone frame is emitted only for an ok result carrying a well-formed tone block, otherwise no event rather than a partial one.
- 0183 An approval pause is recorded durably as a reserved held: checkpoint on the root run plus the verbatim call sealed under a distinct credential kind.
- 0184 Only an APPROVAL pause is held; a QUESTION pause seals nothing.
- 0185 A held-call record failure is not swallowed, unlike the event relay.
- 0186 A run-terminal credential sweep is skipped while any held call is still paused on that run.
- 0188 A successful consequential invocation appends a run-effect ledger row with its inverse or an honest not-undoable row.
- 0189 The inverse registry ships empty and an unregistered or raising builder yields not-undoable.
- 0190 A revert run records nothing, so an undo cannot re-enter the ledger.
- 0191 Crossing the one-to-two active-author boundary via a control. verb writes its own audit row, measured by counting rather than by a verb list.
- 0192 Cost accounting is not on the dispatch path; tokens_used and cost_micros are never set by the chokepoint.
- 0193 An unknown model price falls back to the cost tier and unattributed tokens are billed at the input rate, never free.
- 0194 A hard-stop budget reservation debits every scope or none, re-checked under lock.
- 0195 boltrig/kernel/** may not import boltrig.fleet or services, enforced by a deny-list scan that fails when the kernel root is missing.
- 0196 RecordingDispatcher delegates both attribute reads and writes so collaborators swapped by assignment reach the real dispatcher.

0199 The one-chokepoint claim (P2, US-KER-01, K-1) is cited in the code but declared in no invariant, so nothing in the gate would fail on a second execute path.

AREA 02

The kernel HTTP surface

95 requirements

- 0200 The kernel exposes exactly one FastAPI application, built by `create_app`, carrying 294 routes registered from `boltrig/kernel/`.
- 0201 `create_app` refuses to construct with no `principal_resolver` when a production signal is present.
- 0202 With no auth configured and no dev-auth flag, the selected resolver refuses every request with 401.
- 0203 Identity is taken only from a verified credential; a request body can never set tenant, subject, role, scope or grants.
- 0204 Eight reserved context keys supplied by a caller are dropped before the context is built.
- 0205 A PAT bearer resolves before the configured resolver and yields the owner's effective grants.
- 0206 A PAT naming a workspace its owner does not belong to is refused 403, never silently un-scoped.
- 0207 An invalid or expired PAT is refused 401.
- 0208 Every mutating cookie-authenticated request must echo the session-bound CSRF token in `x-boltrig-csrf`, compared in constant time.
- 0209 Bearer and PAT callers are never CSRF-gated, because the PAT branch returns before the session resolver runs.
- 0210 A session whose user must change password is clamped to a three-path allowlist.
- 0211 A session in an org that requires 2FA, with no enrolled factor, is clamped to a four-path allowlist and refused 403 elsewhere.
- 0212 The active workspace is re-authorised against current membership on every request and drops to None fail-closed.
- 0213 Request provenance (`ip_address`, `user_agent`) is stamped at the door from the request, never from a body field.
- 0214 `CF-Connecting-IP` is honoured only under `BOLTRIG_TRUST_CF_CONNECTING_IP`; `X-Forwarded-For` is never trusted.
- 0215 The RLS tenant is bound to the resolved principal's tenant before any handler runs.
- 0216 Every `BoltrigError` reaching the transport renders one envelope: `{"status", "reason"}`, with 403 alone mapped to `denied`.
- 0217 `POST /v1/invoke` refuses a body-asserted run id owned by another subject with 403 before dispatch.
- 0218 Both `run_id` and `parent_run_id` are fenced by the same predicate.
- 0219 A caller-supplied spawn depth is clamped to a non-negative integer and never raises.
- 0220 `PendingHuman` renders 202 with the HITL request id, and `DegradedMode` renders 503 with a partial output, on both `/v1/invoke` and every `dispatch_control_route` caller.
- 0221 A JSON-RPC notification at `POST /v1/mcp` is answered 202 with an empty body, never a response frame.

- 0222 `POST /v1/mcp` accepts a run token from `x-boltrig-mcp-token` or from a bearer that `is_run_token` recognises, else it resolves a full principal.
- 0223 A user-authenticated MCP call is scoped to the caller's effective grants through the same chokepoint as the site.
- 0224 An invalid MCP run token records an `MCP_AUTH_FAILURE` security event under a reserved tenant marker.
- 0225 `/v1/chat` pulls the first stream event before responding so an authorisation error renders as an envelope, not inside a half-open SSE stream.
- 0226 A mid-run chat steer is acknowledged 202 `queued`, not as SSE.
- 0227 `/v1/work` clamps the page size at the store and returns a keyset `next_cursor` only when the page came back full.
- 0228 A work item outside the caller's departments is answered 404, not 403.
- 0229 `/v1/runs/{run_id}/events` authorises with `visible_run_events` and 404s an unknown or out-of-scope run before streaming.
- 0230 The `?since` cursor on the run stream can only narrow the replay, never widen what a run exposes.
- 0231 `GET /v1/conversations/search` is registered before `GET /v1/conversations/{conversation_id}` so `search` is never captured as an id.
- 0232 Exactly three routes are reachable without authentication: `/v1/branding`, `/healthz`, `/readyz`, and they live in one file.
- 0233 `/healthz` never awaits live adapter I/O, reading a cached snapshot instead.
- 0234 `install_security` installs, in reverse-add order, `TrustedHost`, then `CORS` when configured, then security headers, then the cookie rewriter, then the body cap.
- 0235 Seven security headers are stamped on every response with `setdefault`.
- 0236 The API CSP is `default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'none'`.
- 0237 `CORS` is installed only when `BOLTRIG_CORS_ORIGINS` is non-empty, never reflects `Origin`, and never pairs `*` with credentials.
- 0238 The `CORS allow_headers` allowlist contains five entries and omits `idempotency-key` and `x-boltrig-workspace`, which kernel handlers read.
- 0239 A wildcard host allowlist under a production signal is a fatal boot error.
- 0240 A declared or streamed body over the cap is rejected 413; a non-integer `Content-Length` is rejected 400.
- 0241 The streaming body cap is applied only to `POST`, `PUT` and `PATCH`, so a chunked body on any other method is uncounted.
- 0242 Paths under `/v1/knowledge/uploads/` get a 25 MiB cap by prefix match instead of the global cap.
- 0243 Only the two session cookies are re-scoped by the `mount-path` middleware, and only under `BOLTRIG_TRUST_FORWARDED_PREFIX`.
- 0244 An unrecognised forwarded prefix is ignored, which returns the cookie to `Path=/`, a widening the code names as failing open.
- 0245 The `mount-path` cookie control is bound to no invariant id.

- 0246 HTTP-layer rate limiting exists on exactly three ingress paths: channel inbound (per channel and per sender), channel self-onboarding, and the public workflow webhook.
- 0247 No middleware applies a global per-IP or per-principal request rate limit.
- 0248 Rate-limit windows are FIXED calendar windows, so a bound admits up to twice `max` across a boundary.
- 0249 The public workflow webhook deduplicates before throttling and throttles before the trigger lookup.
- 0250 The public workflow webhook synthesises its principal from the trigger owner and intersects with the trigger's stored grant ceiling.
- 0251 The public workflow webhook reaches the dispatcher through `control.workflow.trigger`.
- 0252 Channel inbound authenticates by HMAC over the RAW wire bytes, not the parsed body.
- 0253 A channel thread ceiling can only narrow the resolved principal's grants.
- 0254 Channel inbound creates a work item by direct store write, with no dispatcher call.
- 0255 The nine channel and call gateway routes authenticate with a run token that must carry `extra.channel_gateway`.
- 0256 A minted gateway session carries an EMPTY `GrantSet`, so presenting it at `/v1/mcp` grants nothing.
- 0257 Minting a gateway session requires an authoring role and is audited.
- 0258 The eleven device-agent and camera-agent routes authenticate with a scoped device session bearer whose device id must match the path.
- 0259 Device enrollment completion authenticates with a one-shot scoped enrollment code carried in the body.
- 0260 `GET /v1/hands/commands` claims each command on return so a second poll cannot double-execute it.
- 0261 A desktop receipt for an unknown or expired command is refused 404 and is not audited.
- 0262 Every desktop receipt that lands is audited with a truncated error string.
- 0263 `dispatch_control_route` lets the `x-boltrig-approval-id` and `idempotency-key` headers override the same body fields.
- 0264 `dispatch_control_route` builds and registers a control-plane adapter at request time when the tenant has none.
- 0265 Held-call settlement after an approved control write is swallowed so cleanup cannot turn a committed write into a client error.
- 0266 A HITL approval response goes through `kernel.hitl.answer`, never the dispatcher, and a non-approval decision is wrapped as untrusted text first.
- 0267 `POST /v1/hitl/{question_id}/answer` answers only a QUESTION, never an approval, and refuses a non-owner 403 with no write.
- 0268 A SECURE question's answer is sealed inside the kernel and only its reference is recorded; even its length is withheld.
- 0269 A sole-author or development-posture approval always leaves a distinguishing audit row.
- 0270 `POST /v1/runs/{run_id}/cancel` is owner-only, writes a cooperative cancel signal, audits it, and never interrupts an in-flight step.

- 0271 `POST /v1/me/active-context` and `/v1/me/active-org` require a first-party session, re-authorise membership, and refuse with no write and no audit when it fails.
- 0272 Session writes bind the identity realm for the write and restore the caller's active tenant in a `finally`.
- 0273 A minted PAT's secret is returned exactly once in the mint response and never stored in the clear.
- 0274 PAT and session revocation routes refuse another user's row with 404, not 403.
- 0275 Memory `MUTATION` routes run the chokepoint with server-derived scopes in `trusted_extra`.
- 0276 Memory `READ` routes query the store directly under `memory_owner_scopes`, without the chokepoint.
- 0277 `POST /v1/memory/query` reads the store directly with no dispatch and no audit row.
- 0278 81 routes enforce authorship in the transport by calling `require_author(p)`, which raises `GrantMissing`.
- 0279 68 of 161 state-changing routes never reach `kernel.invoke` or `dispatch_control_route`.
- 0280 `POST /v1/ai-keys/activate` provisions an external gateway binding with no dispatch and no audit row.
- 0281 `POST /v1/me/agent` writes a personal agent with no RBAC guard and no audit row.
- 0282 `POST /v1/me/agent/invoke` spawns only on behalf of the owner and is capped by the owner's grants.
- 0283 Test-spawn and eval routes require an authoring role and run under the initiator's grants.
- 0285 `GET /v1/admin/credentials` returns credential references only, never values.
- 0286 `PUT /v1/me/approval-posture` and the three sensing writes require `actor_tier = "human"` and an interactive credential kind.
- 0287 Selecting `FULL_ACCESS` approval posture additionally requires an explicit `confirm` value.
- 0288 `DELETE /v1/trajectory/{run_id}` purges the verbatim record for anyone who can read it, leaves the audit row untouched, and writes no audit row of its own.
- 0289 The two familiar emotion routes publish inert relay frames and import nothing from the emotion add-on.
- 0290 An injected service that is absent yields a typed 503, never a 500.
- 0291 Only four routes bind a Pydantic request model; every other state-changing route takes an untyped `body: dict` or no body at all and validates by hand.
- 0292 `WEB-02`, `WEB-03`, `WEB-05`, `WEB-06`, `RES-01`, `US-HEAD-02`, `SEC-02`, `K-3` and `IAM-09` are cited in this area's source but are not declared in `tests/invariants.yaml`.
- 0293 `POST /v1/skills/{skill_id}/test-spawn` is registered twice, once plain and once with a `:path` converter that overlaps it.
- 0294 Session cookies drop to `SameSite=None` for an origin that is both a known desktop webview origin and present in `BOLTRIG_CORS_ORIGINS`.
- 0295 `/healthz` discloses tenant ids and adapter names without authentication.

AREA 03

The permanent fleet: hierarchy, spawn, budget

98 requirements

- 0300 The live serving composition is a flat roster of tier-1 named peers, not a tier1-over-tier2 hierarchy.
- 0301 `build_org` normalises a legacy `hierarchy` manifest into the flat named roster rather than re-constructing Chief/Department authority.
- 0302 With no manifest, `build_org` degrades to a single script-runtime agent addressed `general`.
- 0304 `DepartmentHead` reaches production only as the base class of `NamedAgent`.
- 0305 Every named agent's address is a lowercase slug unique within the roster, and `agents.default` must name a declared address.
- 0306 A named agent's `max_depth` is bounded 1..5 and its runtime restricted to codex, script or python-script at parse time.
- 0307 An ephemeral child holds no named-agent registry row, no mailbox address and an explicit deny on `agent.send` and `chat.present`.
- 0308 A named agent with a registry row takes each work-item turn inside a serialized, heartbeaten cross-worker turn lease.
- 0309 A permanent profile resolves its runtime per call under `pinned_policy=True` and is never a resident process.
- 0310 A spawn resolves exactly one spawn-rule policy snapshot before any spend.
- 0311 An unreadable or schema-invalid spawn-rule policy raises `SpawnRulePolicyInvalid` and never falls back to an empty rule set.
- 0312 Two spawn rules tied at the highest matched priority fail closed.
- 0313 A matched spawn rule refuses a conflicting caller `capability`, `runtime` or `cost_tier` pin rather than ignoring it.
- 0314 A matched spawn rule may add skills and tighten depth but never widen grants.
- 0315 Skills resolve parent-first through `extends` with a cycle guard, merging prompt fragments, tool grants and context schemas.
- 0316 A spawn whose context fails the merged skill schema is refused with `ContextRequirementsUnmet`.
- 0317 Capability selection considers only active profiles in the caller's workspace union the org-wide profiles.
- 0318 With no pin, the selected capability is the cheapest capable one by cost tier, ties broken by name.
- 0319 An explicit `prefer.capability` wins over cost order, and an incapable pin is refused.
- 0320 An explicit `prefer.runtime` wins over cost order and accepts `script` as an alias for `python-script`.
- 0321 Capability selection never consults the configured per-model price table.
- 0322 A spawn whose child depth would exceed `capability.max_depth`, tightened by any rule `max_depth`, raises `DepthExceeded`.

- 0323 The pump builds its execution context with depth 0, so the delegated lane never reaches a capability depth ceiling.
- 0324 `POST /v1/spawn` takes the recursion depth from the caller-supplied request body.
- 0325 A child's effective grants are the merged skill grants intersected with the parent context, and with any caller ceiling.
- 0326 The child context carries the parent's run id as `parent_run_id`, `actor_tier="ephemeral"`, and only the trusted spawn-rule receipt from the parent's extras.
- 0327 The composed prompt is handed to the routing seam as the egress payload so the PII scanner classifies it before the destination is chosen, and a detection reroutes rather than re-writes.
- 0328 An unset `sensitive_endpoint_id` keeps the router fail-closed rather than letting sensitive work escape through a standard provider, including on the spawn path.
- 0329 A caller's sealed adapter bearer is re-sealed under the child run id, best-effort, and retired at the child's terminal on both exits.
- 0330 Budget is reserved before the runtime is resolved or invoked.
- 0331 The pre-run estimate is deterministic: `max(16, chars // 4)` tokens priced at the capability's cost tier.
- 0332 Reservation scopes are the tenant plus a department scope present only when `prefer.department` is supplied.
- 0333 The multi-scope reservation debits every scope or none, in one transaction that re-checks each hard stop under lock.
- 0334 A hard-stop breach at reservation writes an `AGENT_SPAWN` audit row with `status="budget_exceeded"` before returning or raising.
- 0335 With `partial_on_budget=True` a budget-blocked spawn returns a degraded partial envelope and the run never happens.
- 0336 With `partial_on_budget=False` a budget-blocked spawn re-raises `BudgetExceeded`.
- 0337 No budget check runs during a model call; the ledger is corrected only after the run.
- 0338 The post-run true-up applies the signed (`actual - estimate`) delta to every reserved scope and returns the priced figure to the caller.
- 0339 A runtime that raises after reservation refunds the entire estimate.
- 0340 Actual usage is priced per leg when the runtime reports an input/output split, otherwise at a single rate on the total, never at zero.
- 0341 A budget scope with no policy row is unmetered rather than refused, on both stores.
- 0342 A soft budget crossing 0.8 of its cost limit fires exactly one pre-emptive alert and a raising callback never aborts the reservation.
- 0343 The pump claims at most one work item per cycle, atomically, incrementing `attempts` on the claim.
- 0344 An expired lease is reclaimable and a lease is never renewed.
- 0345 Every write to a claimed row is fenced by the backend against the lease tuple minted at claim, never against a value the body re-read.
- 0346 A refused fenced write is a no-op plus a warning, never an exception.
- 0347 A payload carrying no lease token writes unfenced and logs that it did.

- 0348 The durable Hatchet task input contract declares only `tenant_id` and `item_id`, so the lease token does not reach the body on that lane.
- 0349 The pump alternates the peer-mailbox lane and the work-item lane so neither starves the other.
- 0350 `handle_claimed_item` parks a BLOCKED item for a human rather than retrying it, but the claim predicate never returns a BLOCKED row, so the branch is reached only by a direct call or by a status change racing the body.
- 0351 A `workflow:<id>` target is honoured before any agent routing, through the durable governed path under the item's own context.
- 0352 An addressed workflow that does not exist parks the item for a human instead of falling through to agent routing.
- 0353 A bare `workflow:` prefix with no id is not a workflow target.
- 0354 On the flat lane an empty or `cos` target routes to the declared default peer and any other target routes to that exact address.
- 0355 An item addressing an unknown named agent is parked AWAITING_HUMAN with a HITL escalation and nothing is spawned.
- 0356 The routed agent is recorded as `owner_member` and is the `principal_scope` the run's context claims.
- 0357 The pump consults the cancellation marker at three boundaries and never interrupts an in-flight step.
- 0358 A cancel seen after a completed step suppresses follow-on work and workflow learning while preserving the step's own record.
- 0359 The terminal CANCELLED state is written in a `finally`, is idempotent, and emits one `work.cancel` audit row.
- 0360 A transient body failure re-queues the item to PENDING until `attempts` reaches `max_attempts` (3), then records FAILED with the error type and a redacted, truncated detail.
- 0361 A fault raised after the item already reached a settled status never re-opens it.
- 0362 A cancel that lands during a failure record wins over the failure.
- 0363 A human re-queue moves an AWAITING_HUMAN or BLOCKED item to PENDING, clears the lease and resets `attempts` to zero.
- 0364 A FAILED item cannot be re-queued through `WorkPump.requeue`.
- 0365 The human re-queue write is deliberately unfenced because it is an authorised reset.
- 0366 A step producing more than `max_children_per_step` (8) sub-tasks escalates to a human and spawns nothing.
- 0367 The whole step's fan-out is reserved against a per-tree counter atomically, so two pumps over one store cannot jointly exceed it.
- 0368 The per-tree fan-out counter is keyed by the tree ROOT work-item id, found by a cycle-safe parent walk.
- 0369 The per-tree fan-out counter has no decrement, reset or purge operation.
- 0370 A step whose children report more than `max_new_items_per_step` (16) follow-ons escalates, carrying the children on the escalation record.

- 0371 Children run bounded-parallel under a semaphore of `max_children_per_step`, and a child exception becomes a degraded failed-child record rather than raising past the join.
- 0372 Each sub-task is persisted as an `IN_FLIGHT` child work item with a `NULL` lease, which neither store's claim predicate can match.
- 0373 A budget-skipped child is recorded `DONE` with `degraded=True`, because only `status = "error"` marks a child `FAILED`.
- 0374 A cap or budget escalation files a `HITLType.ESCALATION` scoped to the escalating agent and returns a structured escalated outcome.
- 0375 An escalated outcome parks the item `AWAITING_HUMAN` against the HITL id the agent already filed.
- 0376 A convergent item whose joined aggregate is degraded parks for a human and is never `DONE`.
- 0377 A terminal item is stamped with a deterministic outcome score: 1.0 clean `DONE`, 0.5 degraded `DONE`, 0.0 `FAILED`, null for `AWAITING_HUMAN` and `CANCELLED`.
- 0378 Follow-on work items inherit the parent's principal, workspace and stamped authority ceilings and are created `PENDING` at `parent.depth + 1`.
- 0379 A follow-on whose intent already exists among the parent's children is skipped, and the check is not a concurrency fence.
- 0380 A delegated run executes at the requesting principal's authority, resolved through the same `effective_grants_for_request` the request path uses.
- 0381 An item naming no principal, or one the store cannot identify, carries `EMPTY_GRANTS` and never the tenant ceiling.
- 0382 The channel-thread ceiling and the creation-time creator ceiling are each intersected into the execution grants.
- 0383 The tenant permission ceiling is a separate axis, enforced independently at dispatch and never folded into the execution grants.
- 0384 Seating a peer adds `agent.send` to the run's allow set unconditionally, including for a system-originated item with no principal.
- 0385 Post-run reflection carries a two-verb system seat (`memory.remember`, `memory.propose`) and never the principal's or the tenant's authority.
- 0386 Reflection is off unless `BOLTRIG_REFLECT=1`, and a reflection failure never fails the run.
- 0388 Executor selection returns a durable Hatchet executor when the SDK is installed and configured, otherwise a declared non-durable local fallback, and logs which.
- 0389 With `BOLTRIG_REQUIRE_DURABLE` set, a durable-engine failure refuses to boot instead of falling back.
- 0390 Enqueuing an unregistered task name fails closed with `KeyError` on both executors.
- 0391 The durable and direct lanes run the same registered work-item body.
- 0392 A durable task body re-enters the kernel chokepoint and refuses a payload whose tenant differs from its context envelope.
- 0393 The Hatchet work-item task refuses to run when no pump is wired rather than dropping the item.
- 0394 Each spawn writes exactly one `AGENT_SPAWN` audit row carrying capability, runtime, spawn-rule receipt, filtered model route, degrade reason, latency, tokens and cost.

- 0395 The audit write happens before the observability sink, and a sink failure never loses the audit row.
- 0396 Every published `subagent` open frame is paired with a `subagent_end` settle frame for the same child run id on the same parent relay, on success and on a runtime raise.
- 0397 The pump publishes one windowed throughput receipt per window reporting processed count and whether any cycle raised, and never escalates to a stalled verdict.
- 0398 A FAILED item's recorded error detail has URLs redacted and is truncated to 200 characters.
- 0399 One fleet-worker process serves exactly one tenant id for its lifetime.

AREA 04

The runtime protocol and the Codex lane

98 requirements

- 0400 The Runtime protocol declares `runtime`, `cost_tier` and `async run(prompt, context, *, tools) → AgentResult`, and implementations are stateless per call.
- 0401 `build_runtime` maps `codex` to the trusted Codex constructor, `script/python-script/go-binary` to `ScriptRuntime`, and every other name to `UnavailableRuntime`.
- 0402 `build_runtime` accepts an `endpoint_lookup` argument for call-shape compatibility and discards it unread.
- 0403 `UnavailableRuntime` executes nothing and returns a degraded result naming the requested runtime and reason `runtime_unavailable`.
- 0404 `AgentResult.degrade` returns `ok=True` with `degraded=True` and never embeds the prompt, carrying only its sha256 and byte length.
- 0405 `degrade_reason` is bounded to 200 characters and carries a runtime tag plus an exception class name, never prompt, output or exception args.
- 0406 The user-facing reply is `output["text"]`; `summary` is the audit line and only the fallback.
- 0407 `ScriptRuntime` is deterministic, offline and non-model, reporting zero tokens and zero cost.
- 0408 No module under `boltrig/` defines or exports a provider-native, `OpenCode`, `Rivet`, `Pi` or `Hermes` runtime class.
- 0409 `BOLTRIG_ENABLE_LEGACY_RUNTIMES` is read by no code in `boltrig/`; setting it changes nothing.
- 0410 `boltrig doctor` reports a `warn` (never a `deploy block`) for a manifest that enables any of the eight retired runtime names.
- 0411 The retired-name set is exactly `pi hermes openai claude-api opencode rivet rivet_agentos rivet-agentos`.
- 0412 No retired-runtime binary, plugin or vendored client is present in the tree.
- 0413 No configuration-only path can construct a governed non-Codex, non-script agent lane.
- 0414 The `AgentRuntime` port declares exactly seven members: `start_thread`, `resume_thread`, `start_turn`, `steer_turn`, `interrupt_turn`, `events`, `close_thread`.
- 0415 `RuntimeThreadSpec` defaults to `PhaseMode.READ_ONLY` and `SandboxPolicy.READ_ONLY`.
- 0416 A turn spec carries only user input; policy is never expressed through the prompt.
- 0417 `CodexAgentRuntime.name` is `codex_app_server` and `production_ready` is the constant `False`.
- 0418 `CodexAgentRuntime` refuses construction unless `allow_test_only_runtime=True`.
- 0419 At most 64 Codex phases are active per runtime instance, and one phase never has two owners.
- 0420 `events()` never carries model output; the turn's text is read back through `read_turn_output`.
- 0421 Every `thread/start` and `thread/resume` sends `approvalPolicy: never`, `sandbox: read-only`, and `thread/start` additionally sends `ephemeral: true`.
- 0422 `Resume`, `steer` and `interrupt` bind to the exact requested thread and turn identifiers; a mismatch is a binding terminal.
- 0423 `close_thread` raises `CodexRuntimeOperationError` when cell cleanup failed.

- 0424 The event stream admits exactly one consumer at a time.
- 0425 Wire frames omit the `jsonrpc` header, use exact key sets, reject duplicate JSON keys and non-finite numbers, and are capped at 1 MiB per line.
- 0426 No protocol error message includes a request or response payload.
- 0427 `initialize` declares `capabilities.experimentalApi = false`.
- 0428 The only server-initiated request the client answers is `item/tool/requestUserInput`; every other method is refused with `-32601` without crashing the pump.
- 0429 The approve label is read from Codex's own offered options; zero or multiple candidates is refused with `-32602` and never an auto-deny.
- 0430 The approval handler inspects no verb consequence and duplicates no kernel predicate, so it cannot drift from the kernel gate.
- 0431 A server request is answered on a side task and its failure never terminates the single-reader notification pump.
- 0432 A `thread/started` notification whose `cliVersion` is not the pinned version terminates the phase.
- 0433 Twelve named notification methods invalidate quarantined evidence and terminate the phase; the sole exemption is the admitted kernel MCP server's `starting/ready` startup update.
- 0434 A token-usage report from a foreign thread is recorded as an observation and never billed to this run.
- 0435 An unrecognised notification method produces an UNKNOWN event carrying only a digest of the method name.
- 0436 The pinned Codex CLI version is `0.144.3` and exactly two reviewed Linux artifact digests are admitted, one `x86_64-musl` and one `aarch64-musl`.
- 0437 Binary verification requires a regular non-symlink executable that is not group- or world-writable, opens it `O_NOFOLLOW`, re-stats before and after hashing, and admits only a digest in the reviewed set.
- 0438 `check_codex_protocol.py` always verifies the checked-in manifest and root schema, verifies the installed binary and its regenerated 267-file bundle only when a binary path is supplied, and states which legs ran.
- 0439 `check_codex_pin_health.py` is fatal only when `BOLTRIG_CODEX_BINARY` is set, reports satisfiability otherwise, and is never fatal on version drift.
- 0440 Both release Dockerfiles declare the same version, both digests and both target triples as the runtime pin module.
- 0441 The version literal `0.144.3` also appears in `codex_runtime_config`, `skill_config`, `stage_desktop_codex` and the Tauri local agent without a mechanical binding to the pin authority.
- 0442 The protocol manifest permits only `stdio` and `private-unix-socket` transports and records `remoteWebSocketAllowed: false`.
- 0443 The trusted wall requires `BOLTRIG_CODEX_TRUSTED` under both postures.
- 0444 The trusted wall refuses any production or staging signal under both postures.
- 0445 Posture (a) is `BOLTRIG_DEV_AUTH=1` with no OIDC, Cloudflare Access or session-login ingress configured.

- 0446 Posture (b) is detected from the deployed privilege state (a live inherited spawner socket or uid-0-with-capability), never asserted by an operator flag.
- 0447 A real ingress posture without per-cell uids is refused before any admission or bearer mint.
- 0448 The wall is asserted at three seams: the composition factory's flag check, `build_trusted_codex_runtime`, and `provider.acquire`.
- 0449 `build_trusted_codex_config` returns `None` unless the trusted flag, the binary path and the stack root are all present, constructing nothing.
- 0450 `build_trusted_codex_runtime` accepts only an exact `TrustedProxyCodexPhaseCellProvider`; any other object yields a typed unavailable runtime.
- 0451 The trusted provider refuses construction unless its supervisor was built with `auth=None`, so the cell environment can never carry the upstream key.
- 0452 Composition proves the read-only sandbox actually engages on the host with three legs (unsandboxed write succeeds, sandboxed write leaves no artefact, sandboxed read succeeds) and refuses to build a provider otherwise.
- 0453 Composition proves the shared auth helper is a non-symlink regular file, foreign-owned, un-writable by this account, outside the mutable stack root, with an unwritable directory chain, and that Yama `ptrace_scope ≥ 1`.
- 0454 While `config.toml` is not protected by per-cell uids, a second concurrent cell is refused outright.
- 0455 Each cell's bearer ingress binds a fresh random abstract socket name, never a filesystem path derived from the cell id.
- 0456 A bearer is minted per attested peer connection at a strictly higher generation, and per-cell uids are re-confirmed against the kernel at the instant of issuance.
- 0457 A cell identity is registered from the slot the cell was `ALLOCATED`, never from the observed process credentials, and root is never a lawful cell identity.
- 0458 Production admission is closed by two independent constants and the shared projection reports `ready` only when both are true and no quarantined blocker remains.
- 0459 `QuarantinedCodexPreflightReceipt.production_complete` is always `False` and the receipt must carry the complete seven-blocker tuple.
- 0460 A quarantined receipt whose protocol version or schema bundle digest differs from the pin is refused.
- 0461 A runtime config receipt with any non-empty surface attestation is refused because no governed verifier exists.
- 0462 `/readyz` and `boltrig doctor --production` report the same Codex posture from one shared projection.
- 0463 The `core` release mode plus an enabled trusted Codex lane is a hard `release_mode_conflict` failure in both readiness and doctor.
- 0464 Codex runtime config is composed from a private revalidated snapshot and rejects any non-empty ambient override mapping.
- 0465 The composed config is ASCII, at most 768 KiB, and must match its receipt on digest, byte count and every re-rendered field.
- 0466 The App Server argv pins the security-critical config leaves and is derived from the receipt rather than stored beside it.

- 0467 The cell environment is exactly `CODEX_HOME`, `HOME`, `LANG`, `LC_ALL`, `PATH` plus an optional supervisor-managed auth token; additions may extend it but never override a base key, capped at eight entries of bounded printable ASCII.
- 0468 The rendered config disables 19 dynamic Codex features, with `multi_agent` the only variable one, and sets `approval_policy = "never"`, `sandbox_mode = "read-only"`, `web_search = "disabled"`, `history.persistence = "none"` and `shell_environment_policy.inherit = "none"`.
- 0469 A root-owned mode-0444 managed config on the read-only image mount pins the cell-invariant security leaves, and it is documented that naming a leaf defends that leaf but not the table it sits in.
- 0470 The kernel-tools lane is selected by a capability whose `supported_skills` contains `*`, or by an explicit `force_kernel_tools` from the permanent seam.
- 0471 The lane's tool ceiling is the tenant permission set intersected with the run grants, derived through the same `offer_candidates` the kernel MCP face uses.
- 0472 Verb ids become Codex wire names through one sanitizer, are unique, sorted, bounded to 128 names of at most 128 characters, and a malformed ceiling is refused rather than cleaned up.
- 0473 A ceiling over the 128-name bound, or an empty ceiling, falls back to the read-only phase with a logged reason and never silently truncates.
- 0474 The run-scoped kernel MCP token reaches the cell only as a child-process environment variable, never in the config file, argv, a repr or a pickle.
- 0475 The run-scoped token is revoked and the scope discarded in a `finally` whether the phase ran, failed or never started.
- 0476 The per-cell model proxy strips Codex built-in tools and unlisted verbs from the request, flattens the MCP namespace, and reattaches the namespace on in-ceiling response function calls.
- 0477 A model-call body the proxy cannot parse raises `ToolCeilingViolation` rather than being forwarded, because an unverifiable tool set is not an admitted one.
- 0478 An admitted cell's observed MCP inventory must be exactly zero servers on the read-only lane and exactly one on the kernel-tools lane.
- 0479 Both shipped Codex lanes compile a native-subagent policy whose defaults and ceiling are all zero.
- 0480 `CodexPhaseAdmission` refuses any policy that is not read-only, that declares enabled runtime tools, or whose native subagent limits are not `(0, 0, 0)`.
- 0481 The only native-collaboration tool namespace the wire gate admits is `multi_agent_v1`, with a fixed five-tool set.
- 0482 A Codex run without both a run id and a workspace id degrades with `no_read_only_phase_scope` rather than fabricating a scope.
- 0483 Any lifecycle exception in a Codex turn degrades with `codex_turn_failed:<ExceptionClass>` and never raises into the caller.
- 0484 A turn that produced no assistant text degrades as `codex_empty_output`, or `codex_empty_output_after_error` when runtime errors were observed, and still reports the tokens the turn consumed.
- 0485 A turn's tool steps are capped at 16 by default; `BOLTRIG_CODEX_MAX_TOOL_STEPS` overrides it, an explicit `0` disables the cap, and an unparsable or negative value keeps the default.

- 0486 Exceeding the tool-step budget interrupts the turn before degrading, so a runaway loop stops burning wall clock.
- 0487 Reasoning, messages and plans are not tool steps; only the seven declared tool item types count against the budget.
- 0488 A degraded turn reports what the provider was already paid, including the input/output split, so a failed turn is not refunded in full.
- 0489 A failed assignment model-binding registration degrades with `co-dex_model_binding_failed:<Class>` and the binding is discarded in `finally`.
- 0490 A permanent profile resolves its runtime with `pinned_policy=True`, and any exception yields a typed `permanent_runtime_unavailable:<Class>` degrade with the budget reservation refunded.
- 0491 A pinned permanent profile whose resolved endpoint model differs from the composed Codex model raises `PinnedRuntimePolicyUnavailable` rather than running a different model.
- 0492 The Codex execution ledger stack is off by default and constructs nothing, parking `None` on the platform object.
- 0493 `shadow_admit` records at most one execution-neutral `RootEngineDecision` per root run and swallows every failure so it can never break a live turn.
- 0494 The only rollout policy any composition constructs is generation 1 in mode OFF, and the router short-circuits to `LEGACY` before reading compatibility or workload.
- 0497 Cell stderr is classified against a seven-token allowlist of Codex-internal strings and only the stable labels are surfaced, never the surrounding text.
- 0498 The kernel MCP endpoint the tools lane points at is read from `BOLTRIG_CODEX_MCP_URL`, then `BOLTRIG_MCP_URL`, defaulting to `http://kernel:8000/v1/mcp`, and is validated before wiring.
- 0499 The read-only lane's cell id, cell root and workspace path are deterministic functions of the assignment id, so the adapter and the provisioner agree without a round trip.

AREA 05

Persistence: the store layer and the migration ledger

84 requirements

- 0500 The kernel reaches durable state only through the `Store Protocol`, never a concrete database.
- 0501 Two backends satisfy `Store`: `InMemoryStore` and `PostgresStore`, differing only in the five connection-lifecycle methods.
- 0502 Every store method is tenant-scoped: an id lookup is always qualified by `tenant_id`.
- 0503 Runtime store construction never applies `schema.sql`; `apply_schema` is `False` from every production path.
- 0504 `PostgresStore` is composed from per-domain mixins so no single module holds the whole method surface.
- 0505 The `Store Protocol` does not declare the camera-binding or camera-lease methods that routes call off `kernel.store`.
- 0506 `schema.sql` declares 138 tables and requires the `pgvector` extension before any `vector` column.
- 0507 `schema.sql` is the fresh-database bootstrap only and is mounted as a Postgres initdb hook.
- 0508 Alembic is the authoritative upgrade path and its head is `0086_conversation_addressing`.
- 0509 The packaged readiness head is graph-checked against the Alembic revision tree.
- 0510 The Alembic baseline is immutable, pinned by sha256, and never reads the mutable bootstrap.
- 0511 The revision graph forked at 0076 and is rejoined by the schema-free merge revision 0081, leaving exactly one head.
- 0512 Revisions 0001, 0022 and 0024 raise on downgrade; rollback past them is a restore, not a migration.
- 0513 The Alembic head and the `schema.sql` bootstrap produce identical table, column, constraint, index and sequence catalogues.
- 0514 The parity check does not compare row-security state or policies, so an RLS divergence between the two paths passes it.
- 0515 A data-bearing RLS database upgrades from 0021 to head with rows preserved and the fence extended.
- 0516 Alembic resolves its URL from `DATABASE_URL`, then `BOLTRIG_DATABASE_URL`, then `BOLTRIG_TEST_DATABASE_URL`, and otherwise falls back to a localhost DSN.
- 0517 Online migrations run on the `psycopg` driver because the baseline is a multi-statement bootstrap `asyncpg` cannot execute.
- 0518 The roll gate migrates each stack to the head its `TARGET IMAGE` asserts, never to the checkout's head.
- 0519 A database found ahead of its image aborts the roll non-zero and nothing is deployed for that stack.
- 0520 Five revisions (0044, 0045, 0057, 0074, 0075) use unguarded DDL and cannot run against a `schema.sql`-bootstrapped database.

- 0521 The RLS overlay is opt-in and is not part of the default `schema.sql` boot.
- 0522 Under the overlay every tenant table carries a `tenant_isolation` policy with FORCE row security, gating reads by USING and writes by WITH CHECK.
- 0523 An unset `app.tenant_id` yields zero rows and refuses every insert; the fence is fail-closed, never wide open.
- 0524 Exactly four tables sit outside the fence: `channels`, `identity_orgs`, `personal_access_tokens`, and `organisations` which is fenced on `id` instead.
- 0525 Every table carrying a `tenant_id` column is either fenced or in the documented exclusion set, measured against a live catalogue.
- 0526 The policies bind only when the connection drops to the non-bypassing `boltrig_app` role; setting the GUC alone does nothing under a superuser.
- 0527 Every public tenant-carrying coroutine binds the GUC from its own first argument, so fence and WHERE clause cannot drift.
- 0528 Every method holding its own transaction binds the tenant explicitly or is one of eight declared exemptions.
- 0529 `list_orgs` is the sole RLS-exempt read, reachable only from three named fleet janitors, and the caller set fails the build in both directions.
- 0530 `readiness_snapshot` deliberately reads outside the fence because `SELECT 1` and `alembic_version` are global catalogue facts.
- 0531 The overlay revokes UPDATE and DELETE from `audit_log`, `security_log`, `config_revisions` and `execution_events`.
- 0532 Four tables documented as immutable or append-only (`conversation_summaries`, `memory_events`, `agent_messages`, `workflow_trigger_deliveries`) keep full DML grants.
- 0533 Fourteen migrations enable FORCE row security unconditionally, so a migrated database is fenced on ~19 tables whether or not `rls.sql` was ever run.
- 0534 Every dict written through `set_credential_ref` rests as a versioned Fernet envelope; `credential_refs.data` never holds plaintext written after sealing landed.
- 0535 The sealing key is derived with scrypt over a fixed salt; the original unsalted SHA-256 derivation is retained for decrypt only.
- 0536 A missing or default `BOLTRIG_SEAL_KEY` under a production signal is fatal, and the error names the signal, never the key.
- 0537 Unsealing a tampered or wrong-key envelope fails closed with `CredentialResolution` and leaks no ciphertext.
- 0538 A legacy row without the sealed marker is returned verbatim and is re-sealed only when the row is next written.
- 0539 `has_credential_ref` answers presence without selecting or unsealing the encrypted column.
- 0540 Run-scoped secure-input credential rows are deleted by an exact `run:<run_id>` prefix match using `strpos`, not LIKE.
- 0541 Password credentials live in their own table as an argon2id PHC string, apart from the user identity row.
- 0542 Every other bearer in the identity group rests as a sha256 digest: PAT, session, recovery code, 2FA challenge, password reset, invitation.

- 0543 `user_sessions.csrf_token` is stored in plaintext, the only unhashed bearer in the identity group.
- 0544 The TOTP shared secret is never stored on the enrolment row; only a `secret_ref` into the sealed store is.
- 0545 `ai_configs` has no plaintext key column; a row carries only a `credential_ref` into the sealed store.
- 0546 An AI-key proposal expires within 15 minutes and a database CHECK ties its status to whether `secret_ref` and `consumed_at` are set.
- 0548 `audit_log` and `security_log` are separate per-tenant hash chains, each unique on `(tenant_id, seq)` with `prev_hash` chaining to `hash`.
- 0549 `audit_append` is a plain INSERT; cross-process ordering rests on the unique constraint, not on a store-side lock or retry.
- 0550 A faulted audit append defers its scrubbed payload to `audit_outbox` with no chain fields, and the janitor re-chains it at drain.
- 0551 Audit rollup anchors are per tenant and optional workspace, with `workspace_id IS NULL` selecting the org-wide stream rather than any stream.
- 0552 An idempotency claim is bound to actor, delegation, workspace, noun, verb and canonical request hash; any mismatch returns MISMATCH.
- 0553 An idempotency claim retries the insert up to three times when the conflicting row is released under it, then answers IN_PROGRESS and logs a warning.
- 0554 An idempotency release deletes a row in either `claimed` or `executing` state so a transient failure does not park the key as a permanent conflict.
- 0555 `idempotency_keys` has no retention sweep anywhere in the tree; completed rows and their cached results persist indefinitely.
- 0556 A work-item write on a claimed row succeeds only if the row still carries the lease the caller was given at claim, evaluated in the same statement.
- 0557 The lease fence makes the work-item RECORD single-writer and explicitly does not make execution exactly-once.
- 0558 `claim_work_item` is one statement using FOR UPDATE SKIP LOCKED, reclaims expired leases, and increments `attempts` per claim.
- 0559 Work-item status transitions are compare-and-set on the expected status, so a losing writer fails rather than overwriting the winner.
- 0560 `try_increment_fanout` applies the whole increment or none, and refuses an over-cap first increment up front.
- 0561 A run-cancel request is an idempotent marker row; a repeat never overwrites the original requester.
- 0562 A multi-scope budget reserve locks every scope FOR UPDATE in sorted order and debits all or none.
- 0563 Budget window identity is derived, never stored raw: a run window keys on a sha256 prefix of the run id and calendar windows key on the UTC boundary.
- 0564 Closed conversations, their messages and their derived summaries are hard-deleted past the retention cutoff in one transaction.
- 0565 The audit log is exempt from the retention purge and is never touched by it.

- 0566 The retention janitor is started by the fleet worker on `BOLTRIG_RETENTION_INTERVAL`, and a value of zero or less disables it.
- 0567 Channel delivery dedup is an atomic record-and-check, TTL-bounded, with expired markers evicted opportunistically on write rather than by a janitor.
- 0568 The channel outbox claim re-imposes oldest-first order in a CTE because Postgres defines no ordering for RETURNING.
- 0569 Artifact bytes are content-addressed and validated at write: exact `bytes`, length equal to `size`, and a constant-time digest comparison.
- 0570 The artifact row enforces a safe name, a sha256 digest, a 100 MB size ceiling matching the stored bytes, and a revision chain rooted at revision 1.
- 0571 A run effect's sequence is assigned inside the INSERT, so two concurrent recorders cannot both claim it.
- 0572 Settling a run effect is a compare-and-set on the expected status, so an inverse can never execute twice.
- 0573 The trajectory stream is a standalone store, not a mixin on `Store`, so no holder of a `Store` incidentally holds the unscrubbed prompts.
- 0574 The trajectory sequence is assigned by the database in the same statement as the insert, never read-then-incremented in Python.
- 0577 Background-job and birth-profile receipts are row-count bounded by an in-transaction advisory lock plus a prune, so concurrent boots cannot leave the cap exceeded.
- 0578 List reads are clamped server-side: work pages to 500, memory lists to 200, observability pages to 10,000, audit search offset to 10,000.
- 0579 The knowledge repository is a second holder of the RLS pool outside the `PostgresStore` MRO and carries its own tenant-binding decorator.
- 0580 The in-memory store enforces no foreign key, no CHECK constraint and no RLS, so those defect classes are visible only on the Postgres leg.
- 0581 A missing `BOLTRIG_TEST_DATABASE_URL` ends the test run non-zero rather than silently skipping the Postgres legs.
- 0583 Backups are `pg_dump -Fc`, each archive is verified parseable by `pg_restore` before upload, and the recovery-set manifest is uploaded last.
- 0584 `credential_refs.expires_at` is written for rotation alerts and read by no store query.
- 0585 `agent_capabilities` deliberately has no primary key and is unique on `(tenant_id, coalesce(workspace_id, ''), name)` so two workspaces may each own the same profile name.
- 0586 `workspace_members` carries `tenant_id` in its primary key because workspace ids collide across orgs by construction.
- 0587 `identity_orgs` is a pre-tenant membership index holding no secret and no business data, and is never the authority for an access decision.

AREA 06

Adapters, egress policy, and MCP consumption

98 requirements

- 0600 Every adapter implements one Protocol carrying `id`, `version`, `runtime`, `describe()`, `execute()` and `health()`.
- 0601 `describe()` returns a list of `VerbSpec` values and performs no I/O.
- 0602 `VerbSpec` declares verb `id`, noun `id`, input and output schema, consequence, description, rate limit, degraded mode, idempotency mode, implements and capability version as data.
- 0603 `Credential.material` is repr suppressed and `__str__` renders a wrapper, so no format string or traceback prints the secret.
- 0604 `bearer_token` derives a token from credential material by trying `token`, `api_key`, then `value`, returning `None` when absent.
- 0605 Adapters map native errors onto a seven member `ErrorClass` taxonomy.
- 0606 `Result` carries `ok`, `output` and an optional `AdapterError` with retry hints.
- 0607 `register_adapter_verbs` upserts a noun, a verb and a binding for each declared spec, so a new integration needs no kernel code change.
- 0608 Adapter registration never replaces a binding whose target type is `AGENT`.
- 0609 Two adapters may publish the same verb; the last registration wins and manifest order picks the provider.
- 0610 When an `EffectLog` is passed, registration records the exact inverse of every noun, verb and binding change at apply time.
- 0611 An external provider's whole operation catalogue is ingested as capability source operations; builtin adapters ingest only specs that declare `implements`.
- 0612 A declared `implements` produces an approved capability binding only for a first party connection; every other source lands proposed and routes nothing.
- 0613 Optional adapter members `source`, `activated`, `mcp_resources`, `inverses`, `setup_without_probe`, `review_and_activate` and `connect` are read by `getattr`, so an adapter omitting them is unchanged.
- 0614 Registering an adapter writes an `adapters` row carrying version, runtime, source, module ref and the review gate flag.
- 0615 The loader holds live instances keyed by `(tenant_id, adapter_id)` and hot replaces on re-register without a kernel restart.
- 0616 `AdapterLoader.load_module` never raises: a failed import records health `down` keyed by the module ref and returns `None`.
- 0617 `refresh_health` bounds every probe at 2.5 seconds and records a hung backend as `down`.
- 0618 `health_snapshot` serves the cached posture and schedules a bounded background refresh when stale, never awaiting adapter I/O.
- 0619 An adapter declaring `setup_without_probe` registers with health `degraded`, not `ok`; every other adapter registers `unknown`.
- 0620 `unload` forgets a live instance and its health row and is a no-op when absent.

- 0621 `HttpAdapter` maps 401/403 to UNAUTHORISED, 404 NOT_FOUND, 409 CONFLICT, 429 RATE_LIMITED, 5xx UNAVAILABLE, other 4xx INVALID and anything else INTERNAL.
- 0622 Only GET, HEAD and OPTIONS are auto retried, so a mutating call is never re-issued after a transport error or 5xx.
- 0623 `Retry-After` is honoured as delta seconds or as an HTTP date, clamped to the policy max delay.
- 0624 Each HTTP adapter instance holds a sliding window client side rate limiter acquired before every attempt.
- 0625 `HttpAdapter._client` builds a pinned transport and FAILS construction rather than shipping an unpinned client when the guard refuses.
- 0626 Every adapter HTTP client sets `follow_redirects=False`, so a 30x cannot be chased into internal space.
- 0627 `HttpAdapter.request` asserts the egress guard on the EFFECTIVE joined URL before the request leaves.
- 0628 Credential material becomes either an `httpx BasicAuth` object or an Authorization header, and only the derived value leaves `_auth`.
- 0629 A per tenant base URL may ride in on credential material and overrides the class default.
- 0630 Every outbound response is buffered under a fixed byte cap with transparent decompression disabled, and a server ignoring identity encoding is refused before body iteration.
- 0631 Link and offset pagination helpers stop at 50 pages.
- 0632 An unexpected exception inside an HTTP adapter is converted to an INTERNAL `Result` rather than propagating to the kernel.
- 0633 `HttpAdapter.health` probes the base URL through a plain unguarded, unpinned `httpx` client.
- 0634 A read scoped SQL binding cannot write, and the refusal lands before any statement reaches the driver.
- 0635 SQL caller values are always bound parameters; the SQL text never interpolates a caller value.
- 0636 A DSN may ride in on credential material or be built from host, database, user, password and port with URL quoting, else the adapter default is used.
- 0637 A missing `sqlalchemy`, a missing dialect driver or an unreachable backend degrades to UNAVAILABLE instead of crashing the kernel.
- 0638 `check_network_policy` decides in fixed order: scheme, air gap, block list, allow list, then the SSRF guard over every resolved address.
- 0639 An unparseable address is refused, and a host that did not resolve is refused.
- 0640 `allow_internal` skips ONLY the internal address check; scheme, air gap and the domain lists still apply.
- 0641 `resolve_host` resolves once, preserves resolver preference order and de-duplicates, returning an empty list on `OSError`.
- 0642 Pinned clients connect to the audited IP literal while TLS SNI and certificate verification keep the original host, and force `follow_redirects=False`.
- 0643 With `https_proxy` configured the client is proxied and unpinned, and the local guard still runs before the client is built.

- 0644 An operator supplied `ca_bundle` builds a context trusting only those roots and a missing or malformed bundle fails closed rather than falling back to public roots.
- 0645 The composition root installs one process wide `NetworkConfig` that binds every adapter built by a plain factory, and an explicit constructor config always supersedes it.
- 0646 An egress refusal message reaching an agent never names the resolved internal address; the addresses go to the kernel log at the raise site.
- 0648 The OpenAPI to adapter transform is deterministic and offline; no model is called.
- 0649 Each operation derives a verb id, a noun id, merged input schema, output schema, consequence by HTTP method, a rate limit and a pagination style.
- 0650 A generated adapter is inert until a named reviewer activates it, and `execute` itself refuses while inert.
- 0651 An OpenAPI spec URL is fetched through the pinned egress guard, streamed, and refused past a four megabyte cap; a local file path requires an explicit opt-in.
- 0652 A generated adapter persists a bounded executable projection, never the OpenAPI document, capped at one megabyte and 512 operations, validated by round trip before any row is created.
- 0653 A generated adapter declares no canonical capability: neither the generator nor the durable projection carries `implements`.
- 0654 MCP registration validates an absolute http(s) URL with no embedded credentials, query or fragment, and refuses raw secret material in favour of a credential reference.
- 0655 The MCP store row persists a JSON spec of url, `allow_internal` and credential id, and a pre-flag plain url row rehydrates with the guarded default.
- 0656 Tool discovery follows `tools/list` cursors under four bounds: 50 pages, an accumulated scan cap of 5000, a 2KB cursor and a repeat cursor refusal.
- 0657 A tool whose prefixed id is not a publishable verb id is skipped and reported once per page, never once per tool.
- 0658 A duplicate tool name, within a page or across pages, invalidates the whole discovery.
- 0659 The assembled snapshot is validated once, against the whole collection, at 5000 rows and two megabytes of canonical JSON.
- 0660 A consumed tool with no positive low risk signal registers as consequence high; absence is never read as evidence of safety.
- 0661 An external `implements` claim is accepted only as a bounded identifier with no version pin, and the binding it produces lands proposed.
- 0662 Every external tool description is prefixed as data rather than instructions before it reaches model context.
- 0663 A consumed tool's verb honours the server's declared `outputSchema` and does not assert an object shape the protocol does not require.
- 0664 Every MCP POST carries both the bearer and the boltrig token header, the handshake is lazy, and only a 400 or 404 earns one handshake and one retry.
- 0665 An SSE framed JSON-RPC response decodes to the last data event carrying a result or an error.
- 0666 A final MCP HTTP refusal carries only the status code; the response body never crosses the transport boundary.

- 0667 The MCP transport bounds every response body at the shared four megabyte JSON ceiling before buffering.
- 0668 A consumed MCP server refuses execution until the review gate has activated it.
- 0669 A consumed MCP call with no resolved credential fails closed with UNAUTHORISED and never reaches the transport.
- 0670 The external server receives the bare tool name; the Boltrig namespace prefix never crosses the wire.
- 0671 A non-Boltrig MCP result's text blocks are joined and fenced as data, not instructions, before becoming verb output.
- 0672 An MCP consumer reports health `ok` only after a round trip answered in this process; rehydrated specs alone report `degraded`.
- 0673 A probe returns only a closed failure code; remote bodies, exception messages, URLs and credential references never cross the probe boundary.
- 0674 MCP activation requires a prior probe, a fresh probe whose snapshot digest matches the approved one, and a recorded HITL reviewer.
- 0675 A failed MCP activation reverts every row it added or displaced, and a losing CAS never unpublishes an identical winner.
- 0676 Deactivation unpublishes only the verbs the adapter owns and drops only the nouns nothing else references.
- 0677 The dispatch adapter provider is store authoritative: a generated adapter serves only when activated, an MCP consumer only when its lifecycle is active, and a missing row unloads the instance.
- 0678 Boot rehydration rebuilds MCP and generated instances from their persisted projection and degrades one unusable stored snapshot rather than failing startup.
- 0680 `McpResourceSpec` is declarative data mapping a uri prefix onto a governed list verb and read verb with named projection keys.
- 0681 Adapter declared MCP resources are registered only through `Kernel.register_adapter`; loader-only registration paths publish none.
- 0682 MCP `resources/list` and `resources/read` invoke the named verbs through `kernel.invoke`, so grants, HITL, credential resolution and audit all apply.
- 0683 A resource is visible only when the tenant ceiling AND the run token grant BOTH its list verb and its read verb.
- 0684 A resource read refuses an empty id or one containing a path separator, and any failure returns one indistinguishable not-found-or-not-permitted error.
- 0685 Only the browser CLI adapter and the knowledge adapter declare MCP resources.
- 0686 Builtin adapters are registered by explicit factory calls and manifest module refs; nothing scans the builtin directory.
- 0687 `web.fetch` refuses a policy blocked target before any network call and an agent supplied `max_bytes` can only shrink the 256KB cap.
- 0688 `channel.send` is consequence high and its outbound webhook leg uses a pinned client carrying the manifest network posture.
- 0689 Browser navigation validates the URL as public http(s), refuses localhost forms, runs the egress guard and then applies the optional domain allow list.

- 0690 Five audio adapters publish overlapping `voice.*` verbs and the split is made stable by withholding a verb rather than by binding precedence.
- 0691 The local whisper and pocket voice adapters carry an explicit `allow_internal` network config that replaces, rather than merges with, the operator's manifest egress posture.
- 0692 The cloud audio family's `setup_without_probe` posture holds only until the first background health refresh, after which an inherited unguarded probe can report `ok`.
- 0693 Script runtime adapters run their child process under an allow-listed scrubbed environment with an output cap and a terminate-on-timeout.
- 0694 The voice tone package measures prosody in pure stdlib and reports no tone until a per speaker baseline has heard six utterances.
- 0695 Changing a builtin adapter's declared verb data requires re-registering it per tenant through `scripts/resync-builtin-verbs.py`, because boot rehydration cannot reconstruct a builtin row.
- 0696 Live adapter reads against real providers are opt-in only, gated on `BOLTRIG_LIVE_SMOKE` and the per adapter credential env, and never run in the offline suite.
- 0697 A manifest adapter whose module ref no longer imports warns and is skipped, and boot continues.
- 0698 A manifest `mcp.consume` entry using the documented `credential:` key raises `ControlConflict` out of an unguarded boot path and aborts startup.
- 0699 Registration refuses a reserved or malformed adapter id, and activation refuses duplicate verb ids, reserved core verb prefixes and a verb already owned by another target.

AREA 07

Identity, authentication, seats and capabilities

95 requirements

- 0700 The system recognises exactly six Principal-producing resolvers: federated OIDC, Cloudflare Access, first-party session, personal access token, dev header, and the two internal builders (channel-bound sender, workflow webhook owner).
- 0701 A Principal carries `credential_kind` as a separate axis from `actor_tier`, defaulting to `machine` so an unlabelled resolver is refused rather than admitted.
- 0702 Exactly one authentication posture is selected at boot, in the order session, Cloudflare Access, generic OIDC, dev auth, deny-all.
- 0703 With no authentication configured every request is refused 401.
- 0704 `BOLTRIG_DEV_AUTH=1` under any production signal aborts boot with a FATAL RuntimeError.
- 0705 `create_app` called with no principal resolver under a production signal raises rather than defaulting to the header resolver.
- 0706 A partial manifest OIDC trio refuses boot and a manifest trio differing from the process trio refuses boot.
- 0707 OIDC verification pins an asymmetric algorithm allowlist so `alg=none` and HS* confusion are rejected by construction.
- 0708 A verified token must carry `iss`, `aud` and `exp`, and clock leeway is clamped to at most 120 seconds.
- 0709 A token's absolute lifetime is capped at 24 hours by two independent checks, so a missing `iat` cannot let a far-future `exp` through.
- 0710 A kid miss forces at most one JWKS refetch per 30 seconds, then fails closed with 401.
- 0711 An ID token presented as an access token is rejected 401 when the token says so via `token_use` or `typ`.
- 0714 Identity comes only from the verified assertion: tenant and subject are never read from the request body or an untrusted header on any production resolver.
- 0715 Cloudflare Access admits only the tiers superadmin, admin and member; any other mapped role is denied 403 fail-closed.
- 0716 Every admitted Cloudflare Access tier receives tenant-wide `{all: true}` scope and is differentiated only by authoring rights and the HITL gate.
- 0717 An email is normalised to lower case on both sides of the Cloudflare Access role map.
- 0718 The dev header resolver is reachable only in local development and stamps `credential_kind="dev-header"`.
- 0719 A caller in several mapped IdP groups receives the highest-privilege matched role and the union of matched scopes.
- 0720 A caller matching no mapping receives role `none` and an empty scope.
- 0721 An empty or missing scope yields `EMPTY_GRANTS`, so authority derives from scope and never from role.
- 0722 Departments are visibility metadata and never widen the verb GrantSet.
- 0723 A GrantSet is deny-dominant, fail-closed, and matches only terminal wildcards.

- 0724 A verb id that is not a safe identifier after NFKC never matches any grant pattern.
- 0725 A role stated with no scope at all is filled from `ORG_ROLE_DEFAULT_SCOPE` for superadmin, admin and org-admin only, and a stated-but-narrow scope is never widened.
- 0726 Provisioning a role with no scope and no canonical default logs a warning naming the consequence rather than failing silently.
- 0727 A workspace membership can only intersect authority DOWN; the result is always a subset of the org grants.
- 0728 A workspace viewer keeps only concrete read-action grants; every wildcard collapses.
- 0729 An unknown workspace role yields `EMPTY_GRANTS`.
- 0730 A caller with no active workspace keeps their full org grants unchanged.
- 0731 A deactivated user's current grants are `EMPTY_GRANTS`.
- 0732 A previously provisioned user's stored role, scope and status are authoritative on every later login, so re-login cannot self-revive a deactivated account.
- 0733 An unmapped, un-invited federated identity is denied 403.
- 0734 An expired invitation confers nothing and a lost consume race confers nothing.
- 0735 The session cookie is httpOnly, Secure and SameSite=strict, with a 12-hour bounded lifetime and a JS-readable CSRF mirror.
- 0736 An explicitly CORS-allowlisted packaged Tauri origin receives `SameSite=None` cookies; CSRF stays mandatory.
- 0737 Only the sha256 of a session secret is persisted; the secret leaves in the Set-Cookie header and is never stored or logged.
- 0738 A session refresh rotates the secret and CSRF token in place and refuses past a 7-day creation-anchored absolute cap.
- 0739 Every mutating cookie request must echo the session-bound CSRF token in `x-boltrig-csrf`, compared in constant time; bearer callers are exempt because they never reach this code.
- 0740 Each request is bound to exactly ONE active org, re-authorised against `org_members` after binding that tenant, never trusting the client hint.
- 0741 A membership-model session whose every org membership was revoked fails closed rather than inheriting realm access.
- 0742 An org switch is the only place the active org changes and is membership-re-authorised with no write on refusal.
- 0743 A workspace switch refuses an unknown workspace 404 and a non-member workspace 403, both with no write and no audit.
- 0744 An account still holding its provisioning credential reaches only the CSRF, change-password and logout routes, checked on every request rather than only at login.
- 0745 When the active org requires two-factor and the caller has no activated factor, only the enrollment surface and logout are reachable.
- 0746 An account exists only by consuming a single-use, hashed, expiring invitation; there is no self-signup route.
- 0747 Accept-invite is throttled per IP before the argon2 spend, because it is the only public endpoint paying 64 MiB before a bearer is claimed.

- 0748 Unknown, expired and already-used invitations share one generic 400 so a probe cannot distinguish them.
- 0749 Passwords are argon2id PHC strings with a per-user embedded salt, at least 12 and at most 1024 characters.
- 0750 The absent-user login path always spends a full argon2 verify against a fixed decoy hash.
- 0751 Login enforces both a per-IP and a per-identity rate limit before touching the credential store and returns one byte-identical generic failure.
- 0752 Every login, logout, invite acceptance and 2FA event is audited keys-only, positionally, so the secret is absent from the dict the call site passes.
- 0753 The shared password credential is held ONCE at the identity realm keyed by the normalised email and is never duplicated per org.
- 0754 The TOTP shared secret is sealed in `credential_refs` and returned to the client exactly once at enroll-begin.
- 0755 Re-enrolling an already-enabled factor is refused 400, so the secret is never re-revealed.
- 0756 Recovery codes are ten 130-bit codes shown once, stored only as sha256, and consumed atomically.
- 0757 An enrolled user's correct-password login issues NO session and returns a single-use five-minute challenge instead.
- 0758 Only a TOTP code activates enrollment; a recovery code cannot bootstrap the factor.
- 0759 A wrong second factor does not consume the challenge; a correct one consumes it atomically and a lost race fails.
- 0760 Self-disabling two-factor requires a fresh TOTP or recovery code and overwrites the sealed secret material.
- 0761 Two-factor state travels with the identity realm and is never written under the active org.
- 0762 A PAT is recognised by the `boltrig_pat_` prefix, stored only as sha256, and bounded by an expiry clamped to at most 365 days.
- 0763 A PAT's stored scope is the requested patterns the owner's grants actually permit, so it can never be minted above the user.
- 0764 A PAT's effective authority is re-computed on every call as its scope intersected with the owner's current grants narrowed by the active workspace ceiling.
- 0765 PAT resolution binds the tenant before the RLS-scoped owner read, because the token table is the one legitimate cross-tenant lookup.
- 0766 A PAT naming a workspace its owner cannot reach is refused 403 and never falls back to unscoped org authority.
- 0767 With no requested workspace, a PAT binds a single membership unambiguously and stays unscoped at zero or many.
- 0768 The PAT list view carries neither the secret nor its hash.
- 0769 `POST /v1/me/tokens` accepts any authenticated principal, including a PAT, with no interactive-credential check and no rate limit.
- 0770 The host-boundary mint caps at the target's ORG grants and attributes the audit row to `host-boundary`, never the target.

- 0771 The identity command set at the host boundary is exactly `initiate`, `set-password`, `mint-token`, pinned by a static read of the router's literal.
- 0772 `boltrig initiate` seats exactly one founding owner, flags the account for forced rotation, and refuses to run twice.
- 0773 `boltrig set-password` operates only on an existing identity, never creating one, and discharges the forced-rotation flag.
- 0774 `boltrig set-password` does NOT revoke any existing session for the rotated identity.
- 0775 Self-service password rotation proves the current password before setting a new one and refuses an unchanged password.
- 0776 Self-service rotation revokes every other session of the identity while keeping the caller's own.
- 0777 Password-reset requests are non-enumerating, rate-limited per identity and per IP, and mint nothing at all when no delivery notifier is configured.
- 0778 Reset redemption is one atomic transaction that claims the token, rotates the credential, clears the forced-rotation flag, revokes every session and deletes every pending 2FA challenge.
- 0779 Reset redemption issues no replacement session and deletes both cookies.
- 0780 Reset delivery evidence discloses no recipient, provider, address, exception or message content and never represents notifier acceptance as inbox delivery.
- 0782 An AI key is accepted once, sealed before persistence under an opaque proposal with a 15-minute maximum TTL, and never echoed in any response or audit row.
- 0783 Caller-supplied `approval_id`, `proposal_id` or `secret_digest` on the AI-key intake is refused as server-owned evidence.
- 0784 An org-level AI key requires an admin role; a workspace or user key additionally requires the org's `allow_own_ai_keys` flag.
- 0785 Connecting your OWN user-level key folds the approval answer into the same press, and a refusal from the HITL layer leaves the pending response standing.
- 0786 AI-key resolution precedence is user, workspace, org, then the manifest default, and the user and workspace levels are skipped entirely when the org has not opted in.
- 0787 An absent organisation row is treated as `allow_own_ai_keys = False`.
- 0788 A vision request falls back to the same scope's text key before descending a level.
- 0789 A provider seat is re-mintable in place: its credential ref is a deterministic digest of tenant, level, scope and modality, and `ensure` re-provisions onto the same ref when the binding is not usable.
- 0790 `POST /v1/ai-keys/activate` re-mints a provider seat with no fresh secret and no fresh approval, gated only by the same authorisation as the set route.
- 0792 A personal integration credential is preferred over the org's only when the org sets `allow_own_integration_credentials`; otherwise the user row is skipped entirely.
- 0793 Personal integration setup is refused BEFORE anything is sealed when the org has not opted in, and refused when the caller has no personal scope.
- 0794 The roster routes reject granting a role ranked above the caller's own and restrict all-authority scope to the owner tier.

- 0795 Provisioning a brand-new org from an invitation is superadmin-only at invite creation, refused 403 with no invitation written.
- 0796 Device enrollment requires a human actor tier and an available Ed25519 signer, and stores only the sha256 of a ten-minute authorisation code.
- 0797 A device lease is issued only against a CONSUMED approval whose respondent is not in the requester set, with no relief admitted.
- 0798 The sole-author approval relief lifts independence without consulting `credential_kind`, so a PAT principal in the sole author's name can answer its own approval.

AREA 08

Memory planes, the Knowledge extension, and distillation

96 requirements

- 0800 Five memory planes are named and exactly three (semantic, episodic, procedural) are writable through the typed write gate.
- 0801 `memory.propose` refuses any plane outside the writable three, naming source knowledge and working state as not writable memory.
- 0802 A semantic fact occupies the slot `subject_type::subject_id::predicate::owner_scope`, independent of its natural-language wording.
- 0803 A procedure occupies the slot `procedure::procedure_key::owner_scope` and its key must be `::separated` with at least three non-empty parts.
- 0804 A semantic proposal naming a predicate outside the closed per-subject registry is rejected `REJECT_INVALID_PREDICATE` before any lock is taken.
- 0805 Present-state wording is rejected `REJECT_TRANSIENT` unless the caller explicitly asserts `is_durable` true.
- 0806 A proposal below the 0.50 review floor is rejected `REJECT_UNSUPPORTED` and never becomes a candidate.
- 0807 A source of strictly lower authority than the slot's current value is rejected `REJECT_LOWER_AUTHORITY` with both ranks recorded in the event detail.
- 0808 An equal-authority conflict with a different current value is parked as a candidate for review, never applied.
- 0809 Accepting a changed semantic value closes the previous row as superseded with `valid_to` set and opens a new row at `version+1` carrying `supersedes_id`.
- 0810 Re-observing the current value normalises to `CONFIRM_EXISTING` and writes no new fact row.
- 0811 Episodes are append-only: they are written with `memory_key` unset and have no supersession path.
- 0812 An episode is written only from a terminal run with a non-empty `retrieval_text` and an outcome from the four-value vocabulary.
- 0813 A procedural proposal is always born a candidate; no confidence value activates it.
- 0814 Explicit review through the high-consequence `memory.candidates.review` verb is the only path that activates a candidate.
- 0815 A candidate whose status is not exactly candidate cannot be reviewed, so a rejected candidate can never be re-activated.
- 0816 Every write-gate decision appends a policy-versioned `memory_events` row recording the decision code and never the memory content.
- 0817 Concurrent writes to one slot are serialised in-process by a per-slot asyncio lock, with the DB partial unique index as the authoritative arbitrator.
- 0818 Active procedures are resolved deterministically by role/workflow/owner-scope specificity ranking and never by similarity.

- 0819 Only facts whose status is active and whose valid_to has not passed can enter the bundle's procedural lane, so candidates never govern a prompt.
- 0820 The bundle's semantic lane returns only active subject-resolved slots and re-filters every row to the caller's permitted scopes.
- 0821 Episodes are retrieved by their stored problem representation and rendered as advisory precedent with failed attempts kept distinct from the resolution.
- 0822 Working context supplied to memory.bundle is passed through to the rendered prompt and is never persisted to any memory plane.
- 0823 The rendered bundle states that only content inside active_procedures may establish operating instructions and that all lower sections are evidence.
- 0824 Each bundle section clips to its per-plane character budget and appends a named warning, never clipping silently.
- 0825 Per-plane toggles and budgets merge manifest defaults with per-call config overrides at one point, so an ablation needs no retrieval-code change.
- 0827 Recall results are re-filtered to the caller's permitted scopes after the engine or projection returns them, whatever the source.
- 0828 The recall audit row records query, mode, scopes and count and never records fact contents.
- 0829 Content is screened for injection markers and for API secrets before it becomes memory, and a hit is refused fail-closed with a denied audit row.
- 0830 Memory classified sensitive is refused unless the configured embedding endpoint is in the local endpoint set, and the misroute is audited.
- 0831 memory.forget requires a target or a source_ref so an empty erasure can never be a silent no-op.
- 0832 Erasure writes a MemoryErasure ledger row carrying the requester, target, scope, facts_removed and completion time, and audits the act.
- 0833 The memory engine is a manifest choice among local, vector, pgvector and cognee behind one Protocol, and the kernel core imports none of them.
- 0834 The pgvector engine scope-filters every read in SQL and loads only edges whose both endpoints are in scope, so a cross-scope edge is structurally unfollowable.
- 0835 No approximate-nearest-neighbour index exists for any vector column in the tree, so vector recall is a sequential scan.
- 0836 Absent an embedding.base_url and embedding.model, memory embeddings are deterministic feature hashes from HashingEmbedder rather than a semantic model.
- 0837 Each projection operation records a status row from a closed vocabulary and an out-of-vocabulary backend answer is recorded failed and never retried.
- 0838 Inline projection retry is exactly one bounded reattempt when fanout.retry_failed is true, and the applied budget is recorded on the status row.
- 0839 A queued projection task refuses to run when its payload tenant differs from its context envelope tenant.
- 0840 Projection recall re-hydrates every hit from the kernel ledger, labels the projection source, and returns None on any exception so the engine answers instead.

- 0841 Cognee maps one dataset per (tenant, owner_scope) with a sha256 suffix that keeps the mapping injective after slugging.
- 0842 CogneeEngine.remember refuses secret-bearing content before the cognee package is imported, so nothing secret can reach its stores.
- 0843 Cognee erasure drops each affected dataset and rebuilds it from the surviving facts only, so a recall after forget cannot return the erased fact from any layer.
- 0844 Cognee request configs are built with model_construct so pydantic-settings never absorbs unrelated process environment variables, and undeclared fields are refused.
- 0845 Cognee reuses the caller's ordinary tenant/workspace/user chat connection through one Bi-frost virtual key and exact model route; provider plaintext stays in the kernel.
- 0846 Cognee health reports down when the package is unimportable and degraded with a reason when it is importable but has no usable model, never claiming an outage.
- 0847 A Knowledge upload proceeds begun then staged then committed, and only a staged upload may commit.
- 0848 Commit re-verifies the staged bytes against the recorded sha256 digest and byte length before extracting or promoting them.
- 0849 Committing an already-committed upload returns the same asset id with replayed true rather than creating a second asset.
- 0850 Every search hit carries a citation object with asset_id, revision_id, segment_id, locator, source_kind, source_ref and content_hash.
- 0851 Knowledge permitted scopes are derived server-side from the authenticated principal and never from caller-supplied context extras.
- 0852 The asset-access scope predicate is inside the search WHERE clause, so an out-of-scope candidate is never scored or ranked.
- 0853 Knowledge search scores title exactness, ts_rank_cd over a generated tsvector, and 0.35 times clamped pgvector cosine in one query.
- 0854 Production Knowledge always embeds with HashingEmbedder because register_knowledge passes no embedder and no manifest or environment key selects one.
- 0855 Asset erasure removes derived segments immediately and removes the original object only when no surviving revision references its digest.
- 0856 Both ObjectVault implementations refuse an object key that escapes their root or prefix and validate the digest before promoting a staged object.
- 0857 Extraction supports only text, Markdown and PDF and refuses a document with no extractable text, over 5000 segments, or over 5000000 characters.
- 0858 knowledge.context.build returns a bounded envelope labelling every item untrusted_source_content with authority original_revision plus an omissions count and a context hash.
- 0859 A degraded or failing Cognee compiler cannot roll back the canonical Knowledge commit, because compile runs after the ingestion transaction and returns statuses rather than raising.
- 0860 Cognee is the only provider id the Knowledge service will construct, list publicly, or enable; Mem0 and Supermemory are retired, not disabled add-ons.
- 0861 Persisted Mem0 and Supermemory provider rows are rewritten to disabled and retired on every Knowledge registration and are hidden from public reads.

- 0863 A distillation corpus refuses a target endpoint or a record outside the corpus tenant rather than silently thinning itself.
- 0864 A distillation corpus may only target a sensitive-classed endpoint, because run-level sensitivity is not durably recorded per run.
- 0865 A conversation covered by any MemoryErasure is excluded from every corpus built afterwards and the erasure watermark is folded into the corpus digest.
- 0866 The corpus digest hashes record CONTENT rather than record ids, so a change to the scrubber changes the digest.
- 0867 Exact duplicate records are collapsed at build keeping the best-signal copy at its original position, and the count is reported on the receipt.
- 0868 The held-out split is a deterministic ten percent by record-id hash and is guaranteed non-empty whenever any sft record exists.
- 0869 Corpus text is PII-redacted first and then refused entirely if secret material survives, and one secret in the prompt poisons the whole record.
- 0870 The distill.train schema has no field naming any starting point other than the composed base pin, so adapter-on-adapter training is unrepresentable.
- 0871 The sidecar refuses a corpus whose header base_pin differs from the request, and the adapter refuses a train response reporting any other base.
- 0872 The register gate promotes only when the candidate assigns strictly higher held-out mean log-likelihood than the incumbent; a tie keeps the incumbent.
- 0873 A candidate whose seeded distinct-2 diversity is below 0.8 times the incumbent's is held with reason entropy_collapse regardless of likelihood.
- 0874 Every gate run writes one hash-chained audit receipt carrying the digest, base pin, both scores, both diversities and the reason, on promote and on hold alike.
- 0875 distill.promote activates an endpoint only when a passing gate receipt matching the exact corpus digest and model is found in the newest 500 audit rows.
- 0876 Promotion installs the per-model price into the cost accountant in the same act, closing the unpriced-local-model cost-tier trap.
- 0877 One night is one governed verb running build then ship then train then gate inside the adapter, because step outputs cannot thread between workflow steps.
- 0878 distill.night refuses before any corpus construction or sidecar request unless the organisation setting behaviour.overnight.enabled is exactly true.
- 0879 A night whose corpus has zero records returns success with reason empty_corpus and never trains.
- 0881 The distill sidecar binds loopback by default and performs no authentication of any request.
- 0882 Held-out records are excluded from the training stream for both adapter kinds, so a candidate cannot pass by memorising its own exam.
- 0883 The trainer replays human-anchored signals harder than clean runs and doubles records from the last seven days, applying weights to the train split only.
- 0884 Conversation compaction appends a new conversation_summaries row and never edits a frozen message record.
- 0885 Between compactions the composed task stays a byte-prefix of the next turn's, because the summary block is byte-stable and the tail only appends.

- 0886 Superseded turns are excluded from both the summarised older set and the verbatim tail, because both are drawn from the superseded-filtered live set.
- 0887 The shipped summariser is deterministic and offline; an optional model summariser seam falls back to it on failure or empty output.
- 0888 Compaction thresholds are tighten-only: a manifest may lower the threshold or keep fewer recent turns but never grow the verbatim window past the code ceiling.
- 0889 Session distillation is off unless `memory.ingest.on_session_end` is exactly true, and a malformed idle window reverts to sixty minutes rather than guessing.
- 0890 A re-distilled thread has its prior summary retired before the new one is written, so a crash leaves it summary-less rather than double-summarised.
- 0891 Each distillation acts under a seat built for that thread with `on_behalf_of` set to the thread owner and only `memory.remember` and `memory.forget` granted.
- 0892 Session distillation writes through `kernel.invoke` rather than the store, so the anti-poisoning screen and the grant check both run.
- 0893 Migration 0076 adds eight nullable typed columns plus two partial unique indexes enforcing one active row per semantic and per procedural slot.
- 0894 Migration 0034 creates the thirteen knowledge tables with a generated `tsvector` column, a GIN index, a fixed `vector(256)` column and a `RESTRICT` reference from revisions to blobs.
- 0895 Every memory and knowledge table is in the opt-in RLS overlay with a `FORCE` tenant-isolation policy that yields zero rows on a null GUC.
- 0896 Seven memory manifest keys are advertised in the example manifest and read by no code, and `ingest.screen_content` is read only as a readiness warning.
- 0897 Memory, Knowledge and distillation are each default OFF and register nothing unless their manifest section explicitly enables them.
- 0898 Distillation refuses to register at all when `base_pin` is missing, because an unpinned base would make promotion state underivable.

AREA 09

Workflows, work items, skills and the YAML libraries

92 requirements

- 0900 A workflow is data: the executable graph lives entirely in the WorkflowDefinition.definition JSON column and adding one requires no code change.
- 0901 The interpreter orders steps by Kahn topological sort honouring each step parents list and keeps definition order among independent steps.
- 0902 A step naming a missing parent or lying in a cycle is recorded skipped with reason missing_parent_or_cycle and fails the run.
- 0903 Every non-control step is dispatched through kernel.invoke and there is no second dispatch path in the interpreter.
- 0904 A step action is an unconstrained string: no allowlist, no registry pre-check and no comparison against the author own grants exists at authoring or run time.
- 0905 A workflow step executes under the triggering caller own InvocationContext; no service or system seat is minted for a workflow run.
- 0906 The control nouns trigger, flow and code are resolved inside the interpreter and never reach kernel.invoke; a control noun with an unknown verb records skipped and fails the run.
- 0907 code.run is recognised and records its script length but never executes anything.
- 0908 Branch predicates are declarative left/op/right structures with no eval; an unknown operator or a TypeError from an ordering comparison evaluates false.
- 0909 A multi-case flow.branch returns the first matching case label in declaration order, else default_label, else the literal false; a case with no conditions is the unconditional else arm and a case without a non-empty string label never matches.
- 0910 A step that declares no branch key always satisfies the branch gate and runs.
- 0911 An unresolvable \$step.path reference resolves to None rather than raising.
- 0912 Capability step params are passed to kernel.invoke verbatim: reference resolution happens only in branch predicates, multi-case conditions and items_from.
- 0913 loop_bindings replace a whole top-level param value with a deep copy of the item or the zero-based index and never interpolate a string.
- 0914 A flow.loop must declare exactly one item source, and an items_from reference must point at an ancestor step output.
- 0915 Loop items are capped at 100 selected values and 256 KiB of canonical JSON, and the discarded excess is recorded as skipped_overflow.
- 0916 Nested loops are refused at authoring time and per-iteration clone ids use the reserved <id>__<n> namespace that an authored id may not occupy.
- 0917 An invalid loop contract fails the entire run with a value-free receipt before any step is dispatched.
- 0918 A loop body is the maximal descendant sub-graph in which every step has all of its parents inside the loop or the body.

- 0919 A step mixing a loop-body parent with a parent outside the body is skipped with reason `mixed_loop_parent` rather than dispatched with unresolvable references.
- 0920 `on_item_error` continue or drop absorbs a failed iteration, keeps the aggregate ok, clears the clone from the failed set and reports errors and `errored_indexes`.
- 0921 A loop declaring parallel between 2 and 10 runs a capability-only body concurrently through a semaphore window, and a body containing any control step falls back to the sequential walk.
- 0922 Per-step retry is clamped to at most 5 additional attempts and 60000 ms, a non-integer retry value disables retry, and an `on_error` value outside `{fail, branch, default}` fails closed to fail.
- 0923 `on_error` branch writes branch fail on an exhausted failure and stamps branch success on a successful step, while `on_error` default substitutes `default_output` with `error_message` and `error_type` overriding same-named keys.
- 0924 An absorbed step failure records status exception, increments the run `exceptions_count` and is checkpointed ok so a resume replays the absorption.
- 0925 A HITL pause is never retried, is never absorbed by an error strategy, and with the checkpoint seam wired is checkpointed paused with its request id before the walk stops.
- 0926 A resumed paused step re-invokes the same verb with the checkpointed approval id so the consume-if-approved CAS executes it exactly once.
- 0927 A rejected or timed-out approval resolves the step through its error strategy without re-dispatching; any other disposition falls closed to resume so the kernel gate decides, and the pre-check never writes.
- 0928 Checkpoint keys are workflow-scoped as `<workflow_id>:<step>` and a step with a prior ok checkpoint replays its recorded output instead of being re-dispatched.
- 0929 A replayed `flow.loop` re-resolves its items and re-expands only when the recorded output matches exactly, otherwise the step fails `loop_replay_mismatch`.
- 0930 A checkpointed step dispatches with the deterministic key `workflow:<wf>:<run>:<step>` unless the verb is idempotency-disabled, and an `IdempotencyConflict` falls back to exactly one keyless invoke.
- 0931 The run status is paused if any step paused, else failed if any step genuinely failed, else completed; branch and propagation skips never fail the run.
- 0932 A step with any failure-lineage parent is skipped `parent_failed`, while a step whose parents are all benign branch skips is skipped `parents_skipped`, so a merge node after a branch runs exactly once.
- 0933 The run inputs are referenceable as `$inputs.<key>`, seeded only when no authored step claims the id inputs.
- 0934 Every step publishes a `workflow_step` event and the run publishes a terminal `workflow_run` marker, both inside a swallowing try so the relay can never affect a run.
- 0935 Draft rows under the reserved `__draft__` prefix are excluded from `get`, `match`, `trigger` and `execute`, and the prefix cannot be smuggled through the ordinary upsert.
- 0936 A workflow is resolvable only when it is org-wide or belongs to the caller active workspace, and an archived workflow fails closed at `trigger`, `execute` and `schedule`.
- 0937 `trigger` freezes the definition into a canonical-JSON sha256 snapshot, refuses a caller-supplied expected digest that differs, and the task body fails closed on `digest`, `schema`, `identity` or `workspace` drift when restoring it.

- 0938 A durable task payload whose tenant differs from its context envelope tenant is refused before any dispatch.
- 0939 WorkflowLibrary.execute runs single-shot with the executor boundary and no checkpoint store.
- 0940 The trigger path task body wires both the executor boundary and kernel.store, so it combines per-step boundaries with checkpoint resume.
- 0941 An infrastructure exception inside the workflow task body is re-raised so the engine may retry and the logical occurrence stays in flight.
- 0942 Every scheduled occurrence re-authorizes the bound human: active user, workspace membership, and live effective grants intersected with the captured ceiling permitting control.workflow.trigger.
- 0943 A schedule created by a caller with no matching active user row persists with observed status needs_action and never runs.
- 0944 A logical occurrence is claimed atomically by (tenant, workflow, scheduled_for) under a lease and carries the deterministic run id wfs_<sha256>.
- 0945 Occurrence catch-up is capped at 3 per schedule per cycle and the remainder is skipped forward with reason missed_occurrences_truncated.
- 0946 A cron schedule is validated at definition time for field count and IANA timezone, and the parser applies day-of-month OR day-of-week when neither is a wildcard while returning only instants that exist in the target zone.
- 0947 An occurrence whose workflow snapshot or authority-bearing schedule digest changed is failed with reason occurrence_snapshot_changed.
- 0948 A failed dispatch marks the occurrence retryable while attempts are under 3 and failed thereafter, with reason schedule_dispatch_failed.
- 0949 The scheduler forever-loop derives its overdue count from the stored next_due_at values rather than from the reconcile path, so inaction is distinguishable from idleness.
- 0950 The workflow scheduler reconciles exactly one tenant, the worker manifest tenant.
- 0951 The scheduled dispatch path calls WorkflowLibrary.trigger directly, so it is grant-checked but not HITL-gated per occurrence.
- 0952 Webhook and channel trigger deliveries dispatch control.workflow.trigger through the kernel chokepoint and can return a pending_human receipt.
- 0953 A trigger authority is always a live re-derivation of a stored human owner or an authenticated channel principal, intersected with the trigger stored grant ceiling.
- 0954 Trigger delivery is idempotent by sha256(source_scope NUL source_event_id) and a repeat returns the recorded receipt as duplicate.
- 0955 A trigger event body larger than 256 KiB of canonical JSON is refused with reason event_too_large.
- 0956 The channel intake bridge tries the HITL reply path first and only then delivers workflow triggers, answering 202 with the per-trigger outcomes.
- 0957 A v1 conversational routine is one goal plus one trigger and must contain zero graph steps.
- 0958 A routine companion_id must be exactly familiar or jarvis and every routine field is closed against unknown keys.

- 0959 Each routine occurrence requires an authenticated human owner and allocates one deterministic owner-scoped conversation before enqueue, re-read to verify all seven binding fields.
- 0960 Routine trigger inputs and human answers enter the model wrapped as explicitly untrusted data.
- 0961 A routine turn runs through the ordinary chat runtime under the triggering context grants, workspace and scope; it never mints authority of its own.
- 0962 A routine turn idempotency key is derived from the occurrence run id and, on resume, the exact HITL request and response ids.
- 0963 Routine completion is checkpointed before the optional notification, and a notification failure cannot change the recorded outcome.
- 0964 Routine inputs are capped at 64 KiB of JSON and must be JSON serialisable.
- 0965 `parse_skill` validates id, semver version, string types and that `context_requirements` is a valid Draft 2020-12 schema, reporting every error at once.
- 0966 The governed `control.skill.upsert` path performs none of the `parse_skill` validations.
- 0967 `resolve_skill` merges an extends chain parent-first: prompts concatenated, grants order-preserving union, requirements merged, base never mutated.
- 0968 A cyclic extends chain raises in the skills loader but silently resolves to a partial chain in the fleet spawn resolver.
- 0969 A skill `tool_grants` list is a request, not an award: the spawner intersects it with the caller grants and any explicit ceiling.
- 0970 `skill.search` returns selection metadata only and never the `prompt_fragment` body, clamped to at most 100 results.
- 0971 `skill.load` validates the supplied job context against the inheritance-merged requirements and raises `ContextRequirementsUnmet` on any mismatch.
- 0972 `skill.load` returns the resolved `tool_grants` as data and grants nothing, so loading a skill can never escalate the caller.
- 0973 All three skill shelf verbs are consequence low and run the ordinary dispatch chokepoint, tenant-scoped.
- 0974 The boot skill loader reads only libraries/skills, skips non-mapping documents, and a load failure is logged and swallowed so boot continues.
- 0976 `libraries/workflows/*.yaml` is authoring data that no runtime path loads.
- 0978 `libraries/emotion/*.yaml` loads into frozen dataclasses, any parse failure returns `None` so the feature stays off, and its event map consumes the interpreter own `workflow_step` events.
- 0979 `schemas/` holds versioned external contracts under `<contract>/<version>/` and the Codex protocol pin is digest-verified by a gate rather than trusted.
- 0981 `boltrig.work.normalise` is the single translation from any raw source payload to a `WorkItem`, preserves the raw payload verbatim, and derives convergent mode from a deterministic weighted-facet confidence score at threshold 0.7.
- 0984 Channel provenance is kernel-authored: four fields are derived after authentication, provider identifiers are bounded to 512 chars and kept out of the public projection.

- 0987 Workflow synthesis uses a reasoning runtime when one is supplied and falls back to the fixed five-stage deterministic pipeline on any failure.
- 0988 `harvest_reuse_signal` feeds free feedback only into memory reweighting through the chokepoint under the caller own context, and swallows every failure.
- 0989 `control.workflow.upsert`, `publish`, `schedule`, `trigger` and `execute` are HIGH consequence with only `draft.upsert` LOW, and the approval for a trigger, execute or schedule binds the exact definition snapshot digest re-checked before the mutation.
- 0990 The only definition validations performed at authoring time are the loop contract and the routine contract.
- 0991 A workflow definition has no maximum step count and the interpreter maintains no recursion depth counter.
- 0992 Draft rows are excluded from the runnable shelf but not from the workflow read routes, which filter only on workspace.
- 0993 The workflow list route projects each stored routine through `routine_spec`, which raises rather than degrading on a malformed contract.
- 0994 `BOLTRIG_WORKFLOW_SCHEDULER_INTERVAL` defaults to 15.0 seconds; a non-positive value disables the scheduler and an unparseable value falls back to the default.
- 0995 The queued context envelope carries the caller grants verbatim and nothing re-derives them when the worker later executes the task.
- 0996 The DAG-parity semantics (error strategies, retry, OR-join, multi-case branch, parallel loops, approval branch handles) are covered by 28 tests that bind no invariant id.
- 0997 The shipped skill library is gated: no wildcard over a consumed namespace, no expansion above `MAX_KERNEL_TOOLS`, and no granted namespace that resolves to zero registered verbs.
- 0998 The shipped workflow `lead-followup-cadence.yaml` fails the loop contract and therefore cannot be persisted through `control.workflow.upsert`.

AREA 10

Model catalogue, routing, and the sensitive-to-local rule

100 requirements

- 1000 A model endpoint is a tenant-scoped record of id, kind, model, base_url, fallback, data_class, is_active, modalities and revision.
- 1001 A model endpoint rejects any modality outside text, vision, stt, tts and realtime.
- 1002 A model endpoint lowercases, trims and dedupes its modalities and defaults an empty set to text.
- 1003 A model endpoint revision is a positive integer.
- 1004 The cost tier vocabulary is closed to cheap, standard and expensive across manifest and control authoring.
- 1005 An agent capability folds its legacy text and vision endpoint columns into the generic model_routes map, with the legacy value winning.
- 1006 Capability route authoring requires every named endpoint to exist, be active and advertise its bound modality.
- 1007 A capability with one text endpoint and no vision route must name a multimodal endpoint.
- 1008 The manifest models section parses to endpoints, default, sensitive_endpoint and a per-model price table.
- 1009 Applying a manifest upserts every declared endpoint and installs its price table on the cost accountant.
- 1010 A malformed manifest price entry is dropped at parse and the model falls back to its cost tier rather than blocking boot.
- 1011 The manifest endpoint parser applies no sensitive-implies-local check.
- 1012 An endpoint id must be one bounded URL-safe path segment and is never normalised.
- 1013 An exact model id refuses every mutable alias segment including latest, auto and stable.
- 1014 A user-connected model id keeps shape and path refusals but accepts the provider's own alias tags.
- 1015 Governed authoring refuses a sensitive endpoint whose kind is not local.
- 1016 Governed authoring requires an exact immutable model id for a standard bifrost, xai, x.ai or grok endpoint.
- 1017 Governed authoring of a standard bifrost endpoint proves the exact model is advertised for its declared modalities.
- 1018 The catalogue preflight runs before the approval is raised, and only for standard bifrost, xai, x.ai and grok rows.
- 1019 A fallback reference may not be the endpoint itself, must exist, and must be active.
- 1020 An approved endpoint upsert compare-and-swaps against the approved endpoint, its fallback target and its reference snapshot.
- 1021 An ordinary replacement upsert preserves the stored is_active flag so a retired endpoint cannot be edited back into service.

- 1022 Retire and restore are high-consequence controls bound to the exact endpoint state by compare-and-set.
- 1023 Every mutating endpoint operation takes a tenant-scoped advisory lock over the reference graph before reading it.
- 1024 Changing a fallback bumps the revision of the old and new fallback targets so their in-flight approvals invalidate.
- 1025 Model endpoint lifecycle behaves identically on the in-memory and PostgreSQL stores.
- 1026 The `model_endpoints` table is tenant-fenced by the row-level-security overlay.
- 1027 The router selects a capability's endpoint by modality, falling back from vision to the vision override then the primary.
- 1028 The router resolves exactly one named endpoint reference and never traverses its fallback.
- 1029 A missing or retired endpoint is audited as `model_endpoint_unavailable` and refused with a typed 409.
- 1030 An endpoint that does not advertise the requested modality is audited and refused with a typed 409.
- 1031 Standard-classified data returns its resolved endpoint unconstrained.
- 1032 Sensitive-classified data returns the capability's own endpoint when that endpoint is `data_class` sensitive.
- 1033 Sensitive-classified data on a hosted endpoint is redirected to the configured sensitive endpoint when that endpoint is itself sensitive.
- 1034 Sensitive-classified data with no reachable sensitive endpoint is audited as `sensitive_data_misrouted` and refused with a typed 403, never downgraded to a standard provider.
- 1035 A configured sensitive endpoint that is missing or retired raises `model_endpoint_unavailable` from inside the sensitive branch rather than falling through to a standard endpoint.
- 1036 The spawn path enforces the sensitive-to-local rule end to end.
- 1037 The caller's `data_class` classification is not trusted alone: the deterministic PII scanner over the outbound prompt forces the sensitive route.
- 1038 The PII scanner classifies only and never rewrites the outbound payload.
- 1039 A payload with no pattern hits stays standard and keeps its governed hosted endpoint.
- 1040 A user AI configuration naming an external provider cannot move a sensitive call off the local endpoint, and cannot rewrite a standard governed endpoint either.
- 1041 A caller model choice is refused for a non-Codex capability, a sensitive request or a pinned profile.
- 1042 A malformed, blank or oversized model choice id is refused with a typed `model_endpoint_unavailable`.
- 1043 A model choice cannot change while a conversation turn is active.
- 1044 A resolved model choice must be a standard, text-capable endpoint with an exact immutable model id, proven against the live catalogue.
- 1045 A vision request whose chosen model lacks catalogue image support is refused.

- 1046 The whole Codex-to-Bifrost endpoint composition is skipped for sensitive-classified requests.
- 1047 The default Codex route uses the composed process model rather than the capability's stored endpoint model.
- 1048 Automatic Codex routing is refused when no model gateway is configured.
- 1049 A composed runtime that cannot honestly satisfy an authored pinned profile refuses rather than serving a different model.
- 1050 An unavailable Codex runtime carries no speculative model route.
- 1051 The recorded served-model route projects model and provider only, never a base URL or credential.
- 1052 Process composition refuses to start when the injected sensitive endpoint id disagrees with the manifest.
- 1053 The startup birth-profile receipt records the sensitive role as absent or configured without disclosing the endpoint id.
- 1054 The model gateway binds per conversation, not per run, and keeps a conversation on one model across turns.
- 1055 The gateway never re-routes sensitive data and is completely inert when no gateway URL is set.
- 1056 The conversation binding table is TTL-bounded and size-bounded at 4096 entries, sweeping expired entries before evicting the earliest-expiring live ones.
- 1057 An explicit server-validated model switch rebinds the conversation's affinity rather than losing to the existing binding.
- 1058 Bifrost is wired into the stack as a profile-gated service on the sandbox network with a loopback admin port, and genesis and dev-up start that profile.
- 1059 The bifrost and local-model images are pinned to digests.
- 1060 The secure overlay removes the published host ports of both bifrost and local-model.
- 1061 Readiness reports a configured gateway with no armed probe as unchecked with reason `configured_but_health_check_disabled`, never as disabled.
- 1062 Platform status reports the gateway with cache and profile posture only and never a URL, key or token.
- 1063 Optional gateway live-health polling is internal-host-only, bounded, and fail-safe.
- 1064 Catalogue discovery calls only the configured internal `/v1/models` route on a known host with an exact `/v1` path.
- 1065 Catalogue discovery disables redirects and environment proxies and forces identity content encoding.
- 1066 Catalogue discovery bounds body bytes, row count, page count and cursor format, and validates the pinned pagination schema.
- 1067 Catalogue pagination is all-or-nothing: partial pages are never retained.
- 1068 The catalogue cache single-flights concurrent callers, expires on a clamped TTL and fails closed.
- 1069 Every catalogue failure returns a typed unavailable result with zero models and a reason from a closed ten-value vocabulary.

- 1070 Catalogue membership matches on the exact model id only: display names, aliases, duplicate rows and malformed snapshots all fail closed.
- 1071 A store endpoint's declared modalities may stand in for a catalogue row only when that row carries no `input_modalities` key at all.
- 1072 A catalogue row carrying a malformed `input_modalities` key is refused even when the store declares modalities.
- 1073 A declared vision modality is compared as image, matching the advertised form.
- 1074 The Bifrost model catalogue route is author-only and re-projects even an injected provider so upstream fields cannot escape.
- 1075 Chat model choices are filtered to active, standard, text-capable endpoints and expose an opaque choice id, exact model name, modalities and availability reason, never provider topology.
- 1076 A catalogue failure marks every model choice unavailable rather than admitting an unproven one.
- 1077 A provider outside the native Bifrost set is bound as an OpenAI-compatible custom provider and requires an explicit base URL.
- 1078 A custom provider's base URL is written to both `network_config` and `custom_provider_config`, and is read back from `network_config` first.
- 1079 An existing custom provider row bound to a different address is refused rather than silently re-pointed.
- 1080 A native Bifrost provider is created with no address.
- 1081 A self-hosted provider named in the provider-URL table carries its server URL in the provider-specific key block, and a missing or blank URL is refused with an actionable message.
- 1082 A keyed provider does not require a URL and gets no provider-specific block.
- 1083 The Bifrost administration transport accepts only an internal host on an exact `/v1` path and refuses redirects, non-identity encodings, oversized bodies and non-object JSON.
- 1084 A scoped Bifrost binding is derived idempotently from tenant, level, scope and modality and stores only the narrowed virtual key.
- 1085 A binding proven unusable is diagnosed as a wrong address or a wrong model name using the gateway's own provider-key status, which is diagnostic only.
- 1086 Revoking a scoped binding deletes the virtual key then the provider key, tolerating 404 but refusing any other status.
- 1087 Cost is priced as tokens times micros per token, with a configured per-model rate winning over the cost-tier default.
- 1088 A per-model rate may be a single blended number or the rate card's input and output pair, and a missing leg falls back to the other leg rather than to zero.
- 1089 A sub-micro per-token price is charged rather than truncated to free.
- 1090 Tokens the runtime did not attribute to a leg are billed at the input rate, so a run reporting only a total is never billed as free.
- 1091 A negative usage report is never a credit: token counts are floored at zero and the priced product is floored at zero.
- 1092 The pre-run budget estimate is deterministic and always priced at the cost-tier rate with no model.

- 1093 The budget ledger is trued up post-run by the signed actual-minus-estimate delta, and a degraded run refunds the whole estimate.
- 1094 Post-run pricing derives its price key from the capability's declared model endpoint rather than the endpoint that actually served the call.
- 1095 The Codex runtime reports an input and output token split and a non-positive or non-integer leg is recorded as zero.
- 1096 The process model-policy projection names the real consumer of each role, marks the default role inactive, and discloses no endpoint topology.
- 1097 The model-policy projection marks a sensitive role eligible only when its endpoint is active, of kind local, and data_class sensitive.
- 1098 Static readiness fails when the manifest's sensitive endpoint is not data_class sensitive, and fails when a configured memory endpoint is not local-sensitive.
- 1099 A distillation corpus may only target a sensitive-classed endpoint, so sensitive runs cannot be laundered into weights served off-box.

AREA 11

Configuration: the manifest, the environment, and release mode

62 requirements

- 1100 `load_manifest` parses YAML, interpolates `${ENV}` references, then builds frozen dataclasses.
- 1101 An unset `${VAR}` with no default interpolates to the empty string and never raises.
- 1102 The `${VAR:?message}` form is not recognised and remains a literal string in the document.
- 1103 `tenant_id` is the only mandatory top-level manifest key; its absence raises `KeyError`.
- 1104 A manifest declaring both `agents` and `hierarchy` is rejected at load.
- 1105 A manifest advertising `identity.provider: saml` is rejected at load rather than falling back to env-selected auth.
- 1106 The `agents` section is a closed key set and must declare at least one named agent.
- 1107 Each named agent's address and `scope_id` must match a lowercase slug, its runtime must be `codex`, `script` or `python-script`, its skills 1..64 bounded strings, and its depth 1..5.
- 1108 Named-agent names and addresses are unique and `agents.default` must name a declared address.
- 1109 A legacy `hierarchy` manifest is normalised to the flat roster, the chief taking the address `cos`.
- 1110 `spawn_rules` is a closed schema bounded to 128 unique rules, priority 0..1000, depth 1..10, with `name`, `priority`, `match` and `capability` required.
- 1111 The cost-tier vocabulary is closed to `cheap`, `standard` and `expensive` across manifest and control.
- 1112 The budget window vocabulary is closed to `run`, `daily` and `monthly`.
- 1113 A model price is a float or an input/output pair in micros per token; a negative or unparseable rate is dropped so the model falls back to its cost tier rather than billing zero.
- 1114 The default approval window is 86400 seconds at both the dataclass and the parser fallback, and a stated window is honoured unclamped.
- 1115 Nine chat ceilings are tighten-only: a manifest value is clamped into `[0, code default]` and a malformed value keeps the default.
- 1116 `chat.heartbeat_seconds` is honoured as supplied and is not tighten-only; a value at or below zero disables the heartbeat.
- 1117 `FleetManifest.extra` retains exactly fourteen named raw sections.
- 1118 `FleetManifest.section(name)` returns an empty dict for any key outside the retained set.
- 1119 `development_posture` is parsed from the manifest root and a malformed `expires_at` or covers yields `None` and `()`, which the posture gate refuses.
- 1120 `apply_manifest` installs the manifest's network posture as the process-wide egress default before any adapter is constructed.
- 1121 Manifest discovery tries `BOLTRIG_MANIFEST`, `/app/manifest.yaml`, `manifest.yaml`, then `manifest.example.yaml`, reading the environment variable `live`.

- 1122 `manifest.yaml` is gitignored, absent from the referent, and not baked into the kernel or fleet images.
- 1123 Compose bind-mounts the operator's `manifest.yaml` read-only at `/app/manifest.yaml`.
- 1124 `compose_process_model_runtime` loads one snapshot and exports its gateway policy before the Codex provider and model catalogue are built.
- 1125 `build_kernel_async` uses an injected manifest snapshot verbatim and never re-reads the file.
- 1126 Composition aborts when a caller-supplied sensitive endpoint disagrees with the snapshot's.
- 1127 The API threads one composition snapshot into the kernel factory, spawner factory, principal resolver, platform factory and birth receipt.
- 1128 The standalone fleet worker holds exactly one manifest snapshot for its kernel, spawner and pump.
- 1129 The Hatchet worker holds one manifest snapshot and builds its routine chat service from that snapshot's chat config.
- 1130 The API chat factory re-reads the manifest file at lifespan startup and swallows every exception, yielding the fail-closed empty chat config.
- 1131 `AdminConfig` opens the manifest file a third time in the API process and does not interpolate `${ENV}`, so its export carries placeholders rather than values.
- 1132 `register_boltrig_tasks` constructs its routine chat service without a manifest chat config, so it uses the code-default maximum caps.
- 1133 Every serving process publishes a per-boot receipt carrying an opaque digest of its effective typed manifest.
- 1134 The birth-profile projection compares every instance against the latest API receipt and reports a manifest-generation mismatch without refusing service.
- 1135 `apply_manifest` exports runtime env, installs egress defaults, plans reconciliation, seeds the projection, then reconciles, in that order.
- 1136 Capability reconciliation is declarative over `source='manifest'` rows and additive over control-plane grants, scoped org-wide.
- 1137 A manifest declaring no capabilities, or dropping more than `max(3, half)` of the active manifest-sourced set, aborts the whole apply before any write unless explicitly confirmed.
- 1138 A manifest adapter whose module cannot be imported is logged and skipped so boot continues.
- 1139 Manifest adapter credentials are seeded as `{store, ref, kind}` references and bound to the adapter, never as secret material.
- 1140 The tenant verb ceiling is the union of role-mapping grants, with `*` for org-admin, and always adds `agent.send` and `chat.present`.
- 1141 `export_runtime_environment` fills only environment keys that are absent, so an explicit process value always wins.
- 1142 `Settings` is a frozen dataclass built from the environment by `load_settings`, with no settings framework and no singleton.
- 1143 Production classification accepts `BOLTRIG_PRODUCTION` or `prod/production/staging` on ENV, `BOLTRIG_ENV` or `APP_ENV`; development classification requires an affirmative `dev/development/local/test` and never infers from absence.

- 1144 A placeholder audit HMAC key under a production signal is a fatal boot failure, and a warning on every boot otherwise.
- 1145 An unset or default credential sealing key under a production signal is a fatal failure; otherwise the in-source dev key is used.
- 1146 The audit HMAC key is bound at module import, so a later environment change is not observed by the chain.
- 1147 One predicate answers whether a value is a placeholder secret, listing every placeholder the repository has shipped plus four fragments and a 24-character floor.
- 1148 `boltrig doctor` fails when `DATABASE_URL` still contains a placeholder fragment, regardless of environment.
- 1149 Auth resolver precedence is session, then Cloudflare Access, then generic OIDC, then dev auth, then a deny-all resolver.
- 1150 A partial manifest OIDC trio, or a manifest trio that differs from a fully configured process trio, refuses composition.
- 1151 Thirty-seven `BOLTRIG_*` variables read by shipped code appear in no operator-facing configuration file.
- 1152 `BOLTRIG_RELEASE_MODE` admits exactly `core` or `full` with no trimming or case folding, and absence is preserved as no override.
- 1153 The release Compose overlay binds the release mode on kernel, fleet-worker, browser-executor and hatchet-worker, and the validator refuses a missing, decorated or disagreeing binding.
- 1154 Release mode `core` disables the Codex readiness check, and `core` together with `BOLTRIG_CODEX_TRUSTED` is a hard readiness failure.
- 1155 A `full` release with no packaged-desktop origin in `BOLTRIG_CORS_ORIGINS` is a doctor failure.
- 1156 The release workflow binds the mode, the desktop posture and the runtime admissions into an immutable `release-metadata.json` asset and refuses to change it on a re-run.
- 1157 The product name is derived from the active addon set at runtime, never from a build flag.
- 1158 `boltrig config-validate` parses a manifest with the shipping loader, exiting 0 on success, 1 on rejection with the exception type named, and 2 on a missing path.
- 1159 `config-validate` cannot detect an unset `${VAR}`, because interpolation substitutes the empty string rather than failing.
- 1160 `spawn_rules` is the only manifest section whose Admin Console revision reaches runtime behaviour; every other section edit is export and history only.
- 1162 The shipped example manifest declares a 3600-second approval window, one twenty-fourth of the code floor, and `genesis.sh` copies it verbatim to `manifest.yaml`.

AREA 12

Observability, readiness, doctor and log safety

98 requirements

- 1200 Every kernel action writes exactly one audit row in the same logical step as its effect, chained to the previous row's hash per tenant.
- 1201 The audit row hash is HMAC-SHA256 over a stable sorted-key JSON serialisation of the chain fields under the process audit key.
- 1202 The five Opbox-depth fields enter the canonical body only when non-None, so a pre-enrichment row canonicalises byte-for-byte as before.
- 1203 A detail string matching a secret pattern is replaced entire by a sha256 digest, size and scrubbed marker.
- 1204 A detail string matching only an identity pattern has the matched span substituted and the rest left legible.
- 1205 A secret-bearing dict KEY collapses to the stable string marker and is never copied verbatim into the append-only store.
- 1206 The scrub recurses through nested dicts and through list and tuple values.
- 1207 Every scrubbed string value is truncated to 256 characters.
- 1208 The raw user_agent header is scrubbed to a bounded string before it enters the hash chain and stays a string so the column shape is unchanged.
- 1209 The chain step is serialised by one asyncio lock per tenant, with the Postgres UNIQUE(tenant_id, seq) as the multi-process backstop.
- 1210 An audit append fault defers the scrubbed payload to the audit outbox and returns the event with seq, prev_hash and hash set to None.
- 1211 The outbox deferral marker records only the exception type name, never its message.
- 1212 write_now has no deferral branch, so the outbox janitor cannot fork a second outbox row for an event that already has one.
- 1213 Chain verification re-derives the entire per-tenant chain from seq 1 through the ascending scan seam, never a bounded tail window.
- 1214 A scan page whose last seq does not advance the cursor terminates verification as a failure rather than looping forever.
- 1215 Exactly one key verifies each row: the first retired epoch whose boundary the row predates, otherwise the live key.
- 1216 A malformed retired-key epoch entry is ignored, so its rows fall through to the live key and fail honestly rather than being silently accepted.
- 1217 Retired epochs are parsed once per verification rather than at import, so a rotation is honoured on the read side without a restart.
- 1218 The write-side audit key is bound at module import, so rotating BOLTRIG_AUDIT_HMAC_KEY requires a process restart.
- 1219 The rollup anchor seals and verifies under key_in_force_at(seq_end), so write and read agree by construction across a rotation.

- 1220 boltrig audit-verify exits 0 on a verifying chain, 1 on a bad row, and 2 when it could not look, never reporting success it did not establish.
- 1221 A segment verification refuses to run when the seeding row before the segment does not exist, exiting 2 rather than seeding prev as None.
- 1222 A segment verification prints the skipped range on every run.
- 1223 GET /v1/audit/verify is author-or-admin gated and reports chain, security-chain and anchor integrity as separate fields plus a combined verdict.
- 1224 verify_latest reports anchor_intact true when the tenant has no anchor row at all.
- 1225 The anchor verification path loads the newest one million audit rows into process memory on every call.
- 1226 POST /v1/audit/export is author-gated, bounded at 100000 rows, and writes its own keys-only disclosure audit row.
- 1227 audit_query is a tail read ordered by seq descending under a limit and returned ascending, distinct from the ascending audit_scan verification seam.
- 1228 The execution tree is assembled only from rows the caller may see, with a hidden parent link nulled rather than followed.
- 1229 Tree assembly is cycle-guarded and depth-capped at 256, emitting a degraded marker rather than recursing.
- 1230 The audit tree route reads at most the newest 10000 audit rows, so a root older than that window is an indistinguishable 404.
- 1232 Model telemetry is aggregated from audit rows into provider/model/runtime/profile buckets and capped at 200 rows.
- 1233 No metrics or trace exporter exists; every quantitative figure is reconstructed at read time from audit rows.
- 1234 The Langfuse mirror runs only after the audit row is persisted and cannot block or fail a spawn.
- 1235 A Langfuse emit is bounded by a 0.25 second timeout and every exception increments a failure counter and returns.
- 1236 The Langfuse payload carries only allow-listed route keys and bounded metadata, never a prompt, output, URL or key.
- 1237 A misconfigured Langfuse sink degrades to a typed no-op carrying a reason, never an error.
- 1238 Langfuse delivery is projected as process-local attempt counters for one spawner and never as sink health or delivery lag.
- 1239 An unrecognised sink_state or reason collapses the Langfuse projection to unavailable.
- 1240 log_safe escapes every C0 and C1 control character, DEL, and the Unicode line separators so a caller-chosen value cannot forge a second log record.
- 1241 log_safe truncates at 200 characters and states the original length rather than truncating silently.
- 1242 Both egress refusal log calls pass every format argument through log_safe, asserted structurally by an AST walk that also asserts the call count.
- 1243 log_safety imports nothing from the rest of the package so kernel and adapters share one copy of the rule.

- 1244 Untrusted spans are wrapped in a typed envelope whose literal untrusted delimiters are defanged inside the content.
- 1245 The envelope delimiter matcher tolerates zero-width, soft-hyphen and BOM gaps so an invisible character cannot forge a delimiter.
- 1246 Envelope attribute values are stripped of angle brackets, quotes, CR and LF.
- 1247 Every boltrig process installs one root logging handler with a formatter carrying timestamp, level name and logger name.
- 1248 An unreadable BOLTRIG_LOG_LEVEL falls back to INFO rather than silencing the process.
- 1249 Both the ASGI and worker entrypoints call `configure_logging` at import and neither calls `basicConfig` itself.
- 1250 boltrig doctor performs static checks with no dependency I/O and exits 1 if and only if at least one check is fail.
- 1251 Doctor production mode is inferred from `BOLTRIG_PRODUCTION` or from `ENV`, `BOLTRIG_ENV` or `APP_ENV` in prod, production or staging, or forced by `--production`.
- 1252 Doctor fails any run whose `DATABASE_URL` contains a shipped placeholder fragment, in production or not.
- 1253 Doctor fails a production run whose audit HMAC key is placeholder-like or shorter than 32 characters.
- 1254 Doctor fails `BOLTRIG_DEV_AUTH` under a production signal.
- 1255 Doctor fails a wildcard or unset host allowlist under production and any wildcard CORS origin in any mode.
- 1256 A full release mode with no exact packaged-desktop Tauri origin in `BOLTRIG_CORS_ORIGINS` fails.
- 1257 Doctor warns for every retired runtime name a tenant manifest still enables.
- 1258 Doctor derives the application and Hatchet database names without reflecting any DSN credential and fails when the configured sources disagree.
- 1259 Production doctor fails an unset `BACKUP_REMOTE` or an unset `BACKUP_PASSPHRASE`.
- 1260 Production doctor fails a Browser CLI state root or binary that references a user home or personal tool state.
- 1261 One shared predicate decides whether a tenant needs the browser CLI, consumed by doctor, the fleet entrypoint and readiness.
- 1262 The doctor codex check fails under production whenever the static release posture is not ready.
- 1263 Doctor never fails on Hatchet: an absent durable engine is at most a warning.
- 1264 With no manifest path, doctor records one warn and skips every manifest-gated check family without changing the exit code.
- 1266 `/healthz` answers from the cached adapter posture without awaiting adapter I/O and its top-level status is the literal ok.
- 1267 A stale adapter-health cache schedules exactly one bounded background re-probe off the request path.
- 1268 Each adapter health probe is cut off after 2.5 seconds and recorded down rather than hanging the caller.

- 1269 /readyz is unauthenticated, coalesces concurrent probes under one lock, and serves a deep copy of a briefly cached redacted result.
- 1270 /readyz returns 503 whenever any required check is not exactly ok.
- 1271 Readiness requires the packaged Alembic head to be the only applied head, compared as an exact tuple.
- 1272 Production readiness refuses a process-local event relay with reason wrong_backend before any Redis probe runs.
- 1273 Control-plane readiness requires live registration, persisted verbs, control-owned bindings, an activated adapter record and five wired collaborators.
- 1274 Readiness requires the browser stack tool only where the tenant manifest declares browser automation.
- 1275 The live stack-tool check requires a fresh HMAC-authenticated fleet receipt and maps any unrecognised probe reason to unavailable.
- 1276 A configured but unprobed model gateway reports unchecked rather than disabled and remains not required.
- 1277 Hatchet readiness is required only when the health flag, the durability flag or a client token is configured, and a configured-but-unreachable engine fails closed.
- 1278 Password-reset delivery readiness is required only under its own flag and always reports provider_delivery_proven false.
- 1279 Background-job readiness entries are always not required and can never make readiness not_ready.
- 1280 An unreadable or timed-out background receipt read degrades every job to attempt_evidence_unavailable rather than failing readiness.
- 1281 A background receipt row whose job name this build does not know is skipped rather than taking the whole readiness read down.
- 1282 A background attempt receipt write failure is logged at warning and swallowed so it can never change the janitor outcome.
- 1283 Boot aborts with a fatal RuntimeError when dev auth is selected under any production signal.
- 1284 Boot aborts with a fatal RuntimeError when the audit HMAC key is a known shipped placeholder under any production signal.
- 1285 With no production signal a placeholder audit key logs a warning every boot and does not abort.
- 1286 refuse_dev_auth_in_prod tests only the production signal; its dev-auth precondition is established by its single caller.
- 1287 One placeholder predicate is shared by the doctor, the boot guard and the readiness receipt key so the three sites cannot disagree.
- 1288 boltrig fleet-health exits 0 with an explicit NOT CHECKED message when the heartbeat is disabled by configuration and no receipt can exist.
- 1289 BOLTRIG_FLEET_HEALTH_REQUIRE_RECEIPT converts both NOT CHECKED branches of fleet-health into exit 1.
- 1290 fleet-health treats a merely present receipt as insufficient: it must also be fresh within the configured TTL and authentic for this tenant.

- 1291 GET /v1/platform/status requires an authenticated principal and returns only bounded, redacted projections.
- 1292 Any component metadata key containing a secret-like fragment is dropped and any http or https string value is nulled before projection.
- 1293 The network-policy projection returns no proxy URL, CA path, domain value or provider endpoint and names each surface's coverage limitation.
- 1294 The identity-policy projection returns a mode, a sha256 generation digest and no issuer, audience or JWKS value.
- 1295 The Codex admission projection cannot report ready in this build because all three inputs are false or non-empty constants.
- 1296 Memory-projection receipts carry opaque per-tenant digests, no content, at most 50 rows, and an explicit truncated flag.
- 1297 GET /v1/backup/status reads only the sidecar's bounded atomic epoch marker, never a backup artifact, and never claims restore readiness.
- 1298 A marker file over 32 bytes, non-ascii, non-digit or dated in the future is classified `invalid_marker` rather than parsed.
- 1299 The health-claim gate fails any first-party service whose healthcheck probes liveness on an application that has a readiness route, and its waiver file is empty.

AREA 13

Process composition, bootstrap and the CLI surfaces

86 requirements

- 1300 Each serving process composes exactly one manifest snapshot, one trusted Codex configuration and one Bifrost model catalogue, through `compose_process_model_runtime`.
- 1301 The manifest's non-secret runtime environment is exported before the Codex configuration and the catalogue are built, and that export fills only variables the process environment leaves absent.
- 1302 `compose_api_app` injects the identical `codex_config` object into the kernel factory, the chat wiring, the platform factory and the app spawner.
- 1303 `build_kernel_async` never re-reads the manifest when the caller injects a snapshot.
- 1304 `build_kernel_async` refuses to boot when an injected sensitive endpoint id disagrees with the snapshot's configured sensitive endpoint.
- 1305 The audit-key boot guard runs as the first statement of `build_kernel_async`, before any store connection.
- 1306 `build_store` returns a durable `PostgresStore` when `DATABASE_URL` is set and an in-memory store otherwise.
- 1307 No runtime boot applies `schema.sql`: every `PostgresStore` connection this area opens passes `apply_schema=False`, with or without RLS.
- 1308 The shared rate-limit counter and the event relay are both constructed from `REDIS_URL`, and the relay is told whether a production signal is present.
- 1309 A manifest boot constructs the Kernel with the manifest's blocking-verb set, its HITL approval timeout and its development posture.
- 1310 A boot that finds no manifest seeds a minimal offline demo tenant named 'default' rather than failing.
- 1311 The manifest seed registers `web.fetch` before rehydrating store-persisted adapters, so the web row is never warned about and then registered.
- 1312 Adapter rehydration skips ids the manifest already registered this boot and emits a shape-specific warning for every row it cannot reconstruct.
- 1313 Skill-directory loading is best-effort: a bad skill file warns and the boot continues.
- 1314 The pending desktop-command registry is created, and the `desktop.*` verbs registered, only when `BOLTRIG_DESKTOP_HANDS` is truthy.
- 1315 External MCP servers declared in the manifest are registered inert, pending the review gate, and their bearers bind by credential reference rather than raw material.
- 1316 The agent invoker is set last in kernel construction and carries the same provider, catalogue and sensitive role as every spawner the process owns.
- 1317 `asgi.py` configures root logging before importing anything else from `boltrig`.
- 1318 Both the ASGI entrypoint and the fleet worker resolve the configured add-on set at import, so an unregistered name is a startup failure rather than a mid-turn one.
- 1319 The FastAPI object is built at import while the kernel, spawner, chat service and platform bag are built inside the ASGI lifespan on the serving loop.

- 1320 The API kernel factory publishes an 'api' birth receipt only when the desired-state overlay of the snapshot is available, and publishes nothing when it is not.
- 1321 `select_principal_resolver` refuses to boot on a partial manifest OIDC trio and on a manifest trio that differs from a simultaneously configured process trio.
- 1322 With no authentication configured, the selected resolver refuses every request with 401 rather than falling through to a permissive default.
- 1323 `make_platform_factory` binds every immutable boot input by partial application and leaves only the kernel to be supplied at lifespan.
- 1324 The platform factory registers the durable workers and the Boltrig task definitions, and a registration fault warns without failing the boot.
- 1325 The platform projection carries composition truth only: a configured boolean and the non-secret model id, never provider topology, URLs or upstream keys.
- 1326 `ReadinessService` is constructed with the process manifest snapshot, the selected executor, the stack-tool status provider and the password-reset delivery pair.
- 1327 `worker.py` configures logging and resolves add-ons at module import, using the same declared configuration as the ASGI entrypoint and rolling none of its own.
- 1328 The fleet worker composes the same one-snapshot runtime as the API and builds its kernel through `build_kernel_async` on the running loop.
- 1329 A failed desired-fleet overlay degrades the worker to the minimal default org and never crashes the boot.
- 1330 The worker records a permanent-fleet startup observation only when the desired overlay actually applied.
- 1331 The worker publishes a 'fleet' birth receipt over the overlaid manifest, its own add-on snapshot and the process Codex configuration.
- 1332 The fleet worker starts seven named background loops: audit-anchor, hitl-expiry, audit-out-box, fleet stack-tool heartbeat, retention, workflow-scheduler and session-distillation.
- 1333 Every janitor is disabled only by an explicit interval of zero or less, and the worker logs which choice it made so 'off' is a decision on the record.
- 1334 The fleet stack-tool heartbeat starts only when `REDIS_URL` is configured, and the worker logs which case applied.
- 1335 When the delegation pump loop ends, every started background task is cancelled and gathered and the kernel is closed.
- 1336 The fleet worker installs its own executor-only HITL resume notifier so the expiry sweep's reconciliation pass has a notifier to re-fire.
- 1337 A `KeyboardInterrupt` stops the fleet worker cleanly with a log line rather than a traceback.
- 1338 The distribution installs exactly one console script, `boltrig`, pointing at `boltrig.api.cli:main`.
- 1339 The CLI declares thirteen subcommands, requires one to be given, and dispatches every declared name to a real module.
- 1340 `boltrig serve` binds 127.0.0.1 unless an explicit `--host` is given.
- 1341 The three identity subcommands resolve their tenant identically: the flag, else the session tenant, else 'default'.

- 1342 boltrig smoke and boltrig check-invariants resolve a script from the source tree and exit 2 when it is absent, which is the case inside both shipped images because neither Dockerfile copies scripts/.
- 1343 boltrig doctor merges an optional dotenv over a copy of the process environment before running the static checks.
- 1344 The CLI subcommand set is closed against the non-HTTP feature ledger, so a new subcommand cannot ship without a discover/configure/operate/observe/recover classification.
- 1345 boltrig chat is a thin client: head mode streams POST /v1/chat over SSE with a bearer PAT and embeds no agent runtime.
- 1346 The chat token resolves flag, then BOLTRIG_CLI_TOKEN, then the config file, and never appears in an error message.
- 1347 A malformed CLI config file is a clean error, never a silent fall-through to 'no token configured'.
- 1348 The terminal SSE parser drops comments and malformed or non-object payloads without failing the turn.
- 1349 A 202 accepted-steer response becomes a terminal queued event and any other non-200 becomes a one-line clean error.
- 1350 Gateway mode validates a target slug against the kernel's own slug rule and drops malformed inbound frames.
- 1351 boltrig initiate seats exactly one founding OWNER and refuses to run again once any owner-tier user exists in the tenant, touching nothing on refusal.
- 1352 The seeded founding owner is flagged `must_change_password` so the provisioning credential cannot remain the live production gate.
- 1353 Both host-boundary password commands bind the current tenant before any RLS-scoped read or write.
- 1354 The initiate audit row carries the email, role, org and workspace and never the password.
- 1355 The initiate run-once guard is a read-then-write with no transaction and no uniqueness constraint, so two concurrent runs with different emails both seat a superadmin.
- 1356 boltrig set-password refuses a user that does not exist and never creates an identity or a grant.
- 1357 A successful set-password clears `must_change_password`, so rotating the credential discharges the forced-rotation clamp.
- 1358 The set-password audit row attributes the act to the host boundary with the subject in `on_behalf_of`, and a matching security-stream event is written best-effort.
- 1359 Both password commands resolve the secret as the flag, then BOLTRIG_INIT_PASSWORD, then a confirmed interactive prompt, and exit 2 on a mismatch.
- 1360 `wire_hitl_resume` installs exactly one resume notifier on the kernel HITL manager, and the kernel never imports the fleet to do it.
- 1361 The resume notifier runs four independent legs in a fixed order and each fails safe, so a leg fault never voids the recorded answer.
- 1362 The held-write leg does nothing for a non-APPROVAL request, a request with no run id, or an absent resume callable.

- 1363 The held-write leg defers to origin finalization when replaying the write would discard a one-time webhook secret.
- 1364 A request that another lane also paused on is not replayed by the held-write leg, so the interpreter and held-write lanes never race the approval CAS.
- 1365 Delivering the resume twice, from the notifier and from either run id, executes the held write exactly once and leaves exactly one consumed approval and one tool-call row.
- 1366 A missing or unreadable held-call seal refuses the resume and never reconstructs the call from anything else.
- 1367 The API process wires the executor and held-write legs of the answer bridge and does not wire the pump leg.
- 1368 The chat factory and the held-write resume callable share one ChatService instance through a late-bound holder, so factory ordering in the lifespan does not matter.
- 1369 The API chat factory re-reads the manifest from disk at lifespan time and silently falls back to the default ChatConfig when that read fails.
- 1370 A HITL approval verdict is harvested as a governed endorsement or block reuse signal, best-effort, and a harvest fault never voids the answer.
- 1371 boltrig audit-verify exits 0 when the chain re-derives, 1 when it does not, and 2 when it could not look, and 2 is never reported as success.
- 1372 Segment verification seeds the walk from the row before the window and prints the range it did not check.
- 1373 Segment verification still names tampering that occurs above the cut.
- 1374 The verifier names the number and boundaries of retired key epochs on every outcome, and its failure text names both possible causes.
- 1376 register_agent_support registers the skill shelf, the work-read adapter, peer messaging and chat presentation, and it runs on both the demo and the manifest seed paths.
- 1377 A tenant with no named agents is seeded a 'general' script agent marked as the intake default.
- 1378 Progressive disclosure of SKILLS is the shelf's search verb, which returns descriptions only and never the prompt fragment body.
- 1379 Progressive disclosure of TOOLS is computed inside the kernel and is no part of the composition root, which registers adapters and never ranks or budgets an offer.
- 1380 Device action verbs are registered only when this process holds a lease signing key, and are absent otherwise.
- 1381 Camera lease verbs follow the same key gate as device verbs and are absent when no signer is available.
- 1382 Every serving process publishes one bounded birth receipt whose fields are opaque digests, and a write failure warns and returns False without failing the boot.
- 1383 Exactly three process kinds may publish a birth receipt, api, fleet and hatchet, and the database enforces that set.
- 1384 One declared logging configuration installs a root handler whose formatter carries a timestamp, a level name and a logger name.
- 1385 An unreadable BOLTRIG_LOG_LEVEL falls back to INFO rather than silencing the process.

1386 Only two of the four shipped python entrypoints configure logging; `boltrig.fleet.hatchet_worker` and `boltrig.fleet.browser_executor` configure none, and the gate that enforces the rule checks only `asgi.py` and `worker.py`.

AREA 14

Presence, devices, camera, emotion and the Familiar

97 requirements

- 1400 Device enrollment start refuses any actor whose tier is not human.
- 1401 An enrollment authorization code is an opaque scoped token whose sha256 alone is stored, expiring after ten minutes.
- 1402 Completing enrollment consumes the code atomically and returns a session token plus the Ed25519 lease verifier.
- 1403 A device session bearer is validated by scoped-token kind, device id equality and stored digest, and expires after 24 hours.
- 1404 Session rotation authenticates the live session first, so a lapsed token can never be exchanged for a live one.
- 1405 Revoking a device sets `revoked_at`, sets presence to revoked and NULLs both session columns.
- 1406 A file write lease is refused on a read-only root and a command lease is refused on a root without `command_enabled`.
- 1407 A device lease expires 120 seconds after issue.
- 1408 A device lease materializes only against a CONSUMED APPROVAL HITL request whose action digest matches and whose respondent is neither the requester nor the on-behalf subject.
- 1409 One approval can materialize at most one device lease, enforced by a UNIQUE (`tenant_id`, `approval_id`).
- 1410 Claiming a lease requires the exact stored signature, verifies it with the kernel signer, and is a single-use compare-and-set with a five minute claim window.
- 1411 Settling requires status claimed, a matching claim token digest and an unexpired claim, and accepts only completed or failed.
- 1412 A settlement receipt is rejected, never truncated, above 32000 canonical bytes.
- 1413 The owner-facing lease view projects only bounded, validated receipt metadata and pins command output_captured to false.
- 1414 Device and camera verbs are registered only when a lease signing key is present in the environment.
- 1415 `device.command.run` accepts an argv array only; no schema field anywhere on the device lease path carries a shell string.
- 1416 All four device verbs are HIGH consequence, rate limited to 30 per minute per tenant.
- 1417 A camera lease expires 120 seconds after issue and the store refuses an expiry beyond that window.
- 1418 A camera lease carries no root, no path, no argv and no command field in its canonical payload.
- 1419 Camera bindings are published only over an authenticated device session and the store re-derives `owner_id` from the devices row.

- 1420 A binding claiming `ptz_set_state` proven without the bounded set-readback-restoration evidence string is refused.
- 1421 Camera lease issuance requires a connected binding, a readable or proven get state, a proven set state for `camera.ptz.set`, and a descriptor fingerprint equal to the action's.
- 1422 Camera lease materialization applies the same consumed, non-self, exact-digest approval test as the device path.
- 1423 One approval can materialize at most one camera lease, enforced in SQL and by a UNIQUE (`tenant_id`, `approval_id`).
- 1424 Camera lease claim and settlement use the same signature check, claim token digest and terminal status set as the device path.
- 1425 Owner-facing camera binding and lease reads require actor tier human.
- 1426 The camera lease adapter is the adapter registered at bootstrap; the read-only camera discovery adapter is not.
- 1427 The device-side verifier independently refuses a camera lease whose validity window exceeds three minutes or whose `issued_at` is more than thirty seconds ahead.
- 1428 The device re-checks the live camera descriptor fingerprint against the lease before performing any UVC operation.
- 1429 The shipped camera identity and descriptor fingerprint are both sha256 over a USB topology string, and the camera id is its first 32 hex characters.
- 1430 The Python and Rust camera action digests agree, pinned by a shared fixture value.
- 1431 The device agent holds at most one pending camera claim and persists it before executing, so a crash still settles on the next cycle.
- 1432 The six sensing settings keys are owned by `sensing_policy` and refused by the generic settings write route.
- 1433 A fresh Boltrig has both the camera and presence off, and reports the difference between a safe default and a user override.
- 1434 Every sensing write requires an interactive human credential, not merely a human actor tier.
- 1435 Binding a camera that no device agent has published is refused at write time with 409 `camera_binding_unavailable` and the attempt is audited.
- 1436 Presence cannot be enabled without a room-calibrated threshold, and the served config never reports presence enabled without one.
- 1437 The enrolled face is stored as metadata only, is named as never leaving the kernel, and is reported as not exportable on every projection.
- 1438 The effective capture policy is served to a device-authenticated caller and answered for that device's owner, never for another principal.
- 1439 A refused sensing capability answers status refused with a named reason and 409, never 403 and never an error.
- 1440 The sensing capability route governs consent only; it carries no character identity and does not enforce declaration.
- 1441 The served config carries a maximum staleness of 3600 seconds after which a host stands down without releasing the device.
- 1442 The kernel's only emotion touch is the relay factory call in `Kernel.__init__`.

- 1443 No exception in the emotion path can escape publish, the publisher thread, table loading or the relay factory.
- 1444 The emotion engine reads no clock, performs no I/O and uses no randomness; time enters only through now arguments.
- 1445 Tension decays with a real 1.4 second half-life and is never tempo-scaled.
- 1446 Attachment accumulates only from appraisal deltas, decays on a real-days half-life, and affects only the effective baselines emotions decay toward.
- 1447 The phenotype projection is exactly ten scalars, each clamped to 0..1, derived and never stored.
- 1448 The emotion model, appraisals and event map are YAML data, and any load failure leaves the feature off rather than raising.
- 1449 The emotion relay is enabled by BOLTRIG_EMOTION, falling back to whether orbctl is on PATH.
- 1450 Both emotion files are written through a unique mkstemp, chmod 0644 and os.replace, never through a fixed temporary name.
- 1451 The publisher thread writes the phenotype every tick and the tenant state snapshot every twentieth tick.
- 1452 An emotion reset replaces the tenant engine at model baselines and drops the persisted restore snapshot so a restart cannot resurrect the old mood.
- 1453 A character adoption is appraised at most once per sixty seconds per tenant, so cycling skins cannot pump the mood.
- 1454 Every failure of the phenotype route answers the resting shape with fresh false, never an error status.
- 1455 The phenotype route module imports nothing from boltrig.emotion.
- 1456 The phenotype route reads a process-wide runtime file and does not scope its answer by the authenticated principal.
- 1457 familiar.express is a registered verb with a closed eight-member gesture enum, additional-Properties false, and a ttl capped at fifteen seconds.
- 1458 The express channel is written only by the dispatched handler, and an ungranted or schema-invalid call writes nothing.
- 1459 The desktop hands verbs, the shared registry and the /v1/hands surface exist only when BOLTRIG_DESKTOP_HANDS is set.
- 1460 A pending desktop command expires after thirty seconds and is claimed mark-on-read, so it can never be delivered to two polls.
- 1461 A desktop dispatch waits eight seconds for its receipt, strictly less than the registry TTL, and reports executor_offline on timeout with the audit row standing.
- 1462 A receipt for an unknown or expired command id returns 404 and writes no audit row.
- 1463 The hands registry is kernel-global and applies no tenant partition to the pull surface.
- 1464 The Familiar carries six modes and resolves them by the fixed precedence failure, speaking, listening, working, thinking, standby.
- 1465 FamiliarStageState carries only a closed mode enum and clamped 0..1 numbers, accepting a bands array only at length eight.

- 1466 The Familiar's host-side behaviour is a 24 field numeric tuning struct with per-mode deltas layered over it.
- 1467 The Stage falls back to the CSS badge the moment the WebGL2 renderer reports failed, and never rewrites the look to survive a failure.
- 1468 The Stage announces its state through an aria-live region that is a sibling of the role=img body, never nested inside it.
- 1469 A stale or absent phenotype is handed to the renderer as null, so the inner life resumes wandering rather than holding a stale expression.
- 1470 The renderer ladder ships two rungs, the CSS badge and webgl2; the Unreal tier is deferred and absent from the tree.
- 1471 The Familiar genotype is derived from the canonical capability name alone and carries no runtime, grant, model or mood state.
- 1472 The genotype packs into a positional 32 slot uniform array whose defaults reproduce the neutral circle, and appending is the only safe change.
- 1473 The Worker's vendored familiar.frag is byte-identical to the in-repo canon and to the sha256 pinned by its bundle manifest.
- 1475 The desktop familiar consumes nine phenotype scalars and does not read the attachment scalar.
- 1476 The desktop familiar surface imports nothing from boltrig and couples only through the versioned phenotype file, checked statically.
- 1477 Absence of a compositor, a GPU or a phenotype degrades the desktop familiar to a typed message or the resting baseline, never a crash.
- 1478 make install runs the severance and layer-hygiene check, so a build that would cover the KVM capture strip cannot be installed.
- 1479 BOLTRIG_ADDONS naming an unregistered addon raises rather than shipping a build with no integration seam.
- 1480 An addon harness fragment is bounded to 4096 bytes so it cannot push the governance floor out of the model's attention.
- 1481 At most one active addon may claim the on-behalf adapter id; two is refused rather than resolved by picking one.
- 1482 Consequence hints are read from every registered addon and the highest tier wins, so one addon's low can never mask another's high.
- 1483 An installed package's entry-point addon may not replace a name this build already registers.
- 1484 The pinned birth profile version composes the active addons as semver build metadata rather than forking a second lineage.
- 1485 The product name is derived at runtime from the active addons and served at GET /v1/ branding, never baked into a client bundle.
- 1486 The addon runtime inventory is read-only, tenant and workspace scoped, and never serializes harness text, refs, values or evaluator faults.
- 1487 A camera profile is declarative TOML with a closed top-level key set and is refused if it carries any command, exec, module, script, download or url key.

- 1488 A profile match is a routing hint only; vendor metadata stays inactive unless the profile id is in an explicit trusted set.
- 1489 The camera knowledge cache stores only a digest of the platform hardware key and never a raw serial, registry path or probe payload.
- 1490 Discovery grants a mutating camera verb only where the current evidence proves it, and an unknown HID camera never gains one.
- 1491 The read-only camera discovery adapter refuses camera.snapshot unless the capability state is proven.
- 1494 The device lease verb constraint admits device.file.list only from migration 0072, whose downgrade refuses while such leases exist.
- 1495 Every device and camera table enables and forces row level security with a tenant_isolation policy on app.tenant_id.
- 1496 Desktop hands verbs that change host state carry idempotency mode disabled so a replayed receipt cannot claim an undelivered action.
- 1497 The camera lease verbs are declared idempotency mode cacheable, unlike the state-changing desktop verbs.
- 1498 The face enrolment is published by the host agent over the device transport and forgotten by the human over the owner routes; there is no browser route that records one.
- 1499 Deployment shape for these surfaces is expressed by provisioning and runtime signals rather than by build-time flags.

AREA 15

The worker UI, the primary surface

58 requirements

- 1500 WorkerRoute is a closed set of sixteen ids and any hash naming none of them resolves to the chat route.
- 1501 A route selection id is rejected unless it decodes to at most 256 characters and occupies exactly the second hash segment.
- 1502 Navigation is a window.location.hash write and every route listener is a hashchange handler; no router library and no history API are used.
- 1503 Every route component and the command palette load as React.lazy chunks behind one Suspense boundary.
- 1504 The primary navigation rail exposes five of the sixteen routes; the other eleven are reachable only by hash, palette or in-page link.
- 1505 The command palette carries a destination row for all sixteen routes and every canonical settings section.
- 1506 The chat route mounts LocalChatView under a Tauri runtime and ChatView otherwise, and the two lanes never fall through to one another.
- 1507 The account route renders the settings You pane rather than an account view.
- 1514 All backend access goes through one BoltrigClient instance; exactly one direct fetch exists and it is inert without VITE_SELF_HOSTED_TTS_ORIGIN.
- 1515 Interactive session authority is the httpOnly cookie plus a double-submit CSRF token sent only on mutating methods.
- 1516 On the desktop shell the four session-issuing SDK calls are replaced by origin-pinned native commands while the rest of the surface is unchanged.
- 1517 The desktop fetch shim refuses any URL whose origin is not the configured one or whose path is not under /v1, before invoking native code.
- 1518 Every native response-envelope parse failure collapses to one opaque reason and no partial response is constructed.
- 1519 A desktop build refuses to start unless VITE_API_BASE and BOLTRIG_DESKTOP_API_ORIGIN are both present, canonical, secure and identical.
- 1520 A desktop bundle with no configured API origin renders a named refusal instead of a sign-in form.
- 1521 The auth gate probes the existing session and routes only two typed 403 details to their own screens; every other failure becomes unauthenticated.
- 1522 The session is rotated every four hours and on every return to visibility, and a 401 re-probes the session before evicting it.
- 1523 A failed sign-in reports one generic message that does not distinguish a wrong credential from an unreachable server.
- 1524 The desktop account bridge fails open to ready when the device list is temporarily unreachable, so a transient failure cannot destroy a valid local key.
- 1525 Onboarding is gated on one kernel settings key equalling one integer version.

- 1526 The live chat transport is server-sent events read from a fetch body; the only WebSocket in the Worker belongs to realtime voice.
- 1527 A chat send has exactly two outcomes: a 202 queue receipt with no stream, or a 200 SSE stream.
- 1528 The follow stream rejects a frame with a malformed cursor or no event, and rejects a cursor that regresses, ending the stream rather than accepting it.
- 1529 A dropped live follow is recovered only by an explicit Reconnect control that replays from the last observed cursor; there is no automatic retry and no backoff.
- 1530 A follow against a conversation with no active run returns an idle result carrying the unchanged cursor rather than raising an error.
- 1531 A truncated replay window is surfaced to the reader as an explicit continuity notice rather than silently dropped.
- 1532 A route generation counter fences every async chat continuation so an aborted stream cannot be read as a completed turn.
- 1533 The kernel creates the call before the browser asks for microphone permission.
- 1534 A realtime_unavailable call status adopts the text-continuation conversation and opens no socket and no microphone.
- 1535 The voice media bearer is delivered in the first WebSocket frame and never appears in the socket URL.
- 1536 The voice socket URL is resolved against the document in the browser and against the configured API origin in the desktop shell.
- 1537 A voice session that does not become ready within fifteen seconds releases the microphone and reports the release.
- 1538 Voice close codes 4429, 4401 and 1013 map to distinct product messages and distinct terminal or reconnectable states.
- 1539 Voice reconnect is a user action that always mints a fresh session through reopenCall; a spent media bearer is never reused.
- 1540 Text-chat reply speech is opt-in and reaches the provider only through the governed voice.speak verb on the ordinary dispatch route.
- 1541 Twenty-six modules share one exact-approval finalizer that clones the route input, re-reads the approval state before replay, and never treats consumed as applied.
- 1542 A governed response body carrying no run_id and a denied, error, unavailable or degraded status is decoded as a refusal and never as a success.
- 1543 WorkerGlobalContext is the only React context in the application and carries identity plus a three-state load status.
- 1544 There is no shared query cache and no store library; every route view owns its fetch, its load state and its staleness fencing.
- 1545 The per-surface load-state union is declared five separate times in two different shapes, and two of the five cannot express not-found.
- 1546 Appearance is applied to the document element before React mounts, and a first run with no stored preference and no system hint is dark.
- 1547 The URL page modes theme and embed clamp the rendered palette and chrome for one page load and never write localStorage or kernel settings.

- 1548 The console mount prefix is derived from the document pathname rather than declared, treating a directory, an index.html and an extensionless path as a mount.
- 1549 The design token layer is three CSS files imported from the repository docs tree, and the image build copies exactly those three files.
- 1550 The structural gate measures every TypeScript file under apps/worker/src against five limits and refuses to pass on a corpus of fewer than 100 files.
- 1551 All sixty-one structural-debt exemptions carry the same expiry date, so the whole ratchet lapses in one event.
- 1552 The shipped web image serves a same-origin closed CSP, denies camera and geolocation, allows the microphone for self only, and 404s the retired Operator path.
- 1553 The desktop webview CSP forbids inline style while the web image CSP permits it, and twenty modules use React inline style props.
- 1554 The parity route ledger proves only that a named source file contains the literal client method call, not that any route mounts that file.
- 1555 Desktop-local conversations are stored only in one localStorage value capped at 8 MiB, and a save that would exceed the cap throws rather than evicting.
- 1556 The desktop-local chat surface never calls the hosted chat route and its native events are projected into the same ChatEvent vocabulary as the cloud stream.
- 1557 Transcript markdown is rendered with remark-gfm only, with no raw-HTML plugin and no dangerouslySetInnerHTML anywhere in the source.
- 1558 Exactly three VITE_ variables are read anywhere in the source and no import.meta.env.DEV, MODE or PROD branch exists, so the shipped bundle has no dev-only path.
- 1559 The development bench token guard is not installed when BOLTRIG_BENCH_TOKEN is unset, leaving three dev-only write routes unauthenticated on that server.
- 1561 The Worker quality gate is frozen install, structure, audit, typecheck, vitest and build; it contains no browser-driver stage and no accessibility sweep.
- 1562 The Familiar island is a second build target producing one deterministic self-contained HTML page whose inline script is pinned by SHA-256 in its own CSP.
- 1564 The Worker calls 248 of the SDK client's 258 public methods from 98 of its 335 modules, and eight SDK methods have no call site anywhere in the source.
- 1565 Conversation search is absent from the shell by design and the sidebar is asserted never to call searchConversations.

AREA 16

The SDKs and the marketing site

98 requirements

- 1600 The Node SDK ships as the private, unpublished package `boltrig-app-sdk` at version 0.1.0 requiring Node 20 or later.
- 1601 The Node SDK has zero runtime dependencies and imports only `node:crypto` and `node:http`.
- 1602 `createBoltrigMcpServer` refuses to start when the configured token environment variable is unset.
- 1603 Every JSON-RPC method, including `tools/list` and `initialize`, requires the bearer before any dispatch.
- 1604 An unauthorized call is answered `-32001 "unauthorized"` carried out as HTTP 401.
- 1605 The static token comparison hashes both sides with SHA-256 so it is constant time and length independent.
- 1606 Supplying `resolveIdentity` replaces the static compare, resolves the presented bearer to an identity, and passes `{bearer, identity}` to the handler.
- 1607 Omitting `resolveIdentity` leaves the static-token behaviour byte-identical and passes no context to handlers.
- 1608 `validateVerbTable` refuses an empty table, a bad name charset, a reserved core prefix, a duplicate name, a missing description, a non-object schema and a non-function handler.
- 1609 A verb declaring `implements` with an `@` version pin is refused rather than reinterpreted.
- 1610 `tools/list` omits the `consequence` key entirely for a verb that declares none, so absence travels as absence.
- 1611 The kernel reads a tool carrying no consequence, no add-on risk class and no annotation as HIGH consequence.
- 1612 A successful handler return is wrapped as `{content, isError:false, _boltrig:{status:"ok", output}}`.
- 1613 A thrown `VerbError` becomes `{isError:true, _boltrig:{status, reason}}` and any other throw becomes the fixed reason `"verb handler error"`.
- 1614 `VerbError`'s `"denied"` status reaches no kernel decision: the consumer maps every `isError` result to `ErrorClass.INVALID` and never reads `_boltrig.status`.
- 1615 `VerbDef` cannot declare an output schema and `tools/list` never emits one.
- 1616 The HTTP wrapper accepts POST only, answering 405 with an `allow: POST` header otherwise.
- 1617 The request body is bounded at 1,000,000 characters, answered 413 beyond it.
- 1618 The bearer is read from `x-boltrig-mcp-token` first and from `Authorization: Bearer` second.
- 1619 The server binds `127.0.0.1` by default because an app SDK verb server must not be internet facing.
- 1620 The scaffold answers `initialize` with protocol version `2024-11-05`, mirroring the kernel's own MCP face.
- 1621 `registerMcpServer` sends credential REFERENCES only and has no parameter capable of carrying secret material.

- 1622 Registration and activation return `PendingHuman` on HTTP 202 rather than raising.
- 1623 A previously approved HITL request is replayed as both an `approval_id` body field and an `x-boltrig-approval-id` header.
- 1624 Kernel refusals are surfaced without the token appearing in the message.
- 1625 A login requiring a second factor is mapped to a distinct message directing the caller to a PAT.
- 1626 `baseUrl` trims trailing slashes with a character loop rather than a regex, because a published SDK cannot assume its callers.
- 1627 A missing token resolves through `BOLTRIG_TOKEN` then `BOLTRIG_CLI_TOKEN`, and otherwise raises a `KernelApiError` naming `/v1/me/tokens`.
- 1628 `streamTurn` treats HTTP 202 as an acceptance and yields a synthetic `queued` event instead of reporting a failure.
- 1629 `SseParser` accumulates `data:` lines only and drops comments and malformed or non-object payloads without dying.
- 1630 `streamTurn` carries `onBehalfBearer`, `idempotencyKey` and `origin` as first-class options that ride the turn body.
- 1631 `origin` is a label only: it reaches no authority or routing decision and an unusable value is dropped rather than failing the message.
- 1632 `renderEvent` is a reference rendering that returns null for `message_start`, `heartbeat` and unknown types.
- 1633 The web SDK ships as the public-scoped package `@wlilley93/boltrig-web-sdk` at version 0.2.0 with `sideEffects:false` and a GitHub Packages registry.
- 1634 The web SDK has zero runtime dependencies and every import in its source resolves to a sibling module.
- 1635 The web SDK declares seven subpath exports beyond the root, none of which any file in the repository imports.
- 1636 Publishing the web SDK is a `RESERVED` act requiring a release warrant or Principal authorisation plus a human checkpoint.
- 1637 The web SDK ships no built artefact in the tree: `dist/` is gitignored and produced by `pre-publishOnly`.
- 1638 `BoltrigClient` is the single typed client over the kernel `/v1` surface, and the route-ledger gate enumerates every one of its public methods from its source.
- 1639 Every served HTTP route names either a real SDK method with a Worker call site, a documented richer projection, or one explicit non-UI classification.
- 1640 The eight public SDK methods the Worker does not call are each enumerated with a classification and a rationale, and a new uncalled method fails the gate.
- 1641 A client configured with `accessToken` omits cookies and reads no cookie CSRF token; a cookie client is unchanged; an explicit setting wins over both.
- 1642 Every mutating request carries the `x-boltrig-csrf` double-submit header when a token resolves.
- 1643 Governed writes tolerate a non-2xx status so a 202 pending-human body reaches the caller as a value rather than an exception.

- 1644 Every governed SDK method replays an approval through its own exact route with its exact body and the internal approval id.
- 1645 `streamChat` returns the `ChatQueued` body on HTTP 202 rather than raising.
- 1646 `followConversation` resumes from a caller-supplied cursor, rejects a non-safe-integer cursor client-side, and treats HTTP 409 as an idle conversation rather than an error.
- 1647 A follow frame whose cursor regresses aborts the stream rather than replaying.
- 1648 Heartbeat frames are dropped inside the SSE pump and never reach a consumer.
- 1649 A malformed SSE frame terminates the web SDK's stream, because the pump parses each frame with an unguarded `JSON.parse`.
- 1650 The `ChatEvent` union has exactly twenty one members and `normalizeEvents` handles every one, listing the seven lifecycle-only frames explicitly so none is dropped by silence.
- 1651 A `tool_result` is paired to its `tool_call` by `call_id`, falling back to the last pending entry with the same verb.
- 1652 `subagent_end` settles the matching delegation node in place by `child_run_id`, and a settle for an unseen child is ignored rather than invented.
- 1653 `message_end` carries no token usage, so no consumer can reconcile actual model spend from the stream.
- 1654 The replay frames the kernel emits for an accepted duplicate message carry a `replay` flag and no `run_id`, neither of which the SDK's chat-event types declare.
- 1655 The browser display-object catalogue holds exactly the kinds the kernel catalogue holds.
- 1656 `parseDisplayObject` returns null on any contract violation and never coerces an invalid envelope into a renderable one.
- 1657 Any `url`, `href` or `image_url` inside a display object must parse as an https URL with a hostname and no embedded credentials.
- 1658 A display-object envelope is refused beyond 65,536 encoded bytes, depth 6, 100 array items, 64 object keys or 32,768-character strings.
- 1659 `compileCapabilityForm` refuses 25 JSON Schema keywords, three reserved property names and any schema exceeding 100 fields or depth 5, returning a typed unavailable reason.
- 1660 Secret-shaped and open-map schema fields are refused with a stated reason rather than given an input seam.
- 1661 A capability output with no closed schema is hidden with an explicit reason rather than rendered generically.
- 1662 A registry-supplied `pattern` is compiled to a browser `RegExp` and executed against user input with no complexity or length bound.
- 1663 `sanitizeFamiliarState` bounds every field independently, clamps non-finite numbers, maps unknown enum values to the calm default, and returns null only on a wrong envelope or a stale sequence.
- 1664 The Familiar v2 renderer state carries nine phenotype scalars while the phenotype response type names ten, so `attachment` does not reach a renderer.
- 1665 `registerCharacter` validates the id grammar, the name, the blurb, the phenotype flag, the render function and every voice id, and refuses a duplicate id or a case-folded duplicate name.

- 1666 A registered character is stored frozen, so a JavaScript add-in cannot mutate its id, name or render after validation.
- 1667 A throwing registry listener does not fail the registration that already committed.
- 1668 The character registry is a module-level mutable singleton, so two module instances produce two independent registries.
- 1669 A character bundle carries no executable code: every field of the manifest is JSON.
- 1670 A bundle asset path that is absolute or contains a `..` segment is refused rather than normalised, for the visual assets `assetRef` guards.
- 1671 `parseCharacterBundle` validates nine manifest fields and passes the remaining eight plus any unknown key through unchecked via the object spread.
- 1672 `exportCharacterBundle` carries only the seventeen allow-listed fields out of an install, and a bundle's own `travels:false` outranks the caller's consent to export derived state.
- 1673 The enrolled face never travels in a bundle: it has no field in the manifest schema at all, so the exclusion is structural rather than a filtering step.
- 1674 `bundleVoiceId` returns undefined when a bundle names no voice for a provider, and callers must not substitute another character's voice.
- 1675 `WORKER_INTEGRATION_CATALOGUE` is frozen presentation metadata whose every entry begins `un-`certified, and it is never a connection registry.
- 1676 The web SDK's source tree is compiled directly into the Worker through a `file:` dependency, a `tsconfig` path and two Vite aliases, all pointing at `src` rather than `dist`.
- 1677 The Worker image copies `sdk/web/src` into its build context rather than installing a published tarball.
- 1678 164 Worker source files import the SDK, so an SDK source edit changes the Worker's typecheck, both its bundles and its image with no version bump.
- 1679 `make worker-quality` typechecks and bundles the web SDK source, which is the only automated check either SDK package receives.
- 1680 No Makefile target and no CI workflow runs either SDK package's own test suite.
- 1681 `sdk/web/src` is outside every structural ratchet: the Worker's 400-line file ceiling scans only `apps/worker/src`.
- 1682 A third party embeds Boltrig by serving the Worker at a same-origin subpath behind a stripping proxy and framing it, not by using the web SDK.
- 1683 The Worker derives its API base from the document pathname when no absolute base is baked in, so a mounted console routes every `/v1` call back through the mount.
- 1684 The UI-serving layers permit same-origin framing only: `X-Frame-Options: SAMEORIGIN` and `frame-ancestors 'self'`, on both the nginx image and the production Caddy edge.
- 1685 An embedding host passes `?theme=light&embed=1` per page load, and neither value touches local storage or kernel settings.
- 1686 The embed contract lives entirely in `apps/worker` and is exported by neither SDK.
- 1687 The marketing site is a Next 16 static export served as files, with no Node runtime in production and no `headers()` configuration.
- 1688 The marketing site consumes neither SDK: it declares no Boltrig dependency and imports none.

- 1689 The site's `/console` page calls the kernel's `/v1/console/overview` and `/v1/hitl/{id}/respond` with hand-rolled fetches against an operator-supplied base.
- 1690 The site console never persists a bearer token: it stores only the API base and actively removes any stored token key.
- 1691 The site carries its own narrower copy of the console response contract with no drift guard against the SDK's.
- 1692 The site's `NEXT_PUBLIC_BOLTRIG_API_BASE` bypasses the validated environment schema and is documented nowhere.
- 1694 The site's coverage thresholds are 1 to 2 percent, recorded deliberately as the true figures rather than a number picked to pass.
- 1695 The site's strict lint cannot enforce its own stated `no-any` rule, because the config disables `@typescript-eslint/no-explicit-any`.
- 1696 The site's product claims are pinned only by a six-group, five-item shape check and a four-phrase blocklist, and are outside the repository's Tier-0 claim inventory.
- 1698 Neither SDK carries a changelog, a semver policy or a deprecation process, so no stability promise is recorded in the tree.
- 1699 `docs/SDK-CONTRACT-from-opbox.md` remains the design brief for both SDKs and is stale on three counts: the `ui/` reference tree, the G1 and G2 gaps, and the G5 resume cursor.

AREA 17

The channel gateway, messaging bridges and calls

98 requirements

- 1700 The channel gateway service lives outside the boltrig package and imports nothing from it.
- 1701 Browser-authored channel config rejects secret-like and deployment-topology keys at any depth.
- 1702 A socket-class channel refuses an inline plaintext signing secret and accepts references only.
- 1703 Every control.channel verb defaults to high consequence.
- 1704 A channel approval is bound to the channel's exact configuration revision.
- 1705 Both transport classes terminate at one signed intake route.
- 1706 A channel whose signing secret does not resolve refuses intake with 503 and mints nothing.
- 1707 A missing or mismatched signature is refused 401 before any tenant is bound.
- 1708 Which bytes the intake HMAC covers is decided by the signature header name.
- 1709 The boltrig-native scheme refuses a signed request that carries no timestamp.
- 1710 A platform-signed request whose raw body is unavailable fails closed.
- 1711 A bound timestamp outside the 300 second replay window is refused despite a genuine signature.
- 1712 Replay dedup is a durable atomic record-and-check keyed by tenant, channel and delivery id.
- 1713 A message carrying no stable delivery id is deduped by content hash inside a shorter window.
- 1714 The tenant is bound for RLS only after the intake signature verifies.
- 1715 Declaring allowed_chats switches the channel into allowlist mode and an unknown chat fails closed.
- 1716 A verified sender becomes a Principal only through a tenant-scoped binding row.
- 1717 A channel principal's ceiling is the recorded tier's workspace ceiling and never a wildcard.
- 1718 A deactivated bound user's channel identity stops resolving immediately.
- 1719 A bound user's current grants intersect the tier ceiling downward.
- 1720 An unbound sender is denied fail-closed unless the channel opts into ignore or pair.
- 1721 A pairing code rests only as sha256 and is returned to its author exactly once.
- 1722 Pairing consumption enforces expiry, a constant-time compare, an attempts cap of five and a single-use CAS.
- 1723 No principal may bind an external sender to a role ranked above its own.
- 1724 Self-serve onboarding is opt-in per channel and accepts only the member role.
- 1725 Self-serve onboarding never mints a fresh identity over an existing binding.
- 1726 Self-serve onboarding is rate limited per channel before any binding is minted.

- 1727 A self-onboarded sender acts as a deterministic synthetic subject with no user record.
- 1728 Intake is rate limited per channel and per sender before any work item exists.
- 1729 A configured thread ceiling narrows the resolved principal at intake.
- 1730 Malformed thread-ceiling policy data yields an empty grant set, never a wildcard.
- 1731 Both intake ceilings are re-intersected at execution so a later item edit cannot widen authority.
- 1732 Channel addressing resolves one target in a fixed order and parks unassigned when nothing matches.
- 1733 A per-message target is taken from the signed body after a slug shape check only.
- 1734 Authored channel addressing targets are validated against a caller-scoped catalogue.
- 1735 A workflow target is honoured before agent routing and an unknown workflow parks for a human.
- 1736 A bound sender can answer a pending HITL request from the channel under the API's own eligibility.
- 1737 A plain channel reply answers only a sole pending QUESTION addressed to that sender in that thread.
- 1738 A secure question can never be answered from a channel reply.
- 1739 Every accepted channel work item carries a kernel-authored provenance stamp that replaces any caller value.
- 1740 The public provenance projection omits every raw provider identifier.
- 1741 channel.send is a high-consequence verb dispatched through the ordinary chokepoint.
- 1742 The approver-only comment parameter never reaches the deliver seam.
- 1743 The webhook-class outbound leg is IP-pinned and bound by the manifest network posture.
- 1744 A socket-class send enqueues one durable outbox row whose payload carries no credential.
- 1745 A webhook channel with no outbound_url returns queued with no consumer at all.
- 1746 Outbox claim is an atomic single-winner leased batch taken oldest first.
- 1747 Outbox ack and fail are compare-and-swapped on the live lease owner.
- 1748 A failed delivery retries behind exponential backoff and terminates at the attempt cap.
- 1749 A terminal failed delivery is requeued only through an approval-bound verb against an exact snapshot.
- 1750 Delivery receipts carry metadata only and never payload, lease state or provider error text.
- 1751 A run-completion notice returns to the originating channel thread.
- 1752 A notification is delivered only to a socket channel on which the recipient holds a binding.
- 1753 A team-scoped notification preference names a department and reaches active members only.
- 1754 A gateway session token is author-minted, socket-channel-bounded and carries an empty grant set.
- 1755 Every gateway link is authenticated by the run-token header plus an explicit channel_gateway marker.
- 1756 Exactly one gateway owns a channel, elected by a 45 second durable lease.

- 1757 Only the elected owner receives resolved provider credentials; a standby receives metadata only.
- 1758 A partially resolvable credential bundle yields needs_action rather than a partial spec.
- 1759 The desired-state revision handed to a gateway is a secret-free sha256 digest.
- 1760 Heartbeat observations are shape-validated and then fenced on the lease.
- 1761 Outbox claims are restricted to enabled socket channels this token still owns.
- 1762 The owner lease id is never projected to a browser.
- 1763 An owner lease is explicitly not a claim of process liveness or provider certification.
- 1764 A static channel snapshot is refused in a production posture.
- 1765 A rotated token file is hot-loaded only after an authorization refusal and only when the value changed.
- 1766 Gateway readiness is distinct from liveness and requires token, reconciliation and adapter convergence.
- 1767 The gateway egress guard refuses link-local and metadata addresses even for an allow-listed host.
- 1768 Each port emits a complete deliver target as its thread value, used verbatim as the outbound target.
- 1769 An intake POST that fails at the transport level is logged and the inbound message is dropped.
- 1770 The generic adapter accepts any peer that can open its TCP listener; the seam has no authentication.
- 1771 The WhatsApp adapter's inbound listener accepts any caller that can reach it.
- 1772 The WhatsApp bridge gates its send endpoint on a Host header only, not on authentication.
- 1773 Every shipped socket port treats its authenticated platform transport as the verification boundary and implements no per-request signature check.
- 1775 A realtime voice session's tool list is built only from kernel tools/list over the run-scoped token.
- 1776 A config-supplied tool list is rejected at voice adapter init before any socket opens.
- 1777 Every provider function call is dispatched back through the kernel MCP face.
- 1778 A browser media bearer is single-use, 90 second TTL and stored only as a digest.
- 1779 Media redemption requires the gateway token's channel scope and the live owner lease before the bearer is compared.
- 1780 Redemption mints a separate short-lived MCP token from the call owner's stored grant snapshot.
- 1781 A call's tool context is the caller's permitted verbs snapshotted at creation.
- 1782 Gateway-authored call events are filtered to an allowlist of type and field before storage.
- 1783 Raw microphone and synthesized audio never enter the call event store.
- 1784 A final call transcript is projected once into the conversation under a deterministic id.
- 1785 Typed mid-call text is bounded by a durable, reconnect-stable per-call budget that fails closed.

- 1786 Concurrent browser calls are isolated per call id and bounded by a pool that refuses rather than evicts.
- 1787 A voice approval parks the exact provider call and only a normalized resolution is projected back.
- 1788 The realtime HITL projection is best-effort and can never change the resumed action.
- 1789 Missing voice configuration returns a typed realtime_unavailable call plus the text conversation.
- 1790 Reconnect rotates the spent media bearer rather than reusing it.
- 1791 Call reads are owner-scoped and answer not_found rather than denied for another owner.
- 1792 Conversation close, restore, rename and project move are owner-scoped.
- 1793 The channels table is deliberately outside the RLS fence because the inbound path resolves it before a tenant is bound.
- 1794 Every other channel and call table is inside the tenant fence.
- 1795 The channel gateway, signal-cli and whatsapp-bridge services are profile-gated and inert by default.
- 1796 The Worker edge proxies the entire gateway prefix, not only the media WebSocket.
- 1797 The gateway status endpoint exposes channel ids, adapters and observations without authentication.
- 1799 The MCP run-token registry is process-local, so a restart invalidates every gateway session.

AREA 18

The iOS companion application

92 requirements

- 1800 Boltrig for iPhone is a native SwiftUI application whose whole source is the Xcode project under ios/, with one app target and one unit-test target and no third-party package dependency.
- 1801 No container image copies ios/, so a kernel, fleet or worker roll never ships the phone app.
- 1802 The app targets iOS 17.0, iPhone only, portrait only.
- 1803 Only a Mac running Xcode 26 can build or test the app; no CI workflow and no Makefile target does.
- 1804 The privacy manifest declares five collected data types, all linked to the user, none used for tracking, all for App Functionality, and one accessed-API reason CA92.1 for UserDefaults.
- 1805 Sign-in posts email and password to POST /v1/auth/login on a cookie-bearing URLSession and resolves to exactly one of four outcomes: signed in, password change required, two-step enrolment required, or two-step challenge required.
- 1806 A two-step challenge yields no session and no stored credential until the code is accepted.
- 1807 A forced password change is refused on the phone before any request when the new password is shorter than twelve characters or does not match its confirmation.
- 1808 Two-step enrolment shows the otpauth URI, the secret and the recovery codes exactly once and stores none of them.
- 1809 The ceremony ends by minting a personal access token named '<device model> app, signed in yyyy-MM-dd' with ttl_days 90 on POST /v1/me/tokens.
- 1810 The only state that survives a launch is one Keychain item holding the instance URL, the token id, the token secret and a creation date, plus the chosen instance address in UserDefaults.
- 1811 Keychain items live under the service ai.boltrig.app with kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly, so a backup restored onto another phone carries no session.
- 1812 Every request made after sign-in carries Authorization: Bearer <token> and no CSRF header, on a URLSession that neither sends nor accepts cookies.
- 1813 A cookie-session request carries the x-boltrig-csrf header on POST, PUT, PATCH and DELETE only.
- 1814 A stored session whose instance URL differs from the current instance is discarded at launch without any network request.
- 1815 A 401 or 403 at restore clears the Keychain and signs out; an unreachable or 5xx keeps the credential and shows the unreachable screen.
- 1816 A non-2xx answer is reduced by reading both server envelopes, {status,reason} and {detail}, and a 401 whose reason is not one of five session reasons is surfaced as the server's own sentence rather than as a demand to sign in.
- 1817 Sign-out asks the server to revoke this phone's token, then clears the Keychain and returns to the sign-in screen.
- 1818 Sign-out clears the stored credential even when the revoke request fails, so a failed revocation leaves a live token the phone has forgotten.

- 1819 The minted phone token carries the whole grant allow set of its owner, because the client requests no scope.
- 1820 Closing the cookie session after minting is best effort and its failure is not reported.
- 1821 Pointing the app at another instance signs the current one out, clears the Keychain and persists the new address.
- 1822 Only an https address is accepted as an instance, and the parser strips path, query, fragment, user and password and lowercases the host.
- 1823 An account whose companion is unset or is any id other than familiar is switched to Familiar once, by writing `agent.character`, then announcing adoption, then re-reading the account.
- 1824 A failed adoption write never blocks sign-in, reports one plain sentence, and skips the announcement.
- 1825 The stored companion identifier is never rendered anywhere in the app.
- 1826 The root view renders restoring, unreachable, the sign-in ceremony, first-run setup or the three-tab workspace of Today, Chat and Settings from the session state alone, and shows nothing private before the server has confirmed the account.
- 1827 An account without `setup.onboarding_version` at 1 or above is sent to first-run setup rather than the workspace.
- 1828 The account's stored theme sets the phone's colour scheme, and system leaves the choice to iOS.
- 1829 Today reads whether a conversation is working from the server's own flag and never infers it.
- 1830 A failed read of the linked computers is not an error and leaves the rest of Today intact.
- 1831 Approve and Not now post a decision to `POST /v1/hitl/{id}/respond` and remove the card only when the server accepts.
- 1832 A linked computer reads as on only when the server says online and its last-seen time is within twenty seconds.
- 1833 Disconnecting a computer calls `DELETE /v1/devices/{id}` and drops it from the list.
- 1834 Archiving a conversation from Today soft-closes it through `DELETE /v1/me/conversations/{id}` and then refreshes Today.
- 1835 A chat turn posts message, origin ios, an idempotency key and base64 attachments to `POST /v1/chat` and reads the reply as a server-sent event stream with a 600 second timeout.
- 1836 A 202 answer to a chat post yields exactly one queued event carrying the conversation id, and a non-2xx answer before the stream begins is mapped to a `BoltrigError` and thrown into the stream.
- 1837 Server-sent event frames are split by hand on newline, several data lines in one frame are joined with a newline, and a frame that is not JSON becomes a synthetic `malformed_event` rather than an error.
- 1838 Twenty-one wire event types are mapped by name and every other type becomes `.other`, so an unknown event never fails a turn.
- 1839 Conversation history hides superseded messages and messages of any role other than user or assistant.

- 1840 A run the server reports active is followed once from cursor zero, and the history reload that follows never triggers another follow.
- 1841 A 409 from the follow route means nothing is running and is reported as idle, and each follow frame carries a cursor and a truncation flag whose true value raises one notice saying earlier activity is not shown.
- 1842 Stop cancels the local stream task and posts POST /v1/runs/{id}/cancel for the live run.
- 1843 A question raised mid-turn is answered on POST /v1/hitl/{id}/answer and the answer is appended to the thread as the person's own message.
- 1844 Attachment limits come from GET /v1/chat/config, falling back to eight files, 256 KiB each and 1 MiB total when the route cannot be read.
- 1845 A chosen file is refused on size before any of its bytes are read, and a chosen photo is re-encoded as JPEG and shrunk until it fits the per-file limit or refused with the size sentence.
- 1846 A 413 answer to a chat post renders one fixed sentence about attachments instead of the raw error.
- 1847 Reading a reply aloud requires the account setting, an adapter bound to voice.speak and a resolved voice, where a per-character override applies only to the local voice provider and only when it matches the voice-id pattern; any of the three missing means silence and no request.
- 1848 A finished reply is spoken at most once per run id, through POST /v1/invoke with noun voice and verb voice.speak.
- 1849 Spoken text is markdown reduced by ten rewrite rules and capped at fifteen thousand characters.
- 1850 Every speech failure is silent: no error is shown and the reply stays on screen.
- 1851 Audio is accepted only when the invoke returns ok, the base64 payload is at most twelve million characters, and the content type begins audio/.
- 1852 The phone never uses on-device speech synthesis for a reply.
- 1853 One WKWebView hosts Familiar's shader page and exactly one surface claims it at a time.
- 1854 State reaches the island as clamped, key-sorted v1 JSON, at most thirty times a second, only after the island has reported ready, and never twice with the same content.
- 1855 The island message carries no text, identifier or secret: every field is a closed enum, a bounded number or a bounded number map.
- 1856 The presence mode follows the web's precedence: error, then speaking, then listening, then working, then thinking, then standby.
- 1857 The island web view cancels any navigation that is not a file URL, and a missing page, a fallback report or a navigation failure falls back to the SwiftUI badge so the presence is never blank.
- 1858 Familiar's phenotype is polled every three seconds only while a surface holds the island and the scene is active, and is dropped to nil otherwise.
- 1859 A phenotype reading that is not fresh yields no values, so Familiar wanders on her own rather than showing a stale mood.
- 1860 Only a genotype whose source is agent_capability.name.v1 binds a visual identity; anything else draws the neutral body, and unknown markings and accessories are dropped rather than guessed.

- 1861 The bundled island page is byte-pinned to the worker source by a manifest and by make familiar-island-check inside worker-quality.
- 1862 First-run setup is four steps, Name, Provider, Image model and Ready, with no companion step and no voice step.
- 1863 The display name is normalised on the phone before it is sent: whitespace collapsed, one to eighty scalars, no control, format or surrogate characters.
- 1864 Finishing setup writes the profile, then agent.character and setup.onboarding_version together, then announces adoption, in that order and exactly once even under concurrent presses.
- 1865 A name the server refuses returns the person to the Name step with the plain sentence, and the settings are not written.
- 1866 Skipping the image model step submits no key and finishes setup.
- 1867 One press of Continue saves the provider key, answers an approval the server raised for the person's own submission, and reads back whether the provider answers.
- 1868 The key field is emptied before the first await, so nothing on screen holds a submitted key and a second press cannot resend it.
- 1869 A proposal that genuinely awaits an administrator is cached so the next press re-checks it instead of resubmitting the key, and any other dead proposal is dropped.
- 1870 An unreachable read-back after a save lets the step pass with an honest sentence, and only a server that positively reports the gateway is not ready holds the step.
- 1871 The three provider routes turn any server-explained refusal, on 4xx and on 503 alike, into an outcome whose sentence reaches the screen, while a refusal with no sentence is still thrown.
- 1872 A typed provider address always wins, and a non-native provider with a published address submits it silently.
- 1873 The bundled provider catalogue is the web's models.dev snapshot byte for byte, decoded off the main thread with its revision pinned by a test, and a self-hosted Ollama entry is inserted immediately before Ollama Cloud with no models, an optional key and a required address.
- 1874 Settings offers eight destinations, searchable by their label and their one-line lead.
- 1875 The approval posture is read-only on the phone, and the screen says a change needs a person signed in on the web.
- 1876 The server refuses a posture write from a personal access token, because the route requires an interactive credential class and a token resolves to kind pat.
- 1877 Changing any appearance value writes all five appearance keys in one request, turning read-out replies on writes that one key, and either write rolls the local value back on failure.
- 1878 Security lists live sessions and live keys, marks the key this phone signs in with, and signs the phone out when that key is revoked.
- 1879 Health reads GET /readyz and treats a 503 as an answer carrying the report rather than a failure.
- 1880 Spending shows total cost and ceilings read-only, in money when a money ceiling exists and in usage otherwise.

- 1881 Archived chats lists only closed conversations, newest activity first, and can bring one back through `POST /v1/me/conversations/{id}/restore`.
- 1882 Deleting an account from the app is disabled and no deletion request of any kind reaches the network while the feature flag is false.
- 1884 A debug-only preview workspace and a debug-only setup preview are reachable by launch arguments, compile out of every release build, and perform no network request.
- 1885 The test suite is ninety-five XCTest methods in twelve classes, every one except the live contract runs against an in-memory URLProtocol stub with no network, and two skip themselves rather than fail.
- 1886 The live contract test signs in against a real instance, asserts the Familiar switch, performs the first-screen reads, sends one turn and signs out, and is skipped unless credentials are supplied.
- 1887 No invariant in `tests/invariants.yaml` binds any iOS behaviour, and there is no UI test target, so every SwiftUI view in the app is unverified by any automated check.
- 1889 Nothing but the credential and the chosen instance address survives a launch, so every screen is empty until the server answers.
- 1895 The client's native-provider set and alias table are a third copy of the kernel's authority with no gate keeping them equal.
- 1896 Familiar's per-provider fallback voice ids are a second copy of the character bundle with no gate keeping them equal.
- 1898 The default instance is the development preview host, and moving it silently signs out every installed copy at the next launch.
- 1899 `FamiliarVisualIdentity.validPalette` compiles its regular expression with `try!` on every call inside a per-frame badge, and `ios/` is absent from `.dockerignore`, so its 1.3 MB of Swift and bundled resources enter every image build context.

AREA 19

Deployment topology, the release train and recovery

98 requirements

- 1900 The base Compose model declares fifteen services and every one carries restart unless-stopped, so no service is left without a restart policy.
- 1901 Three services run the one built fleet image and differ only by command.
- 1902 Every third-party image in the Compose model is pinned to a tag plus an explicit sha256 digest.
- 1903 The hardening anchor sets a read-only rootfs, a tmpfs, all capabilities dropped, no-new-privileges, a pids limit and a memory limit.
- 1904 Six services merge the hardening anchor: kernel, fleet-worker, browser-executor, hatchet-worker, channel-gateway and whatsapp-bridge.
- 1905 The hardening deploy-lint asserts the six properties on only kernel and fleet-worker, leaving the other four merged services unguarded against a regression.
- 1906 The kernel service runs at uid 0 with SETUID and SETGID re-added after all capabilities are dropped.
- 1907 The kernel entrypoint forks a uid-0 spawner, drops the API process to uid 10001 and proves the drop from proc before exec.
- 1908 When the capability is absent the kernel entrypoint execs the given command unchanged, introducing no new failure mode.
- 1909 The kernel declares one shared cell root plus four per-slot tmpfs mounts owned by distinct cell uids, all noexec, nosuid and nodev.
- 1910 The fleet worker and the hatchet worker declare the same cell-root tmpfs as the kernel because all three run the same boot-time sandbox probe.
- 1911 The kernel container healthcheck probes readiness rather than liveness, so an unapplied migration head makes the container unhealthy.
- 1912 The fleet-worker healthcheck reads back the signed stack-tool receipt rather than importing a module.
- 1913 The hatchet-worker healthcheck probes the SDK listener heartbeat server on container loopback, green only after the worker connected and heartbeated.
- 1914 The browser-executor healthcheck is a Unix-socket self-probe and the release validator requires it.
- 1915 The hatchet worker depends on the engine with a started condition because the engine declares no healthcheck.
- 1916 The browser executor is the sole member of the browser-egress network, publishes no host port and owns the only Chromium in the deployment.
- 1917 Kernel, fleet worker and hatchet worker reach the browser only through a read-only socket volume and never mount browser state or a personal profile.
- 1918 The Hatchet engine and its unauthenticated dashboard publish only on loopback in the base model.

- 1919 The sensitive-data local-model port is loopback-only in base and removed entirely by the secure overlay.
- 1920 The Postgres password uses the required-var form so the stack refuses to render unset, and the Hatchet connection string interpolates it rather than hardcoding a credential.
- 1921 The kernel and ui services publish on all host interfaces in the base model; only the dev and secure overlays narrow or remove those bindings.
- 1922 Four profiles keep the model gateway, the on-box model, the channels trio and the backup sidecar out of a bare bring-up.
- 1923 The sandbox network is a plain bridge in the base model and only the secure overlay makes it internal, which the base file states in terms.
- 1924 The Hatchet config directory is a named volume in the base model so a container recreate cannot mint a new signing keyset.
- 1925 The kernel mounts the backup freshness marker read-only and can never read a backup artifact.
- 1926 A fresh Postgres volume creates the separate Hatchet database through a first-boot hook that validates the name and skips a duplicate create.
- 1927 The schema file is a first-boot bootstrap only and Alembic is the authoritative upgrade path for an existing deployment.
- 1928 Redis persists with append-only mode and everysec fsync so relay and counter state survives a container restart.
- 1929 The secure overlay adds a TLS terminator and removes the host ports of kernel, ui, both Hatchet services, the gateway and the local model.
- 1930 The secure overlay makes the sandbox network internal so a sandbox-only service has no route to any external address.
- 1931 The secure overlay parameterises the Postgres data directory so encryption at rest is a deployment choice with no image change.
- 1932 The dev overlay binds the kernel API to loopback on a non-default port and removes the ui host port so Caddy is the single front door.
- 1933 The dev overlay forces the trusted Codex lane off on both kernel and fleet worker so a namespace restriction cannot become a crash loop.
- 1934 The in-process overlay runs the kernel as uid 10001 and is documented as a single-tenant dev convenience that must not reach a client deployment.
- 1935 The VM overlay relaxes only syscall filtering on kernel and fleet worker, retaining the read-only rootfs, the dropped capabilities, no-new-privileges and both resource caps.
- 1936 The VM overlay enables IPv6 on the app networks because outbound address pinning otherwise selects an address a v4-only container cannot route.
- 1938 Compose validation renders base, base plus channels, secure, dev, inprocess and opbox-link, and pipes two release renders into the release compose validator.
- 1939 Compose validation runs from a clean checkout using the example env and a validation-only database password, never a developer env file.
- 1940 The gate-coverage gate derives the compose manifest list by globbing rather than listing it, so a new overlay fails the gate on the commit that adds it.

- 1941 The health-claim gate fails any first-party service whose healthcheck probes liveness on an application that also serves a readiness route.
- 1942 The release overlay resets every first-party build definition, pins each image to a required digest variable and sets an always pull policy.
- 1943 The release overlay pins the fleet worker, the browser executor and the hatchet worker to the one signed fleet digest and the validator refuses any divergence.
- 1944 The release overlay removes the developer bind mount of the backup script so the signed backup image cannot execute mutable host source.
- 1945 The release compose validator rejects any active channels-profile service because those images are not in the signed release set.
- 1946 The release image environment must contain exactly four variables, each an immutable digest reference, with no missing and no extra keys.
- 1947 The release mode must be exactly core or full and all four release runtime services must agree on the same value.
- 1948 The release mode is required by the release overlay and by the runbook but has no line in the example env file.
- 1949 The release start target depends on the validation target, so validation reruns immediately before the pull and the start.
- 1950 The release start target uses no-build so production can never rebuild mutable source on the target host.
- 1951 The release workflow triggers only on a version tag, serialises on one concurrency group and never cancels in progress.
- 1952 The release preflight requires the tag's commit to be an ancestor of the default branch and both canonical workflows to have succeeded for that exact sha.
- 1953 The release preflight proves registry write access by opening and cancelling a blob upload, before spending ten minutes on candidate builds.
- 1954 Each candidate image is vulnerability-scanned with the same acceptance file the security gate uses, so the two gates cannot disagree about one image.
- 1955 Candidates are pushed only under a run-scoped tag until every digest is signed, SBOM-attested and provenance-bound.
- 1956 Promotion fetches each signed manifest and refuses to publish unless the bytes hash to the signed digest.
- 1957 An existing public tag whose digest changed after preflight stops promotion rather than being overwritten.
- 1958 The release asset helper never clobbers: an exact retry compares bytes and refuses a difference, and semantic modes verify the attestation for the exact digest.
- 1959 Only a stable full release may be marked Latest, so a core release or a prerelease cannot displace the desktop updater's manifest.
- 1960 The roll script refuses to run unless its own checkout contains the tag being rolled.
- 1961 The roll repins the tracked overlay source, propagates it, and proves the box copy matches by checksum before bringing anything up.
- 1962 The roll migrates each stack to the head its target image asserts, immediately before that stack's deploy, and never to the checkout's head.

- 1963 A database found ahead of its image is left untouched and the migration gate exits non-zero, naming the situation a rollback that needs a human and a dump.
- 1964 The roll gate asserts kernel health, worker liveness, an expected addons line on both, the running kernel image version, and ui health plus version before proceeding.
- 1965 The canary stack is rolled and gated before the tenant is touched, and the canary-only flag stops after the canary while saying the fleet is now uneven.
- 1966 The roll script is declared a non-production path in both deployment documents and the canonical procedure is the cutover runbook.
- 1967 The deployment verifier asserts readiness and the presence of named symbols inside the running container, exiting non-zero if either fails.
- 1968 The backup sidecar ships profile-gated in the base compose, mounts the backup script and a backups directory, and runs the loop as its command.
- 1969 Each backup run refuses any dump the restore tool cannot parse and refuses an empty archive, before any artifact is published.
- 1970 Every backup artifact gets a checksum sidecar that is re-verified after publish, and both files are deleted on a mismatch.
- 1971 The stack-state archive is refused without a passphrase because it carries Hatchet signing material and user Knowledge.
- 1972 The stack-state tar is piped straight into the encrypter so a killed run can never leave a plaintext archive on disk.
- 1973 The recovery-set completion marker is uploaded off-box last, after every artifact and sidecar, so a partial remote run has no complete-set marker.
- 1974 A configured off-box copy that fails exits non-zero and leaves no success marker, so a broken remote cannot pass silently.
- 1975 An unset off-box remote warns and continues with a local-only backup, the only deliberately fail-open branch of the backup path.
- 1976 Retention prunes whole recovery sets rather than individual files, using a while-read loop because the array builtin is unavailable on bash 3.2.
- 1977 The backup loop lets a failed run exit PID 1 so the restart policy surfaces the failure instead of masking it until the next interval.
- 1978 The backup healthcheck is unhealthy on a missing, malformed, or older-than-interval-plus-grace success marker.
- 1979 The recovery-set verifier is read-only and refuses symlinks, tampering, inconsistent sidecars, empty files, unencrypted dumps and foreign artifacts.
- 1980 The verifier requires the exact configured database set and refuses both a missing required database and an unexpected extra one.
- 1981 Every encrypted artifact must carry the OpenSSL salt header or the recovery set is refused.
- 1982 The recovery rehearsal refuses a remote or ambiguous Docker endpoint before creating any container.
- 1983 The recovery rehearsal strips any ambient test database URL so it cannot select an operator database.

- 1984 The signed release backup archives Hatchet config, Knowledge, libraries and the manifest together under one mandatory-encryption state directory.
- 1985 Redis append-only state is explicitly outside the logical recovery set and connector credential volumes are the deployment's own responsibility.
- 1986 Restore order requires postgres alone first, both dumps restored, the Hatchet config restored with the engine stopped, then engine, kernel and workers, then readiness.
- 1987 The Postgres first-boot hook never reruns against an existing data volume, so an upgraded cluster needs the Hatchet database created explicitly.
- 1988 Genesis fills only blank secrets and derives the database URL from the Postgres variables so the two cannot disagree.
- 1989 Genesis requires an explicit superadmin email and password and bundles no personal or default identity.
- 1990 Genesis exits zero after a failed health or login verification, so its exit code does not distinguish a founded stack from a broken one.
- 1991 Genesis verifies against port 8082 from its own shell, which is not the ui port under the dev overlay's default and does not exist under the secure overlay.
- 1992 The dev bring-up script publishes the tailnet origin whether or not the kernel health poll ever succeeded.
- 1993 The disk-hygiene script prunes least-destructively first and escalates through shorter retention windows only while still over the threshold.
- 1994 The disk-hygiene script reads the actual reclaimable figure before reporting a cause, and exits non-zero when still over threshold.
- 1995 The wheelhouse builder downloads one requirement at a time under hash verification and exits non-zero if any requirement is still missing after five passes.
- 1996 The kernel image consumes an optional wheelhouse through a bracket-glob copy that is a no-op when the directory is absent, with an identical hash contract in both branches.
- 1997 Backup credentials are excluded from git and from every Docker build context by matching both the directory and any rclone config at depth.
- 1999 The named AppArmor profile's allow list carries mount, umount, userns and pivot_root, and the profile must never be replaced by an unconfined one.

AREA 20

Governance: invariants, gates, decisions and the court

97 requirements

- 2000 Every `scripts/check_*.py` is invoked by at least one Makefile recipe, and an orphan fails the gate-coverage gate.
- 2001 Every one of the eight `make quality` prerequisites is invoked, directly or transitively, by a CI workflow `run:` line.
- 2002 All six compose manifests in the tree are `-f` inputs to a `docker compose config` step in the Makefile.
- 2003 `make python-quality` runs eighteen gate prerequisites and is the aggregate CI's `test-and-gate` job invokes.
- 2004 `make check` is a local subset that omits seven `python-quality` targets and is invoked by no workflow.
- 2005 `continuity-projection` and `codex-pin-health` run only in the pre-push hook and in no CI job.
- 2007 The orphan-gate census cannot see a shell gate, because its glob matches only `check_*.py`.
- 2009 The pre-push hook is installed by nothing; `core.hooksPath` must be set by hand per clone.
- 2010 The pre-push hook blocks the push outright when `core.hooksPath` is absolute.
- 2011 `SKIP_PUSH_GATES=1` skips every pre-push gate and `SKIP_QUALITY_GATE=1` skips only the Python suite, each announced on stdout.
- 2012 A pre-push gate refused for lack of memory (exit 75) is reported as not-run, never as failed, and still blocks the push.
- 2013 The absence of the machine-wide heavy-job guard is announced and the gates continue unguarded.
- 2014 On macOS the pre-push Python suite runs inside a Linux VM through one line of dispatch around the same make target CI runs.
- 2015 The merge-drift check fetches the base ref first and reports a failed fetch as evidence of nothing.
- 2016 The pre-push hook prints, on a green pass, the six CI jobs and three `test-and-gate` steps it did not run.
- 2017 The invariant catalogue declares 421 unique ids at the referent commit, with zero duplicates.
- 2018 A repeated invariant id raises rather than resolving last-wins, because a duplicate silently evicted SEC-169 for four months.
- 2019 Binding debt is zero: no declared invariant is unbound and no marker is undeclared.
- 2020 The catalogue also fails on drift: a claimed node id that no marker backs, or a `service_gated` id outside that invariant's own tests.
- 2021 The catalogue may omit a real marker-backed binding without failing; 361 of 1836 real pairs are undeclared.

- 2022 An invariant whose every binding is `service_gated` reports GATED, never ok, so a permanently skipped test cannot discharge it.
- 2023 Exactly six `K-*` ids and one `P*` id are declared; the other 414 invariants are Boltrig-local.
- 2025 The gate is named after K-29 and K-30, neither of which the catalogue declares.
- 2026 `docs/invariants.md` documents 171 of the 421 declared ids and states a stale declared count of 390.
- 2027 The `invariant(id)` marker is registered in pyproject so an unknown marker is not silently ignored.
- 2028 `docs/claim-inventory.tsv` must regenerate byte-identically from the sources or the gate fails.
- 2029 The load-bearing no-subject residue is 206, exactly at its pinned baseline, and may only decrease.
- 2030 A claim naming a subject nothing reaches (SUBJECT-ABSENT) fails unconditionally, regardless of the baseline.
- 2031 The Python structural ratchet is exact in both directions: growth fails and a stale-high baseline also fails.
- 2032 Thirty-four Python files carry a structural exemption, each with owner, reason, ISO expiry and exact per-function baselines; none is expired at the referent commit.
- 2033 The Worker structural debt catalogue is loaded from an immutable Git object supplied by the CI event, never from a second working-tree file.
- 2034 The unreachable-function count is an upper-bound ratchet only: a fall prints a note and passes.
- 2035 Every declared reachability root must give a reason naming what calls it; a root with no reason fails.
- 2036 Seventeen unwired-claims waivers each carry owner, reason and expiry, and a waiver file that waives nothing it can be held to fails the gate.
- 2037 The health-claim exemption file is empty and the tests hold it that way.
- 2038 Every `name==version` pin in a dependency overrides file must appear at that same version in the compiled lock beside it.
- 2039 Every tree scan in `scripts/` declares a floor and fails when it finds fewer, and the sweep tests the property rather than the helper.
- 2040 The vacuous-green sweep runs seventeen of twenty gates against an empty tree and declares its three exemptions with reasons.
- 2041 The sweep's discriminator is the gate's success vocabulary, not its exit code, so an honest skip is not flagged.
- 2042 The exemption reason recorded for `check_user_authority.py` in the sweep names GitHub, while the gate reads a Postgres DSN.
- 2043 Six gate modules import `require_scanned`; the sweep's stated figure of five of eighteen is stale.
- 2044 Prose references resolve across five rule kinds: repo paths, pytest node ids, make targets, boltrig env vars and VJS order citations.
- 2045 Order citations resolve against a vendored citator, never a sibling checkout, so no gate can pass because of a directory on one machine.

- 2046 A negated or past-tense clause exempts a reference in the historical record but never in live source under `boltrig/` or `scripts/`.
- 2047 Every repo-relative path cited in a commit `Refs:/See:` trailer must have existed in that commit's own tree, not at HEAD.
- 2048 A commit-trailer scan that finds zero trailers fails rather than passing.
- 2049 No tracked symlink may dangle, loop, or resolve outside the repository root, and it is deliberately not a ratchet.
- 2050 The full-history secret scan refuses to run over a partial clone rather than scanning what it can reach.
- 2051 Twenty-four court orders are filed, all `status: binding`, carrying 163 directives between them.
- 2052 A directive counts as bound only when a file under `tests/` names the directive id within two lines of a line naming its order.
- 2053 A directive mention in `boltrig/` is the claim, not the enforcement, and never binds.
- 2054 Fifteen court directives are waived, each with owner, reason and unexpired ISO date, and an unused waiver is itself a failure.
- 2055 The waiver key is the order id's tail plus the directive id, so two orders of the same name in different courts share one key namespace.
- 2056 An order's `implementation_status` is read by no gate; five orders read OPEN and fifteen declare no status at all.
- 2057 The waiver file's own header states 102 directives and 47 entries where the tree holds 163 and 15.
- 2058 The founding consolidation ruling has never existed as a file in this repository and is allow-listed rather than reconstructed.
- 2059 Two further orders relied on in live code were never filed and are allow-listed with the finding recorded.
- 2060 A VJS permit is a self-issued front-door record that cannot satisfy a check reserved to the Sovereign or to a constituted bench.
- 2061 `scripts/govern.py` drives a HIGH-consequence verb through the real kernel chokepoint and reports the sole-author exemption rather than assuming it.
- 2062 `docs/decisions/` holds forty-two records governed by no gate, template or index.
- 2063 Decision numbers 0023 and 0030 are each issued twice and 0005 was never issued.
- 2064 Decision status is free text: one record carries no status field and at least three distinct syntaxes are in use.
- 2065 AGENTS.md requires a first-impression decision to be surfaced but does not require it to be recorded or numbered.
- 2066 `.vds/config.toml` is the single VDS anchor and holds paths, globs and governance only, never a design value.
- 2067 The VDS ledger gate verifies ownership, digests, JSX coordinates and route-manifest drift, and makes no visual verdict.
- 2068 The VDS ledger gate floors its scans at ten screen files and six routes so an empty scan cannot pass.

- 2069 The 794 committed VDS proofs are 282 failed, 125 passed and 387 vacuous, and no Python gate reads them.
- 2070 Six VDS proof kinds have never passed and are entirely vacuous because no bound is declared for them.
- 2071 The VDS designpack is pinned to `none@0`, so no external design authority is bound.
- 2072 `ci.yml` declares six jobs that expand to eight runs, collapsed into one required `quality` context that asserts five successes.
- 2073 `security.yml` declares four jobs that expand to nine runs, collapsed into one required `Security gate` context that asserts three successes.
- 2074 `security.yml` never cancels an in-progress run on `main`, because a cancelled required check leaves the ref with no successful evidence.
- 2075 Both aggregator jobs use `if: always()` so a skipped or cancelled dependency fails the context rather than skipping it.
- 2076 The `test-and-gate` job checks out full history because the commit-trailer gate reads commit messages.
- 2077 CI runs the Python suite against a real pgvector Postgres so the durable-store legs execute rather than skip.
- 2078 A pytest run whose Postgres or fakeredis precondition is unmet ends non-zero unless a named opt-out is set.
- 2079 Every product-behaviour environment variable is stripped for the duration of every test, with an eleven-entry keep-list for test selection only.
- 2080 `make gate-status` reports a commit with no completed run as unproven, never as green.
- 2081 `gate-status` filters to push-event runs and groups by workflow name, so a Dependabot failure cannot colour the verdict.
- 2082 The release preflight requires the latest `ci.yml` and `security.yml` runs for the exact release commit to be completed and successful.
- 2083 The release preflight requires the tagged commit to be an ancestor of the default branch before anything is built.
- 2084 Branch protection requires the contexts `quality` and `Security gate` with `enforce_admins` false, so the sole admin retains a bypass.
- 2085 GitHub Actions execute for this repository; the estate-wide private-repo billing block does not apply to it.
- 2086 `docs/security-gates.md` contradicts itself on the required context names, recommending the `ci / quality` form it earlier refutes.
- 2087 `docs/security-conformance.md` still records required branch-protection checks as an open seam and still carries the retired Pi runtime family.
- 2088 `docs/GOAL-trustworthy-gate.md` states its done-criterion against `ui-build` and `ui-e2e`, jobs no workflow declares.
- 2089 `docs/GOAL-claims-must-be-load-bearing.md` states 330 declared invariants, 236 residue claims and eleven python-quality gates, all stale.
- 2090 One `.trivyignore.yml` acceptance expired nine days before the referent commit.
- 2091 Trivy's vulnerability legs honour `.trivyignore.yml` explicitly, because without the flag the file is consulted only for misconfigurations.

- 2092 The Python audit runs through a wrapper that enforces the expiry on the accepted-advisory ledger and prints what it suppresses.
- 2093 `make visual-evidence` runs the existing vitest rather than recomputing the digest, so a second implementation cannot drift from the receipt.
- 2094 `check_continuity_projection.py` fixes the four continuity frozensets to the sets a court order settled, and a change to the table is a change to the order.
- 2095 `check_codex_pin_health.py` is fatal only when `BOLTRIG_CODEX_BINARY` is set, and never fatal on version drift.
- 2096 The Codex protocol pin is checked in as exact constants: version, target, binary digest, schema digest, bundle file count and admitted transports.
- 2097 The two cross-repo drift gates exit 2 with NOT CHECKED when the upstream tree is absent, rather than reporting an agreement they did not observe.
- 2098 `check_user_authority.py` and `check_fleet_drift.py` are operator tools declared not to be CI gates, each needing a live tenant or a live box.
- 2099 Three gates are bound by neither an invariant test nor a selftest: `check_codex_pin_health.py` and the two shell drift gates.

Excluded rows

65 of 1902 register rows

These rows are in the register but are not requirements. A row tagged DEAD or SCAFFOLDED records that something in the tree is unreachable or unfinished; with its tag removed it would read as an instruction to build exactly that. A SEAM row names an external system that supplies the behaviour rather than stating behaviour of this one. They are listed so the omission is visible and reversible.

Dead · unreachable in the tree (42)

0187	<code>CredentialResolver.sweep_run_scoped</code> has no production call site; the wired lifecycle seam is <code>held_call.sweep_run_credentials_if_settled</code> .
0197	The routing attribution keys <code>capability</code> , <code>capability_binding_id</code> , <code>connection</code> , <code>source_operation</code> and <code>route_selected_by</code> are written into <code>meta</code> and read by nothing.
0303	<code>Chief0fStaff</code> is present in the tree but constructed by no production composition root, so its routing code is unreachable in serving.
0387	The workflow-learning branch inside <code>_settle</code> is unreachable in production because nothing writes the <code>generated_workflow</code> key it reads.
0496	<code>SupervisedCodexPhaseCellProvider</code> is never constructed outside tests and is not the production cell provider.
0547	<code>mcp_servers.credential</code> is a credential-shaped column with no writer and no reader in the tree.
0575	<code>expire_trajectories</code> has no production caller, so <code>trajectory_events.expires_at</code> is written and never acted on.
0582	No SQLite store backend exists; <code>aiosqlite</code> is a dev dependency with no importer in the tree.
0647	<code>assert_no_metadata_egress</code> has no production caller and is exercised only by tests.
0679	<code>McpConsumerAdapter.connect()</code> is unreachable in production because <code>control.adapter.activate</code> refuses an MCP consumer row before the connect branch.
0799	The on-behalf-of delegation mechanism is present but unreachable: <code>OnBehalf0f</code> is never constructed and <code>identity_mode</code> is read only by a view renderer.
0862	The non-Cognee compile and erase branches in the projection coordinator are unreachable because the enclosing loop already skips any id outside <code>SUPPORTED_PROVIDER_IDS</code> .
0899	The <code>knowledge_projection_outbox</code> table is created and RLS-fenced but is read and written by no application code.
0975	<code>select_locale</code> and the <code>base@locale</code> variant convention are unreachable: no caller exists and no variant file ships.
0977	<code>libraries/prompts/*.md</code> is referenced by no code in the tree.
0980	<code>schemas/emotion/1.0.0/phenotype.schema.json</code> is documentary: no code loads it and no gate validates against it.

- 0982 WorkItemStore, its legal-transition table, its status compare-and-swap and write_back_discovered have no caller anywhere in the tree, and the fleet writes work-item status directly instead.
- 0983 The QueueAdapter protocol and InternalQueueAdapter have no implementation or caller outside their own module.
- 0985 The workflow learning leg never fires: the pump keys on an outcome field that no production department head writes.
- 0986 select_or_generate_workflow and WorkflowLibrary.match have no production caller; production selects a workflow by explicit id.
- 1161 The dev_egress_loopback manifest key is unreachable: it is absent from the retained-section allow-list, so its resolver is permanently disabled.
- 1231 observability.tree.build_tree is unreachable from any code path in the tree.
- 1265 The doctor helper _csv is defined and never called.
- 1375 No scheduled or automated invocation of boltrig audit-verify exists anywhere in the repository, so the tamper-evidence chain is re-derived only when an operator remembers.
- 1387 bootstrap.build_kernel, the synchronous wrapper, is unreachable from every shipped entry-point despite a docstring naming uvicorn and the worker as its callers.
- 1492 The boltrig.camera discovery package and the read-only camera adapter are reachable only from tests, not from any production endpoint.
- 1493 The device presence value locked is declared by the schema and the SDK and is never written by any code path.
- 1508 AccountView and the nine account sections it composes have no importer in apps/worker/src and are rendered only by tests.
- 1509 AutomationView, the 2870-line workflow, trigger and schedule authoring surface, has no importer in apps/worker/src and is rendered only by tests.
- 1510 InboxHitl, the dedicated approvals inbox, has no importer in apps/worker/src.
- 1511 ConversationControls has no importer, so conversation rename and message regenerate have no mounted affordance.
- 1512 Five further modules are unreachable: CameraDiscoverySettings, settings/OvernightSection, Jarvis/JarvisGauge, voiceTone and voiceSpectrum.
- 1513 Thirty-nine SDK methods have no call site in any component a route mounts.
- 1560 Thirty-one SDK-method existence guards are unreachable in a shipped build because the SDK is resolved from source through a path alias.
- 1563 The second hash segment of the automations route is inert because the mounted RoutinesView reads no route selection.
- 1693 site/src/lib/api-client.ts and site/src/lib/api/index.ts are unreachable: static export forbids route handlers and nothing imports either module.
- 1890 The island contract declares bands, onset and genotype, and the application never produces any of the three.
- 1891 The badge's bound-genotype rendering, four body shapes, four markings and three accessories, is unreachable from any application call site and is exercised only by a unit test.
- 1892 Stored message run ids, HITL request ids, stored attachments and queued message ids are decoded from history and never rendered.

- 1893 The model-readable media types the server publishes are decoded and never used, while the composer's footnote about which files are read is a hard-coded sentence.
- 1894 The session-reason 'not authenticated' in the client's 401 classifier matches nothing the server emits.
- 2006 `scripts/check_brand_core.sh` and `scripts/check_familiar_shader.sh` are invoked by no Make-file target, hook or workflow.

Scaffolded · shape without a body (9)

- 0198 Capability input and output transforms are declared on the binding and applied nowhere; a routed call executes and validates against the provider schema.
- 0284 `POST /v1/integrations/{integration_id}/oauth/start` always answers 409 `unsupported`; no OAuth flow exists.
- 0495 `AssignmentAdmission.admit` is constructed but never called from any production path.
- 0576 `entity_provenance` mints and resolves opaque record refs with an idempotent unique index, but has no production caller and is a recorded deferral.
- 0880 The craft gate always returns UNAVAILABLE until inactive candidates can be evaluated through governed Codex/Bifrost model admission.
- 1474 `FamiliarState v2` is a defined, sanitised, content-free contract with no producer and no consumer in this tree.
- 1798 A gateway session token cannot be revoked by any kernel route; only its TTL and lease bound it.
- 1897 Three public links shipped in Settings point at pages that do not exist.
- 2008 No gate asserts that the pre-push hook covers what CI runs.

Seam · supplied by an external system (11)

- 0712 SAML verification is a seam: `verify` raises without an injected validator, and a manifest declaring `identity.provider: saml` is rejected at load.
- 0713 OIDC and Cloudflare Access are SEAMS whose live legs are an external IdP: the verifier is real, the issuer is not in this tree.
- 0781 Password-reset delivery is a SEAM: the notifier is composed from deployment env and MailerSend is the only supported provider; partial configuration raises at startup.
- 0791 The Bifrost model gateway is a SEAM: the admin base must be an internal host with an exact `/v1` path, and the binding refuses fail-closed when it cannot be proven usable.
- 0826 No runtime prompt assembly consumes `memory.bundle`; the only callers are the adapter and the HTTP route.
- 0999 The interpreter per-step durable boundary is a SEAM, not a durable engine step: `Hatchet-Executor.run_step` awaits the callable directly because the installed SDK exposes no public durable child-step API, so per-step recovery rests on task retry plus checkpoints plus the per-step idempotency key.
- 1697 The marketing site is delivered to `/srv/boltrig-marketing` by a host Caddy that is not in this repository, and no repository script performs the copy.
- 1774 Slack v0 and Discord Ed25519 request verification are supplied by the platform's authenticated socket handshake rather than implemented in this tree.

- 1883 The DELETE /v1/me route the deletion screen would call does not exist in the server tree.
- 1888 The app implements no push notifications, no universal links, no crash or error reporting, no live voice call, no background refresh and no offline copy of the record.
- 2024 The canonical source of the `K-*` ids is an external repository that is present nowhere in this tree.

Uncertain · not settled by the author (1)

- 1998 The `dockerignore` exception for the `docs` directory re-admits the whole tree rather than only the three token files its comment names.

Findings about the governance layer (2)

- 0588 `K-22`, the tenant-isolation kernel invariant cited by `schema.sql` and `rls.sql`, is not a declared id in `tests/invariants.yaml`, so no test is bound to it by that name.
- 1937 The VM overlay is an input to no config step and no test, so it is the one compose manifest that reaches no gate.